

Bölüm 6: Liste

Veri Yapıları

Liste ADT

Iterable ve Collection

List arayüzü

Iterator ve ListIterator

ArrayList

LinkedList

Konumsal liste

Vector ve Stack

CopyOnWriteArrayList

Arrays.asList

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

Öğrenme Çıktıları

Liste ADT'si

Sıralı koleksiyonların temel işlemlerini açıklamak.

Gezinme

`Iterator` , `ListIterator` ve `Spliterator` kullanımını ayırt etmek.

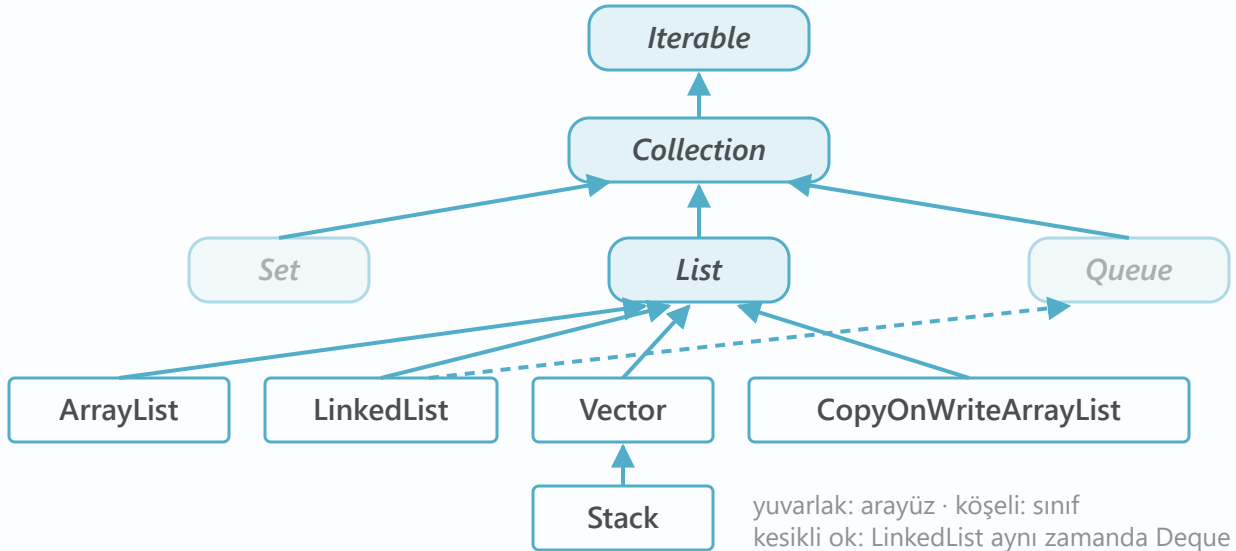
Gerçeklemeler

`ArrayList` ve `LinkedList` yapılarını karşılaştırmak.

API seçimi

İşlem gereksinimine uygun koleksiyon ve metodu seçmek.

Java Collections Framework: Liste Tarafı



- **Arayüz** (interface) ne yapılabileceğini, **sınıf** nasıl yapıldığını belirler: Liste ADT ↔ veri yapısı (Bölüm 1).
- Kodu arayüz türüyle yazmak uygulamayı değiştirmeyi kolaylaştırır: `List<String> l = new ArrayList<>();`

Iterable Arayüzü

Hiyerarşinin en üstündedir; bir nesnenin öğeleri üzerinde **gezinmeyi** sağlar. Bir nesne **Iterable** ise:

iterator()

Öğelere sırayla erişen bir **Iterator** alınabilir.

spliterator()

Öğeleri parçalara bölerek paralel işlemeyi destekleyen bir **Spliterator** alınabilir.

forEach(...)

Her öğe için bir işlem (lambda) çalıştırılabilir.

```
// yazdığımız  
for (var x : a) {  
    System.out.println(x);  
}
```

```
// derleyicinin ürettiği (a Iterable ise)  
Iterator<T> it = a.iterator();  
while (it.hasNext()) {  
    var x = it.next();  
    System.out.println(x);  
}
```

for-each döngüsü yalnızca **a** bir dizi ya da **Iterable** uygulayan bir sınıfın nesnesiyse derlenir. (Diziler için indisi döngüye çevrilir.)

Kod İzleme: Kendi Iterable Sınıfımız

```
1 class Aralik implements Iterable<Integer> {  
2     private final int bas, son;  
3     ▶ Aralik(int b, int s) { bas = b; son = s; }  
4     public Iterator<Integer> iterator() {  
5         return new Iterator<Integer>() {  
6             int k = bas;  
7             public boolean hasNext() { return k < son; }  
8             public Integer next() {  
9                 if (k >= son)  
10                    throw new NoSuchElementException();  
11                 return k++;  
12             }  
13         };  
14     }  
15 }
```



new Aralik(2, 5) · turuncu: son dönen

```
for (int x : new Aralik(2, 5)) :
```

Önce yapıcı, aralık [2, 5).

Adım 1 / 10

DEĞİŞKENLER

bas = 2

son = 5

k = -

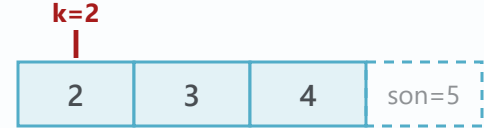
x = -

ÇIKTI



Kod İzleme: Kendi Iterable Sınıfımız

```
1 class Aralik implements Iterable<Integer> {  
2     private final int bas, son;  
3     Aralik(int b, int s) { bas = b; son = s; }  
4     ▶ public Iterator<Integer> iterator() {  
5         return new Iterator<Integer>() {  
6             ▶ int k = bas;  
7             public boolean hasNext() { return k < son; }  
8             public Integer next() {  
9                 if (k >= son)  
10                     throw new NoSuchElementException();  
11                 return k++;  
12             }  
13         };  
14     }  
15 }
```



new Aralik(2, 5) · turuncu: son dönen
for-each gizlice `iterator()` çağırır:
isimsiz iç sınıf, `k = bas`.

Adım 2 / 10

DEĞİŞKENLER

bas = 2

son = 5

k = 2

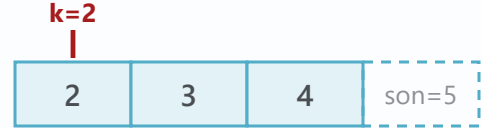
x = -

ÇIKTI



Kod İzleme: Kendi Iterable Sınıfımız

```
1 class Aralik implements Iterable<Integer> {  
2     private final int bas, son;  
3     Aralik(int b, int s) { bas = b; son = s; }  
4     public Iterator<Integer> iterator() {  
5         return new Iterator<Integer>() {  
6             int k = bas;  
7             ▶ public boolean hasNext() { return k < son; }  
8             public Integer next() {  
9                 if (k >= son)  
10                     throw new NoSuchElementException();  
11                 return k++;  
12             }  
13         };  
14     }  
15 }
```



new Aralik(2, 5) · turuncu: son dönen

hasNext() : $2 < 5 \rightarrow \text{true}$

Adım 3 / 10

DEĞİŞKENLER

bas = 2

son = 5

k = 2

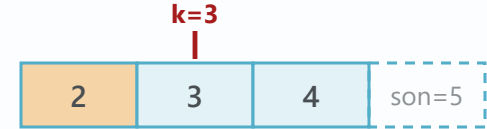
x = -

ÇIKTI



Kod İzleme: Kendi Iterable Sınıfımız

```
1 class Aralik implements Iterable<Integer> {  
2     private final int bas, son;  
3     Aralik(int b, int s) { bas = b; son = s; }  
4     public Iterator<Integer> iterator() {  
5         return new Iterator<Integer>() {  
6             int k = bas;  
7             public boolean hasNext() { return k < son; }  
8             public Integer next() {  
9                 if (k >= son)  
10                    throw new NoSuchElementException();  
11                return k++;  
12            }  
13        };  
14    }  
15 }
```



new Aralik(2, 5) · turuncu: son dönen

`next()` : 2 döner, sonra $k++ \rightarrow 3$.

Gövde `x = 2` yazar.

Adım 4 / 10

DEĞİŞKENLER

bas = 2

son = 5

k = 3

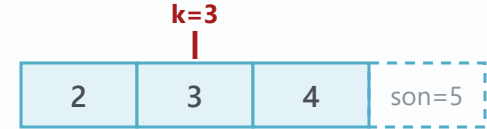
x = 2

ÇIKTI

2

Kod İzleme: Kendi Iterable Sınıfımız

```
1 class Aralik implements Iterable<Integer> {  
2     private final int bas, son;  
3     Aralik(int b, int s) { bas = b; son = s; }  
4     public Iterator<Integer> iterator() {  
5         return new Iterator<Integer>() {  
6             int k = bas;  
7             ▶ public boolean hasNext() { return k < son; }  
8             public Integer next() {  
9                 if (k >= son)  
10                     throw new NoSuchElementException();  
11                 return k++;  
12             }  
13         };  
14     }  
15 }
```



new Aralik(2, 5) · turuncu: son dönen

hasNext() : $3 < 5 \rightarrow \text{true}$

Adım 5 / 10

DEĞİŞKENLER

bas = 2

son = 5

k = 3

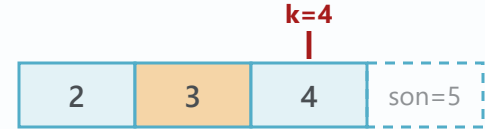
x = 2

ÇIKTI

2

Kod İzleme: Kendi Iterable Sınıfımız

```
1 class Aralik implements Iterable<Integer> {  
2     private final int bas, son;  
3     Aralik(int b, int s) { bas = b; son = s; }  
4     public Iterator<Integer> iterator() {  
5         return new Iterator<Integer>() {  
6             int k = bas;  
7             public boolean hasNext() { return k < son; }  
8             public Integer next() {  
9                 if (k >= son)  
10                    throw new NoSuchElementException();  
11                 return k++;  
12             }  
13         };  
14     }  
15 }
```



new Aralik(2, 5) · turuncu: son dönen

next() : 3 döner, sonra k++ → 4.

Gövde x = 3 yazar.

Adım 6 / 10

DEĞİŞKENLER

bas = 2

son = 5

k = 4

x = 3

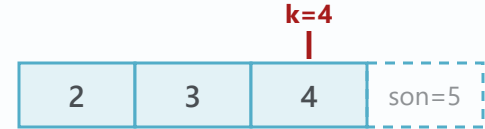
ÇIKTI

2

3

Kod İzleme: Kendi Iterable Sınıfımız

```
1 class Aralik implements Iterable<Integer> {  
2     private final int bas, son;  
3     Aralik(int b, int s) { bas = b; son = s; }  
4     public Iterator<Integer> iterator() {  
5         return new Iterator<Integer>() {  
6             int k = bas;  
7             ▶ public boolean hasNext() { return k < son; }  
8             public Integer next() {  
9                 if (k >= son)  
10                     throw new NoSuchElementException();  
11                 return k++;  
12             }  
13         };  
14     }  
15 }
```



new Aralik(2, 5) · turuncu: son dönen

hasNext() : $4 < 5 \rightarrow \text{true}$

Adım 7 / 10

DEĞİŞKENLER

bas = 2

son = 5

k = 4

x = 3

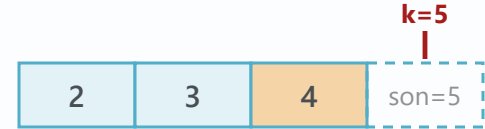
ÇIKTI

2

3

Kod İzleme: Kendi Iterable Sınıfımız

```
1 class Aralik implements Iterable<Integer> {  
2     private final int bas, son;  
3     Aralik(int b, int s) { bas = b; son = s; }  
4     public Iterator<Integer> iterator() {  
5         return new Iterator<Integer>() {  
6             int k = bas;  
7             public boolean hasNext() { return k < son; }  
8             public Integer next() {  
9                 if (k >= son)  
10                    throw new NoSuchElementException();  
11                return k++;  
12            }  
13        };  
14    }  
15 }
```



new Aralik(2, 5) · turuncu: son dönen

next() : 4 döner, sonra k++ → 5.

Gövde x = 4 yazar.

Adım 8 / 10

DEĞİŞKENLER

bas = 2

son = 5

k = 5

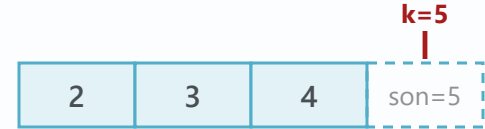
x = 4

ÇIKTI

2
3
4

Kod İzleme: Kendi Iterable Sınıfımız

```
1 class Aralik implements Iterable<Integer> {  
2     private final int bas, son;  
3     Aralik(int b, int s) { bas = b; son = s; }  
4     public Iterator<Integer> iterator() {  
5         return new Iterator<Integer>() {  
6             int k = bas;  
7             ▶ public boolean hasNext() { return k < son; }  
8             public Integer next() {  
9                 if (k >= son)  
10                     throw new NoSuchElementException();  
11                 return k++;  
12             }  
13         };  
14     }  
15 }
```



new Aralik(2, 5) · turuncu: son dönen

hasNext() : $5 < 5 \rightarrow \text{false}$; döngü biter.

Adım 9 / 10

DEĞİŞKENLER

bas = 2

son = 5

k = 5

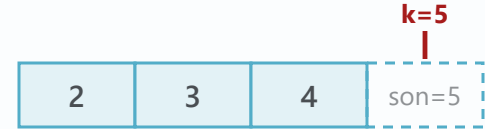
x = 4

ÇIKTI

2
3
4

Kod İzleme: Kendi Iterable Sınıfımız

```
1 class Aralik implements Iterable<Integer> {  
2     private final int bas, son;  
3     Aralik(int b, int s) { bas = b; son = s; }  
4     public Iterator<Integer> iterator() {  
5         return new Iterator<Integer>() {  
6             int k = bas;  
7             public boolean hasNext() { return k < son; }  
8             public Integer next() {  
9                 if (k >= son)  
10                    throw new NoSuchElementException();  
11                 return k++;  
12             }  
13         };  
14     }  
15 }
```



new Aralik(2, 5) · turuncu: son dönen

Sözleşme: öge yokken `next()` →

`NoSuchElementException` .

Adım 10 / 10

DEĞİŞKENLER

bas = 2

son = 5

k = 5

x = 4

ÇIKTI

2
3
4

Collection Arayüzü

Tüm koleksiyonların (liste, küme, kuyruk) ortak işlevlerini standartlaştırır:

Yetenek	Metotlar	Örnek
Öğe ekleme / çıkarma	<code>add(e)</code> , <code>remove(o)</code> , <code>clear()</code>	<code>c.add("Ali")</code> → true
Boyut ve boşluk sorgusu	<code>size()</code> , <code>isEmpty()</code>	<code>c.size()</code> → 1
Üyelik testi	<code>contains(o)</code>	<code>c.contains("Ali")</code> → true
Akış (stream) ile işleme	<code>stream()</code>	<code>c.stream().count()</code>
Diziye aktarma	<code>toArray()</code>	<code>c.toArray(new String[0])</code>
Toplu işlemler	<code>addAll</code> , <code>removeAll</code> , <code>removeIf</code>	<code>c.removeIf(s → s.isEmpty())</code>

Collection'da **indis yoktur**: `get(i)` gibi konuma dayalı işlemler List arayüzüyle gelir.

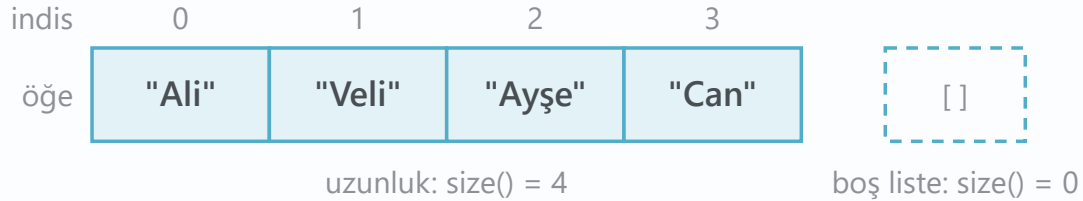
List Arayüzü

- Java Collections Framework'ün üyesidir, `java.util` paketindedir.
- **Sıralı** bir koleksiyondur (ordered collection, *sequence*): öğeler eklendikleri konumda kalır.
- Öğenin nereye ekleneceğini kullanıcı belirler; öğelere **tam sayı indisiyle** erişilir (0'dan başlar).
- Tekrarlanan öğelere ve (çoğu uygulamada) `null` 'a izin verir.
- Liste içinde arama yapılabilir: `indexOf` , `contains` .



List bir **arayüzdür**, doğrudan `new List()` yapılamaz; onu gerçekleyen bir sınıf (`ArrayList` , `LinkedList` ...) kullanılır.

Liste Kavramları



Öge (Element)

Listede depolanan veri.

İndis (Index)

Her öğenin konumunu belirten sayı: 0 ...
uzunluk - 1.

Boş Liste (Empty List)

Hiç öge içermeyen liste: `isEmpty()` → true.

Uzunluk (Length)

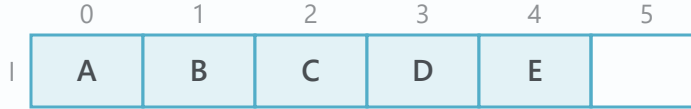
Listedeki öge sayısı: `size()`. Kapasiteyle
karıştırılmamalı.

Liste İşlemleri: ADT ve Java Karşılıkları

ADT işlemi	Anlamı	Java List metodu
Ekleme (Append)	sona yeni öge ekler	<code>add(e)</code>
Araya ekleme (Insert)	verilen konuma ekler, sonrakiler kayar	<code>add(i, e)</code>
Silme (Remove)	konumdaki ya da verilen ögeyi çıkarır	<code>remove(i)</code> , <code>remove(o)</code>
İndisleme (Indexing)	konumdaki ögeyi okur / değiştirir	<code>get(i)</code> , <code>set(i, e)</code>
Dilimleme (Slicing)	bir aralığı seçer [başlangıç, bitiş)	<code>subList(from, to)</code>
Arama (Search)	öğenin konumunu bulur	<code>indexOf(o)</code> → yoksa -1
Uzunluk (Length)	öge sayısını döndürür	<code>size()</code>

`subList` kopya değil, asıl listenin bir **görünümüdür**: `l.subList(1, 3).clear()` asıl listeden 1 ve 2 indisli öğeleri siler.

Kod İzleme: `subList` Bir Görünümdür



5 ögeli bir ArrayList.

Adım 1 / 8

```
► List<String> l = new ArrayList<>(  
►   List.of("A", "B", "C", "D", "E"));  
3 List<String> alt = l.subList(1, 3);  
4 System.out.println(al);  
5 alt.set(0, "X");  
6 alt.add("Y");  
7 alt.clear();  
8 System.out.println(l);
```

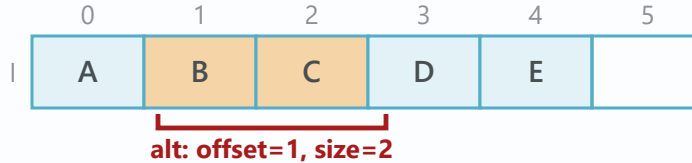
DEĞİŞKENLER

`l = [A, B, C, D, E]` `alt = -`

`offset = -` `size = -`

ÇIKTI

Kod İzleme: `subList` Bir Görünümdür



`subList(1, 3)` kopya üretmez: yalnızca **başlangıç (offset) ve uzunluk** tutan bir görünüm. `[1, 3) → B, C`.

Adım 2 / 8

```
1 List<String> l = new ArrayList<>(  
2     List.of("A", "B", "C", "D", "E"));  
▶ List<String> alt = l.subList(1, 3);  
4 System.out.println(al);  
5 alt.set(0, "X");  
6 alt.add("Y");  
7 alt.clear();  
8 System.out.println(l);
```

DEĞİŞKENLER

`l = [A, B, C, D, E]`

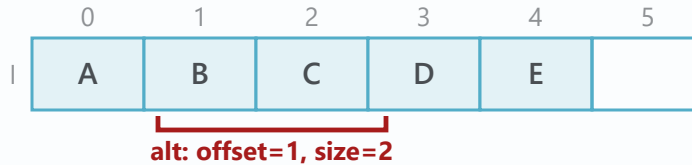
`alt = [B, C]`

`offset = 1`

`size = 2`

ÇIKTI

Kod İzleme: `subList` Bir Görünümdür



Görünüm yazdırılırken asıl listenin 1 ve 2 indisleri okunur.

Adım 3 / 8

```
1 List<String> l = new ArrayList<>(  
2     List.of("A", "B", "C", "D", "E"));  
3 List<String> alt = l.subList(1, 3);  
▶ System.out.println(alt);  
5 alt.set(0, "X");  
6 alt.add("Y");  
7 alt.clear();  
8 System.out.println(l);
```

DEĞİŞKENLER

`l` = [A, B, C, D, E]

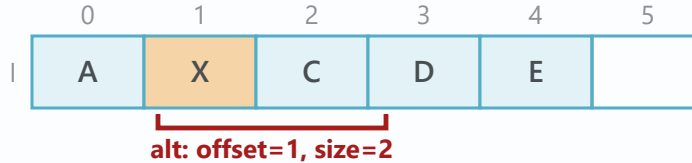
`alt` = [B, C] `offset` = 1

`size` = 2

ÇIKTI

[B, C]

Kod İzleme: `subList` Bir Görünümdür



`alt.set(0, ...)` → asıl listede `l.set(offset + 0, ...)` : `l[1] = X`.

Adım 4 / 8

```

1 List<String> l = new ArrayList<>(
2     List.of("A", "B", "C", "D", "E"));
3 List<String> alt = l.subList(1, 3);
4 System.out.println(alt);
5 ► alt.set(0, "X");
6 alt.add("Y");
7 alt.clear();
8 System.out.println(l);

```

DEĞİŞKENLER

`l = [A, X, C, D, E]`

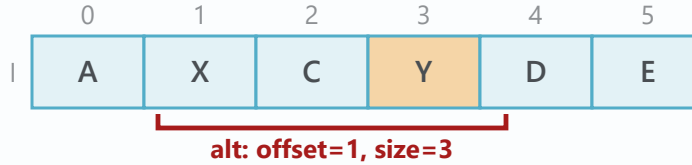
`alt = [X, C]` `offset = 1`

`size = 2`

ÇIKTI

`[B, C]`

Kod İzleme: `subList` Bir Görünümdür



`alt.add("Y")` görünümün sonuna, yani `l`'nin 3. indisine ekler; `D`, `E` sağa kayar, görünüm büyür.

Adım 5 / 8

```
1 List<String> l = new ArrayList<>(  
2     List.of("A", "B", "C", "D", "E"));  
3 List<String> alt = l.subList(1, 3);  
4 System.out.println(al);  
5 alt.set(0, "X");  
6 ► alt.add("Y");  
7 alt.clear();  
8 System.out.println(l);
```

DEĞİŞKENLER

`l` = `[A, X, C, Y, D, E]`

`alt` = `[X, C, Y]`

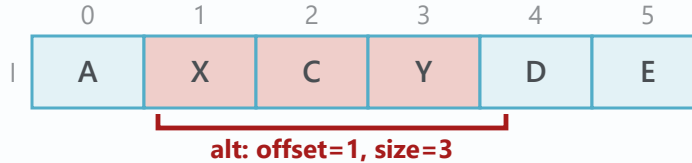
`offset` = `1`

`size` = `3`

ÇIKTI

`[B, C]`

Kod İzleme: `subList` Bir Görünümdür



`alt.clear()` görünümdeki 3 ögeyi (indis 1–3) **asıl listeden** siler...

Adım 6 / 8

```
1 List<String> l = new ArrayList<>(  
2     List.of("A", "B", "C", "D", "E"));  
3 List<String> alt = l.subList(1, 3);  
4 System.out.println(alt);  
5 alt.set(0, "X");  
6 alt.add("Y");  
7 ► alt.clear();  
8 System.out.println(l);
```

DEĞİŞKENLER

`l` = [A, X, C, Y, D, E]

`alt` = [X, C, Y]

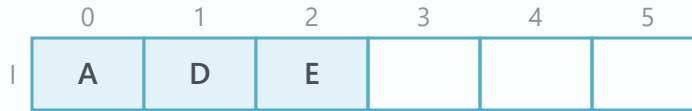
`offset` = 1

`size` = 3

ÇIKTI

[B, C]

Kod İzleme: `subList` Bir Görünümdür



alt: offset=1, size=0

...D, E sola kayar. `l.subList(a, b).clear()` bir aralığı silmenin kısa yoludur.

Adım 7 / 8

```

1 List<String> l = new ArrayList<>(
2     List.of("A", "B", "C", "D", "E"));
3 List<String> alt = l.subList(1, 3);
4 System.out.println(alt);
5 alt.set(0, "X");
6 alt.add("Y");
7 ► alt.clear();
8 System.out.println(l);

```

DEĞİŞKENLER

`l = [A, D, E]`

`alt = []`

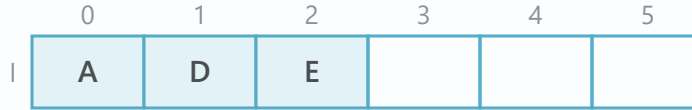
`offset = 1`

`size = 0`

ÇIKTI

`[B, C]`

Kod İzleme: subList Bir Görünümdür



alt: offset=1, size=0

Sonuç [A, D, E]. Dikkat: l doğrudan yapısal değişirse (add/remove) eski alt geçersizleşir → `ConcurrentModificationException`.

Adım 8 / 8

```

1 List<String> l = new ArrayList<>(
2     List.of("A", "B", "C", "D", "E"));
3 List<String> alt = l.subList(1, 3);
4 System.out.println(alt);
5 alt.set(0, "X");
6 alt.add("Y");
7 alt.clear();
▶ System.out.println(l);

```

DEĞİŞKENLER

l = [A, D, E]

alt = []

offset = 1

size = 0

ÇIKTI

[B, C]

[A, D, E]

Değiştirilebilir ve Değiştirilemez Listeler

Değiştirilebilir (mutable)

Oluşturulduktan sonra öge eklenebilir, silinebilir, değiştirilebilir.

```
new ArrayList<>(), new LinkedList<>()
```

Değiştirilemez (immutable)

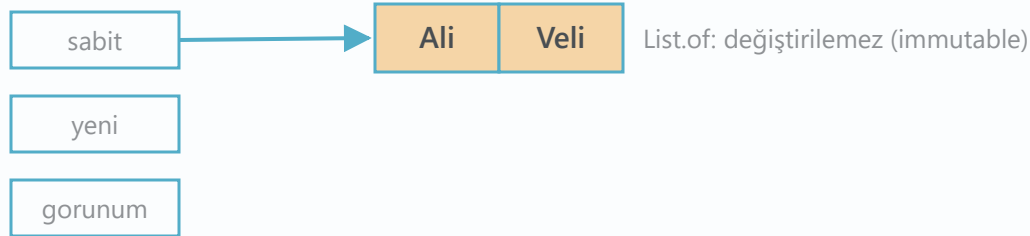
Oluşturulduktan sonra içeriği değişmez; "değişiklik" yeni bir liste üretir.

```
List.of(...), List.copyOf(...)
```

```
List<String> sabit = List.of("Ali", "Veli");  
sabit.add("Can");    // UnsupportedOperationException  
List<String> yeni = new ArrayList<>(sabit);    // değiştirilebilir kopya  
yeni.add("Can");    // [Ali, Veli, Can] – sabit hâlâ [Ali, Veli]
```

- Değiştirilemezlik fonksiyonel programlamanın temel mekanizmalarındandır: paylaşım ve paralel kullanım güvenlidir.
- **Kalıcı veri yapıları** (persistent data structures) değiştirilemez yapılardır; her "güncelleme" eski sürümü korur ve değişmeyen kısımları **paylaşarak** tam kopyadan kaçınır.
- `Collections.unmodifiableList(l)` yalnızca salt okunur bir *görünümdür*: alttaki `l` değişirse o da değişir.

Kod İzleme: Kopya mı, Görünüm mü, Değiştirilemez mi?



`List.of` değiştirilemez bir liste üretir (ve `null` kabul etmez).

Adım 1 / 8

```
► List<String> sabit = List.of("Ali", "Veli");  
2 // sabit.add("Can");           → hata  
3 List<String> yeni = new ArrayList<>(sabit);  
4 yeni.add("Can");  
5 List<String> gorunum =  
6     Collections.unmodifiableList(yeni);  
7 // gorunum.set(0, "Ece"); → hata  
8 yeni.set(0, "Ece");  
9 System.out.println(gorunum);
```

DEĞİŞKENLER

`sabit = [Ali, Veli]` `yeni = -`

`gorunum = -`

ÇIKTI

Kod İzleme: Kopya mı, Görünüm mü, Değiştirilemez mi?



Yorum açılsaydı: yapısal değişiklik yasak → `UnsupportedOperationException` (çalışma zamanı hatası, derleme değil).

Adım 2 / 8

```
1 List<String> sabit = List.of("Ali", "Veli");
2 // sabit.add("Can"); → hata
3 List<String> yeni = new ArrayList<>(sabit);
4 yeni.add("Can");
5 List<String> gorunum =
6     Collections.unmodifiableList(yeni);
7 // gorunum.set(0, "Ece"); → hata
8 yeni.set(0, "Ece");
9 System.out.println(gorunum);
```

DEĞİŞKENLER

sabit = [Ali, Veli] yeni = -

gorunum = -

ÇIKTI

Kod İzleme: Kopya mı, Görünüm mü, Değiştirilemez mi?



`new ArrayList<>(sabit)` öğeleri **yeni bir diziye kopyalar**; artık iki ayrı nesne var.

Adım 3 / 8

```
1 List<String> sabit = List.of("Ali", "Veli");
2 // sabit.add("Can");           → hata
3 ► List<String> yeni = new ArrayList<>(sabit);
4 yeni.add("Can");
5 List<String> gorunum =
6     Collections.unmodifiableList(yeni);
7 // gorunum.set(0, "Ece"); → hata
8 yeni.set(0, "Ece");
9 System.out.println(gorunum);
```

DEĞİŞKENLER

sabit = [Ali, Veli]

yeni = [Ali, Veli]

gorunum = -

ÇIKTI

[Empty output box]

Kod İzleme: Kopya mı, Görünüm mü, Değiştirilemez mi?



Kopya değiştirilebilir: Can eklenir. `sabit` hâlâ [Ali, Veli].

Adım 4 / 8

```
1 List<String> sabit = List.of("Ali", "Veli");
2 // sabit.add("Can");           → hata
3 List<String> yeni = new ArrayList<>(sabit);
4 ► yeni.add("Can");
5 List<String> gorunum =
6     Collections.unmodifiableList(yeni);
7 // gorunum.set(0, "Ece"); → hata
8 yeni.set(0, "Ece");
9 System.out.println(gorunum);
```

DEĞİŞKENLER

`sabit = [Ali, Veli]`

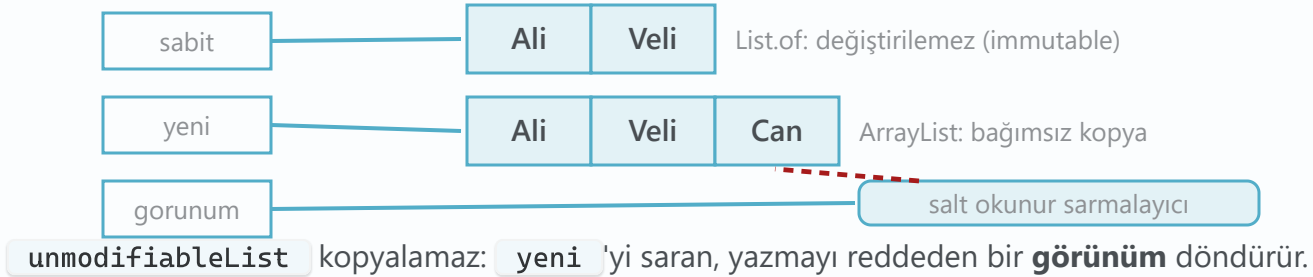
`yeni = [Ali, Veli, Can]`

`gorunum = -`

ÇIKTI

[Empty output box]

Kod İzleme: Kopya mı, Görünüm mü, Değiştirilemez mi?



Adım 5 / 8

```

1 List<String> sabit = List.of("Ali", "Veli");
2 // sabit.add("Can");           → hata
3 List<String> yeni = new ArrayList<>(sabit);
4 yeni.add("Can");
5 ▶ List<String> gorunum =
6   ▶ Collections.unmodifiableList(yeni);
7 // gorunum.set(0, "Ece"); → hata
8 yeni.set(0, "Ece");
9 System.out.println(gorunum);

```

DEĞİŞKENLER

sabit = [Ali, Veli]

yeni = [Ali, Veli, Can]

gorunum = [Ali, Veli, Can]

ÇIKTI

Kod İzleme: Kopya mı, Görünüm mü, Değiştirilemez mi?



Yorum açılsaydı: görünüm üzerinden yazmak yasak → `UnsupportedOperationException`.

Adım 6 / 8

```
1 List<String> sabit = List.of("Ali", "Veli");
2 // sabit.add("Can");           → hata
3 List<String> yeni = new ArrayList<>(sabit);
4 yeni.add("Can");
5 List<String> gorunum =
6     Collections.unmodifiableList(yeni);
7 // gorunum.set(0, "Ece"); → hata
8 yeni.set(0, "Ece");
9 System.out.println(gorunum);
```

DEĞİŞKENLER

sabit = [Ali, Veli]

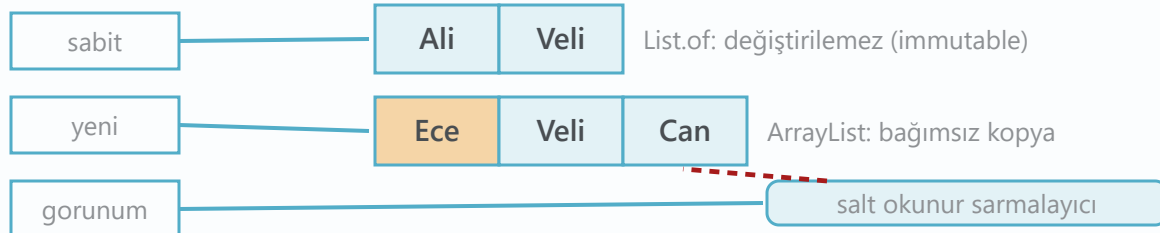
yeni = [Ali, Veli, Can]

gorunum = [Ali, Veli, Can]

ÇIKTI

[Ali, Veli, Can]

Kod İzleme: Kopya mı, Görünüm mü, Değiştirilemez mi?



Ama alttaki **yeni** değişebilir: görünüm aynı diziye baktığından değişikliği görür.

Adım 7 / 8

```
1 List<String> sabit = List.of("Ali", "Veli");
2 // sabit.add("Can"); → hata
3 List<String> yeni = new ArrayList<>(sabit);
4 yeni.add("Can");
5 List<String> gorunum =
6     Collections.unmodifiableList(yeni);
7 // gorunum.set(0, "Ece"); → hata
8 ► yeni.set(0, "Ece");
9 System.out.println(gorunum);
```

DEĞİŞKENLER

sabit = [Ali, Veli]

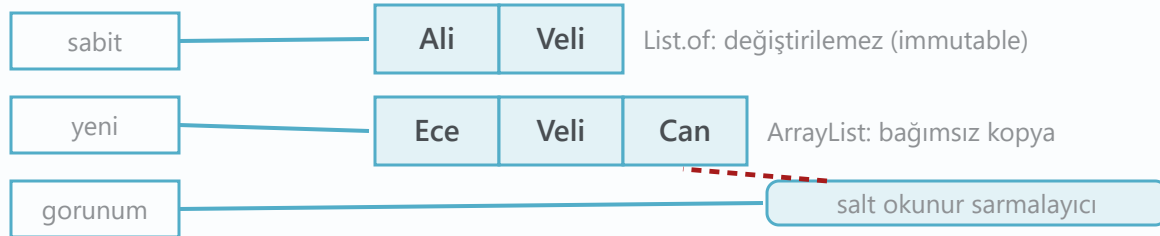
yeni = [Ece, Veli, Can]

gorunum = [Ece, Veli, Can]

ÇIKTI

[Ece, Veli, Can]

Kod İzleme: Kopya mı, Görünüm mü, Değiştirilemez mi?



Salt okunur $\neq$ değiştirilemez. Gerçek değiştirilemezlik için `List.copyOf(yeni)` kullanılır.

Adım 8 / 8

```
1 List<String> sabit = List.of("Ali", "Veli");
2 // sabit.add("Can");           → hata
3 List<String> yeni = new ArrayList<>(sabit);
4 yeni.add("Can");
5 List<String> gorunum =
6     Collections.unmodifiableList(yeni);
7 // gorunum.set(0, "Ece"); → hata
8 yeni.set(0, "Ece");
▶ System.out.println(gorunum);
```

DEĞİŞKENLER

`sabit = [Ali, Veli]`

`yeni = [Ece, Veli, Can]`

`gorunum = [Ece, Veli, Can]`

ÇIKTI

`[Ece, Veli, Can]`

List Arayüzünün Ana Metotları

Grup	Metot	Açıklama
Ekleme	<code>add(E e)</code>	sona ekler, <code>true</code> döner
	<code>add(int index, E e)</code>	verilen indise ekler; sonrakiler bir sağa kayar
Çıkarma	<code>remove(Object o)</code>	öğenin ilk geçtiği yeri siler; bulduysa <code>true</code>
	<code>remove(int index)</code>	indisteki öğeyi siler ve döndürür
Erişim	<code>get(int index)</code>	indisteki öğeyi döndürür
	<code>set(int index, E e)</code>	öğeyi değiştirir, eskisini döndürür
Uzunluk	<code>size()</code>	öge sayısı

Geçersiz indis ($\text{index} < 0$ ya da $\text{index} \geq \text{size}()$; add için $> \text{size}()$) → `IndexOutOfBoundsException`. Liste öğeleri döngülerle gezilebilir (→ Gezinme).

ListDemo: Tekrarlanan Öğeler ve null

```
public class ListDemo {    // import java.util.ArrayList; import java.util.List;
    public static void main(String[] args) {
        List<String> isimler = new ArrayList<>();
        isimler.add("Ali");
        isimler.add("Ali");           // tekrar: izin var
        isimler.add(null);           // null: izin var
        isimler.add("Ali");
        System.out.println(isimler);    // [Ali, Ali, null, Ali]
        System.out.println(isimler.size());    // 4
        System.out.println(isimler.indexOf("Ali"));    // 0 (ilk geçiş)
        System.out.println(isimler.lastIndexOf("Ali"));    // 3
    }
}
```

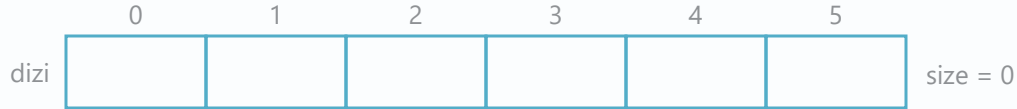
Liste ≠ Küme

Liste tekrarlara izin verir; tekil öğeler için küme (Set) kullanılır (Bölüm 12).

Dikkat

`List.of( ... )` ile oluşturulan liste `null` kabul etmez.

Kod İzleme: Tekrar, null ve Arama



Boş ArrayList.

Adım 1 / 12

```
► List<String> isimler = new ArrayList<>();  
2 isimler.add("Ali");  
3 isimler.add("Ali");      // tekrar: izin var  
4 isimler.add(null);       // null: izin var  
5 isimler.add("Ali");  
6 System.out.println(isimler);  
7 System.out.println(isimler.size());  
8 System.out.println(isimler.indexOf("Ali"));  
9 System.out.println(isimler.lastIndexOf("Ali"));  
10 System.out.println(isimler.indexOf(null));
```

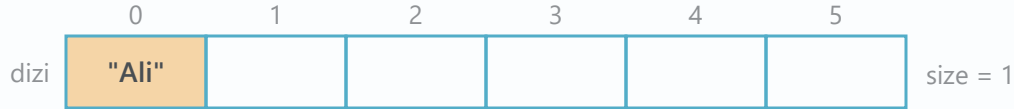
DEĞİŞKENLER

isimler = []

size = 0

ÇIKTI

Kod İzleme: Tekrar, null ve Arama



Sona eklenir: indis 0.

Adım 2 / 12

```
1 List<String> isimler = new ArrayList<>();
2 ▶ isimler.add("Ali");
3 isimler.add("Ali");           // tekrar: izin var
4 isimler.add(null);           // null: izin var
5 isimler.add("Ali");
6 System.out.println(isimler);
7 System.out.println(isimler.size());
8 System.out.println(isimler.indexOf("Ali"));
9 System.out.println(isimler.lastIndexOf("Ali"));
10 System.out.println(isimler.indexOf(null));
```

DEĞİŞKENLER

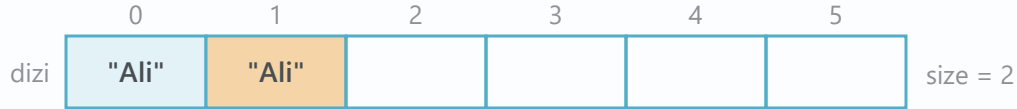
isimler = [Ali]

size = 1

ÇIKTI

[Empty output area]

Kod İzleme: Tekrar, null ve Arama



Aynı değer tekrar eklenebilir: liste (küme değil) **tekrara izin verir**.

Adım 3 / 12

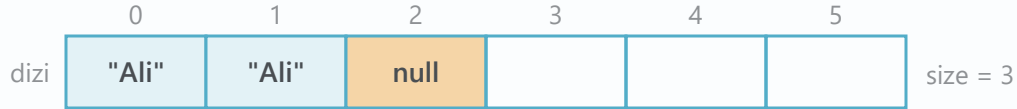
```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 ► isimler.add("Ali"); // tekrar: izin var
4 isimler.add(null); // null: izin var
5 isimler.add("Ali");
6 System.out.println(isimler);
7 System.out.println(isimler.size());
8 System.out.println(isimler.indexOf("Ali"));
9 System.out.println(isimler.lastIndexOf("Ali"));
10 System.out.println(isimler.indexOf(null));
```

DEĞİŞKENLER

isimler = [Ali, Ali] size = 2

ÇIKTI

Kod İzleme: Tekrar, null ve Arama



ArrayList **null** ögeyi de saklar.

Adım 4 / 12

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ali"); // tekrar: izin var
4 ▶ isimler.add(null); // null: izin var
5 isimler.add("Ali");
6 System.out.println(isimler);
7 System.out.println(isimler.size());
8 System.out.println(isimler.indexOf("Ali"));
9 System.out.println(isimler.lastIndexOf("Ali"));
10 System.out.println(isimler.indexOf(null));
```

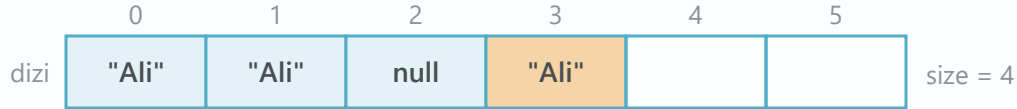
DEĞİŞKENLER

isimler = [Ali, Ali, null]

size = 3

ÇIKTI

Kod İzleme: Tekrar, null ve Arama



Üçüncü "Ali" indisi 3'e.

Adım 5 / 12

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ali"); // tekrar: izin var
4 isimler.add(null); // null: izin var
5 ▶ isimler.add("Ali");
6 System.out.println(isimler);
7 System.out.println(isimler.size());
8 System.out.println(isimler.indexOf("Ali"));
9 System.out.println(isimler.lastIndexOf("Ali"));
10 System.out.println(isimler.indexOf(null));
```

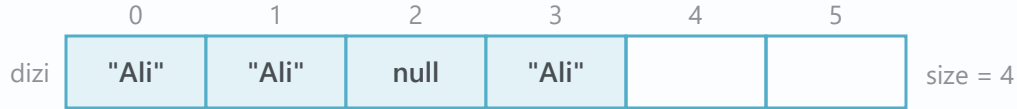
DEĞİŞKENLER

isimler = [Ali, Ali, null, Ali]

size = 4

ÇIKTI

Kod İzleme: Tekrar, null ve Arama



`toString()` öğeleri sırasıyla, köşeli parantez içinde yazar.

Adım 6 / 12

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ali");           // tekrar: izin var
4 isimler.add(null);           // null: izin var
5 isimler.add("Ali");
6 ▶ System.out.println(isimler);
7 System.out.println(isimler.size());
8 System.out.println(isimler.indexOf("Ali"));
9 System.out.println(isimler.lastIndexOf("Ali"));
10 System.out.println(isimler.indexOf(null));
```

DEĞİŞKENLER

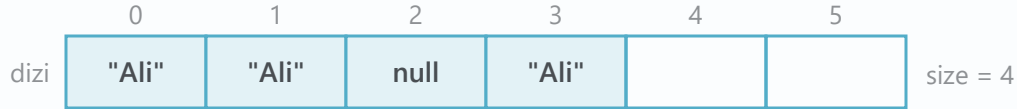
`isimler = [Ali, Ali, null, Ali]`

`size = 4`

ÇIKTI

`[Ali, Ali, null, Ali]`

Kod İzleme: Tekrar, null ve Arama



`size()` : öge sayısı (kapasite değil).

Adım 7 / 12

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ali");           // tekrar: izin var
4 isimler.add(null);           // null: izin var
5 isimler.add("Ali");
6 System.out.println(isimler);
7 ► System.out.println(isimler.size());
8 System.out.println(isimler.indexOf("Ali"));
9 System.out.println(isimler.lastIndexOf("Ali"));
10 System.out.println(isimler.indexOf(null));
```

DEĞİŞKENLER

`isimler = [Ali, Ali, null, Ali]`

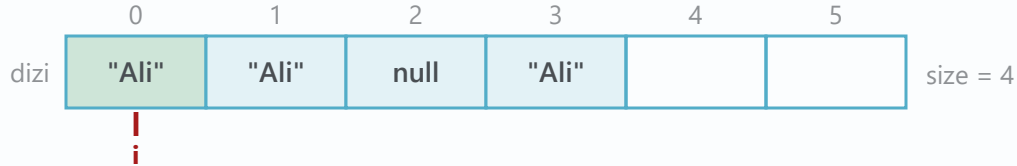
`size = 4`

ÇIKTI

[Ali, Ali, null, Ali]

4

Kod İzleme: Tekrar, null ve Arama



`indexOf` baştan tarar, **ilk** eşleşmede durur: `"Ali".equals(dizi[0])` → 0.

Adım 8 / 12

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ali");           // tekrar: izin var
4 isimler.add(null);           // null: izin var
5 isimler.add("Ali");
6 System.out.println(isimler);
7 System.out.println(isimler.size());
8 ► System.out.println(isimler.indexOf("Ali"));
9 System.out.println(isimler.lastIndexOf("Ali"));
10 System.out.println(isimler.indexOf(null));
```

DEĞİŞKENLER

`isimler = [Ali, Ali, null, Ali]`

`size = 4`

`i = 0`

`sonuc = 0`

ÇIKTI

`[Ali, Ali, null, Ali]`

`4`

`0`

Kod İzleme: Tekrar, null ve Arama



`lastIndexOf` sondan tarar: ilk bakılan indis 3 eşleşir → 3.

Adım 9 / 12

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ali");           // tekrar: izin var
4 isimler.add(null);           // null: izin var
5 isimler.add("Ali");
6 System.out.println(isimler);
7 System.out.println(isimler.size());
8 System.out.println(isimler.indexOf("Ali"));
9 ► System.out.println(isimler.lastIndexOf("Ali"));
10 System.out.println(isimler.indexOf(null));
```

DEĞİŞKENLER

`isimler = [Ali, Ali, null, Ali]`

`size = 4`

`i = 3`

`sonuc = 3`

ÇIKTI

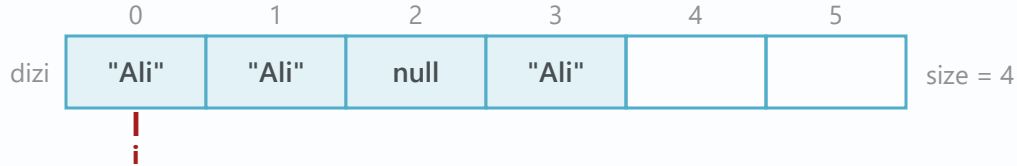
[Ali, Ali, null, Ali]

4

0

3

Kod İzleme: Tekrar, null ve Arama



`indexOf(null)` : aranan null olduğundan `equals` yerine `==` kullanılır. `dizi[0] == null?` hayır.

Adım 10 / 12

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ali");           // tekrar: izin var
4 isimler.add(null);           // null: izin var
5 isimler.add("Ali");
6 System.out.println(isimler);
7 System.out.println(isimler.size());
8 System.out.println(isimler.indexOf("Ali"));
9 System.out.println(isimler.lastIndexOf("Ali"));
▶ System.out.println(isimler.indexOf(null));
```

DEĞİŞKENLER

`isimler = [Ali, Ali, null, Ali]`

`size = 4`

`i = 0`

`sonuc = -`

ÇIKTI

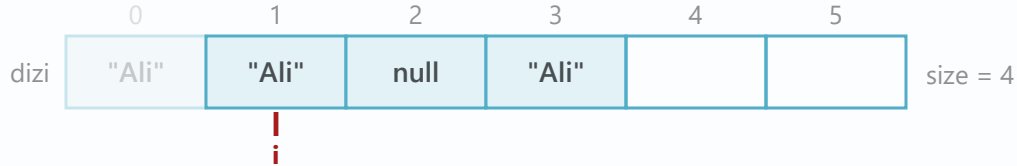
[Ali, Ali, null, Ali]

4

0

3

Kod İzleme: Tekrar, null ve Arama



`dizi[1] == null?` hayır.

Adım 11 / 12

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ali");           // tekrar: izin var
4 isimler.add(null);           // null: izin var
5 isimler.add("Ali");
6 System.out.println(isimler);
7 System.out.println(isimler.size());
8 System.out.println(isimler.indexOf("Ali"));
9 System.out.println(isimler.lastIndexOf("Ali"));
▶ System.out.println(isimler.indexOf(null));
```

DEĞİŞKENLER

`isimler = [Ali, Ali, null, Ali]`

`size = 4` `i = 1` `sonuc = -`

ÇIKTI

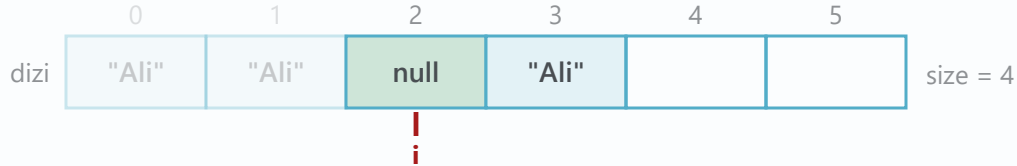
`[Ali, Ali, null, Ali]`

`4`

`0`

`3`

Kod İzleme: Tekrar, null ve Arama



dizi[2] == null? **evet** → 2. (null.equals çağrılıysaydı NullPointerException olurdu.)

Adım 12 / 12

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ali");           // tekrar: izin var
4 isimler.add(null);           // null: izin var
5 isimler.add("Ali");
6 System.out.println(isimler);
7 System.out.println(isimler.size());
8 System.out.println(isimler.indexOf("Ali"));
9 System.out.println(isimler.lastIndexOf("Ali"));
▶ System.out.println(isimler.indexOf(null));
```

DEĞİŞKENLER

isimler = [Ali, Ali, null, Ali]

size = 4

i = 2

sonuc = 2

ÇIKTI

[Ali, Ali, null, Ali]

4

0

3

2

Tuzak: `remove(int)` mi, `remove(Object)` mu?

Liste `Integer` tuttuğunda iki metot karışabilir: `int` argüman **indis** olarak yorumlanır.

```
List<Integer> sayilar = new ArrayList<>(List.of(10, 20, 30));
sayilar.remove(1); // indis 1 silinir → [10, 30]
sayilar.remove(Integer.valueOf(10)); // 10 değeri silinir → [30]
sayilar.remove(Integer.valueOf(99)); // yok → false, liste değişmez
sayilar.remove(5); // IndexOutOfBoundsException
```

`remove(int index)`

Silinen öğeyi döndürür.

E `remove(int)`

`remove(Object o)`

Silindi mi, onu döndürür.

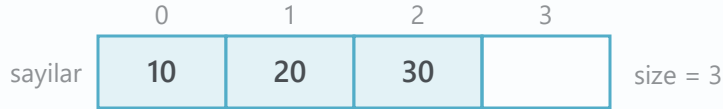
`boolean remove(Object)`

Maliyet

İkisi de dizi tabanlı listede sonraki öğeleri kaydırır.

$O(n)$

Kod İzleme: Hangi `remove` Çağrılıyor?



[10, 20, 30]. `List<Integer>` 'da iki `remove` aşırı yüklemesi (overload) de uygulanabilir.

Adım 1 / 7

```
► List<Integer> sayilar =  
► new ArrayList<>(List.of(10, 20, 30));  
3 Integer a = sayilar.remove(1);  
4 boolean b = sayilar.remove(Integer.valueOf(10));  
5 boolean c = sayilar.remove(Integer.valueOf(99));  
6 sayilar.remove(5); // IndexOutOfBoundsException
```

DEĞİŞKENLER

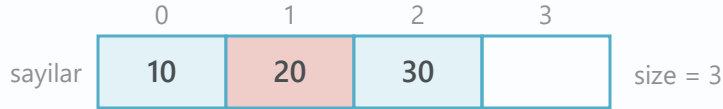
sayilar = [10, 20, 30]

a = -

b = -

c = -

Kod İzleme: Hangi `remove` Çağrılıyor?



Argüman `int` : derleyici kutulama gerektirmeyen `remove(int index)` 'i seçer. Silinecek: indis 1.

Adım 2 / 7

```
1 List<Integer> sayilar =  
2   new ArrayList<>(List.of(10, 20, 30));  
▶ Integer a = sayilar.remove(1);  
4 boolean b = sayilar.remove(Integer.valueOf(10));  
5 boolean c = sayilar.remove(Integer.valueOf(99));  
6 sayilar.remove(5);    // IndexOutOfBoundsException
```

DEĞİŞKENLER

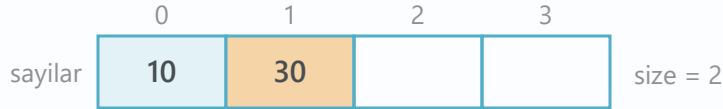
sayilar = [10, 20, 30]

a = -

b = -

c = -

Kod İzleme: Hangi `remove` Çağrılıyor?



30 bir sola kayar; silinen öge **döndürülür**: a = 20.

Adım 3 / 7

```
1 List<Integer> sayilar =  
2     new ArrayList<>(List.of(10, 20, 30));  
3     ▶ Integer a = sayilar.remove(1);  
4     boolean b = sayilar.remove(Integer.valueOf(10));  
5     boolean c = sayilar.remove(Integer.valueOf(99));  
6     sayilar.remove(5);    // IndexOutOfBoundsException
```

DEĞİŞKENLER

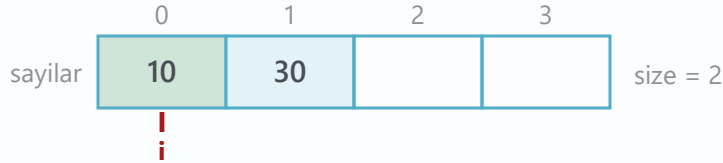
sayilar = [10, 30]

a = 20

b = -

c = -

Kod İzleme: Hangi `remove` Çağrılıyor?



`Integer` argüman → `remove(Object o)` : değ r aranır. `Integer.valueOf(10).equals(10)` → eşleşme indisi 0'da.

Adım 4 / 7

```
1 List<Integer> sayilar =  
2     new ArrayList<>(List.of(10, 20, 30));  
3 Integer a = sayilar.remove(1);  
▶ boolean b = sayilar.remove(Integer.valueOf(10));  
5 boolean c = sayilar.remove(Integer.valueOf(99));  
6 sayilar.remove(5);    // IndexOutOfBoundsException
```

DEĞİŞKENLER

`sayilar = [10, 30]` `a = 20`

`b = -`

`c = -`

Kod İzleme: Hangi `remove` Çağrılıyor?



İlk eşleşme silinir, 30 kayar; dönüş **boolean**: `b = true`.

Adım 5 / 7

```
1 List<Integer> sayilar =  
2     new ArrayList<>(List.of(10, 20, 30));  
3 Integer a = sayilar.remove(1);  
4 ▶ boolean b = sayilar.remove(Integer.valueOf(10));  
5 boolean c = sayilar.remove(Integer.valueOf(99));  
6 sayilar.remove(5);    // IndexOutOfBoundsException
```

DEĞİŞKENLER

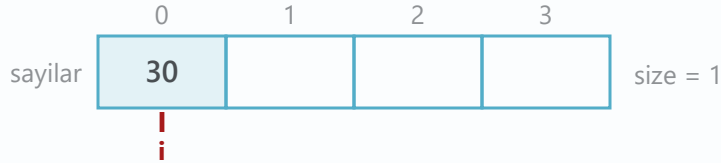
sayilar = [30]

a = 20

b = true

c = -

Kod İzleme: Hangi `remove` Çağrılıyor?



99 aranır: $30 \neq 99$, liste bitti $\rightarrow$ bulunamadı, liste değişmez, $c = \text{false}$.

Adım 6 / 7

```
1 List<Integer> sayilar =  
2     new ArrayList<>(List.of(10, 20, 30));  
3 Integer a = sayilar.remove(1);  
4 boolean b = sayilar.remove(Integer.valueOf(10));  
▶ boolean c = sayilar.remove(Integer.valueOf(99));  
6 sayilar.remove(5);    // IndexOutOfBoundsException
```

DEĞİŞKENLER

sayilar = [30]

a = 20

b = true

c = false

Kod İzleme: Hangi `remove` Çağrılıyor?



`remove(5)` : indis kontrolü $5 < \text{size} (1)$ başarısız → `IndexOutOfBoundsException` : "Index 5 out of bounds for length 1".

Adım 7 / 7

```
1 List<Integer> sayilar =  
2     new ArrayList<>(List.of(10, 20, 30));  
3 Integer a = sayilar.remove(1);  
4 boolean b = sayilar.remove(Integer.valueOf(10));  
5 boolean c = sayilar.remove(Integer.valueOf(99));  
▶ sayilar.remove(5);    // IndexOutOfBoundsException
```

DEĞİŞKENLER

sayilar = [30]

a = 20

b = true

c = false

Lambda ile Toplu İşlem: `replaceAll` ve `sort`

`replaceAll`

```
List<String> isimler =  
    new ArrayList<>();  
isimler.add("Ali");  
isimler.add("Ahmet");  
isimler.add("Mehmet");  
isimler.replaceAll(  
    isim -> isim.toUpperCase());  
System.out.println(isimler);  
// [ALI, AHMET, MEHMET]
```

Her öğe, fonksiyonun sonucuyla **yerinde** değiştirilir.

`sort`

```
List<Integer> sayilar =  
    new ArrayList<>();  
sayilar.add(5);  
sayilar.add(3);  
sayilar.add(1);  
sayilar.sort((s1, s2) ->  
    Integer.compare(s1, s2));  
System.out.println(sayilar);  
// [1, 3, 5]
```

Kısaca: `sayilar.sort(null)` ya da
`Comparator.naturalOrder()` ; ters sıra:
`Comparator.reverseOrder()` .

Not: `toUpperCase()` Türkçe yerelde "Ali"yi "ALİ" yapar; yerelden bağımsız sonuç için `toUpperCase(Locale.ROOT)` .

Kod İzleme: `replaceAll` ve `sort`

	0	1	2
isimler	Ali	Ahmet	Mehmet

Üç isimli bir ArrayList.

Adım 1 / 9

```
▶ List<String> isimler = new ArrayList<>(  
▶   List.of("Ali", "Ahmet", "Mehmet"));  
3 isimler.replaceAll(  
4   isim -> isim.toUpperCase(Locale.ROOT));  
5 List<Integer> sayilar = new ArrayList<>(  
6   List.of(5, 3, 1));  
7 sayilar.sort((s1, s2) -> Integer.compare(s1, s2));  
8 System.out.println(isimler + " " + sayilar);
```

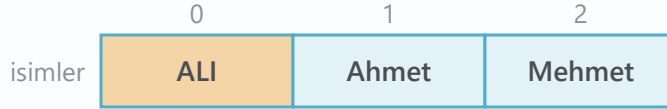
DEĞİŞKENLER

isimler = [Ali, Ahmet, Mehmet]

sayilar = -

ÇIKTI

Kod İzleme: `replaceAll` ve `sort`



`replaceAll` her indis için lambdayı çağırır ve sonucu **aynı yere** yazar: `dizi[0] = "Ali"`.

Adım 2 / 9

```
1 List<String> isimler = new ArrayList<>(  
2     List.of("Ali", "Ahmet", "Mehmet"));  
▶ isimler.replaceAll(  
▶     isim -> isim.toUpperCase(Locale.ROOT));  
5 List<Integer> sayilar = new ArrayList<>(  
6     List.of(5, 3, 1));  
7 sayilar.sort((s1, s2) -> Integer.compare(s1, s2));  
8 System.out.println(isimler + " " + sayilar);
```

DEĞİŞKENLER

`isimler = [ALI, Ahmet, Mehmet]`

`sayilar = -`

`isim = "Ali"`

ÇIKTI

Kod İzleme: `replaceAll` ve `sort`

	0	1	2
isimler	ALI	AHMET	Mehmet

`replaceAll` her indis için lambdayı çağırır ve sonucu **aynı yere** yazar: `dizi[1] = "AHMET"`.

Adım 3 / 9

```
1 List<String> isimler = new ArrayList<>(  
2     List.of("Ali", "Ahmet", "Mehmet"));  
▶ isimler.replaceAll(  
▶     isim -> isim.toUpperCase(Locale.ROOT));  
5 List<Integer> sayilar = new ArrayList<>(  
6     List.of(5, 3, 1));  
7 sayilar.sort((s1, s2) -> Integer.compare(s1, s2));  
8 System.out.println(isimler + " " + sayilar);
```

DEĞİŞKENLER

`isimler = [ALI, AHMET, Mehmet]`

`sayilar = -`

`isim = "Ahmet"`

ÇIKTI

Kod İzleme: `replaceAll` ve `sort`

	0	1	2
isimler	ALI	AHMET	MEHMET

`replaceAll` her indis için lambdayı çağırır ve sonucu **aynı yere** yazar: `dizi[2] = "MEHMET"`.

Adım 4 / 9

```
1 List<String> isimler = new ArrayList<>(  
2     List.of("Ali", "Ahmet", "Mehmet"));  
▶ isimler.replaceAll(  
▶     isim -> isim.toUpperCase(Locale.ROOT));  
5 List<Integer> sayilar = new ArrayList<>(  
6     List.of(5, 3, 1));  
7 sayilar.sort((s1, s2) -> Integer.compare(s1, s2));  
8 System.out.println(isimler + " " + sayilar);
```

DEĞİŞKENLER

`isimler = [ALI, AHMET, MEHMET]`

`sayilar = -`

`isim = "Mehmet"`

ÇIKTI

Kod İzleme: `replaceAll` ve `sort`

	0	1	2
isimler	ALI	AHMET	MEHMET
sayılar	5	3	1

[5, 3, 1]. `sort` Java'da TimSort kullanır; karşılaştırmayı bizim lambdamız yapar.

Adım 5 / 9

```
1 List<String> isimler = new ArrayList<>(  
2     List.of("Ali", "Ahmet", "Mehmet"));  
3 isimler.replaceAll(  
4     isim -> isim.toUpperCase(Locale.ROOT));  
▶ List<Integer> sayilar = new ArrayList<>(  
▶     List.of(5, 3, 1));  
7 sayilar.sort((s1, s2) -> Integer.compare(s1, s2));  
8 System.out.println(isimler + " " + sayilar);
```

DEĞİŞKENLER

`isimler = [ALI, AHMET, MEHMET]`

`sayilar = [5, 3, 1]`

ÇIKTI

Kod İzleme: `replaceAll` ve `sort`

	0	1	2
isimler	ALI	AHMET	MEHMET
sayılar	5	3	1

İlk çağrı `compare(3, 5)` → -1: dizinin başı **azalan** bir dizi (run) başlıyor.

Adım 6 / 9

```
1 List<String> isimler = new ArrayList<>(  
2     List.of("Ali", "Ahmet", "Mehmet"));  
3 isimler.replaceAll(  
4     isim -> isim.toUpperCase(Locale.ROOT));  
5 List<Integer> sayilar = new ArrayList<>(  
6     List.of(5, 3, 1));  
▶ sayilar.sort((s1, s2) -> Integer.compare(s1, s2));  
8 System.out.println(isimler + " " + sayilar);
```

DEĞİŞKENLER

`isimler = [ALI, AHMET, MEHMET]`

`sayilar = [5, 3, 1]` `s1 = 3`

`s2 = 5` `sonuc = -1`

ÇIKTI

Kod İzleme: `replaceAll` ve `sort`

	0	1	2
isimler	ALI	AHMET	MEHMET
sayilar	5	3	1

`compare(1, 3)` → -1: azalan dizi sürüyor; run uzunluğu 3.

Adım 7 / 9

```
1 List<String> isimler = new ArrayList<>(  
2     List.of("Ali", "Ahmet", "Mehmet"));  
3 isimler.replaceAll(  
4     isim -> isim.toUpperCase(Locale.ROOT));  
5 List<Integer> sayilar = new ArrayList<>(  
6     List.of(5, 3, 1));  
▶ sayilar.sort((s1, s2) -> Integer.compare(s1, s2));  
8 System.out.println(isimler + " " + sayilar);
```

DEĞİŞKENLER

`isimler = [ALI, AHMET, MEHMET]`

`sayilar = [5, 3, 1]` `s1 = 1`

`s2 = 3`

`sonuc = -1`

ÇIKTI

Kod İzleme: replaceAll ve sort

	0	1	2
isimler	ALI	AHMET	MEHMET
sayılar	1	3	5

Tüm dizi tek bir azalan run: TimSort onu **ters çevirir** → [1, 3, 5]. Yalnızca 2 karşılaştırma.

Adım 8 / 9

```
1 List<String> isimler = new ArrayList<>(  
2     List.of("Ali", "Ahmet", "Mehmet"));  
3 isimler.replaceAll(  
4     isim -> isim.toUpperCase(Locale.ROOT));  
5 List<Integer> sayilar = new ArrayList<>(  
6     List.of(5, 3, 1));  
▶ sayilar.sort((s1, s2) -> Integer.compare(s1, s2));  
8 System.out.println(isimler + " " + sayilar);
```

DEĞİŞKENLER

isimler = [ALI, AHMET, MEHMET]

sayilar = [1, 3, 5] s1 = -

s2 = - sonuc = -

ÇIKTI

Kod İzleme: `replaceAll` ve `sort`

	0	1	2
isimler	ALI	AHMET	MEHMET
sayilar	1	3	5

`Locale.ROOT` sayesinde "Ali" → "ALI" (Türkçe yerelde "ALİ" olurdu).

Adım 9 / 9

```
1 List<String> isimler = new ArrayList<>(  
2     List.of("Ali", "Ahmet", "Mehmet"));  
3 isimler.replaceAll(  
4     isim -> isim.toUpperCase(Locale.ROOT));  
5 List<Integer> sayilar = new ArrayList<>(  
6     List.of(5, 3, 1));  
7 sayilar.sort((s1, s2) -> Integer.compare(s1, s2));  
▶ System.out.println(isimler + " " + sayilar);
```

DEĞİŞKENLER

`isimler = [ALI, AHMET, MEHMET]`

`sayilar = [1, 3, 5]`

ÇIKTI

`[ALI, AHMET, MEHMET] [1, 3, 5]`

İndisle Erişim ve Jenerik Tipler (Generics)

```
List<String> isimler =  
    new ArrayList<>();  
isimler.add("Ali");  
isimler.add("Ahmet");  
isimler.add("Mehmet");  
String ilkIsim = isimler.get(0);  
System.out.println(ilkIsim); // Ali  
isimler.get(3); // IndexOutOfBoundsException
```

```
List<String> isimler =  
    new ArrayList<>();  
isimler.add("Ali");  
isimler.add(5); // derleme hatası  
  
List<Integer> sayilar =  
    new ArrayList<>();  
sayilar.add(5); // otomatik kutulama  
int x = sayilar.get(0); // kutudan çıkarma
```

- İndisler 0'dan başlar. `<String>` tür parametresi sayesinde yanlış tür **derleme zamanında** yakalanır, `get` sonucu dönüştürme (cast) gerektirmez.
- Tür parametresi ilkel olamaz: `List<int>` yazılamaz, `List<Integer>` kullanılır (autoboxing).

Kod İzleme: `get`, Jenerikler ve Kutulama



Tür parametresi `<String>` : bu listeye yalnızca String girebilir (derleme zamanı denetimi).

Adım 1 / 10

```
► List<String> isimler = new ArrayList<>();  
2 isimler.add("Ali");  
3 isimler.add("Ahmet");  
4 isimler.add("Mehmet");  
5 String ilkIsim = isimler.get(0);  
6 System.out.println(ilkIsim);  
7 List<Integer> sayilar = new ArrayList<>();  
8 sayilar.add(5);           // Integer.valueOf(5)  
9 int x = sayilar.get(0);   // .intValue()  
10 isimler.get(3);          // IndexOutOfBoundsException
```

DEĞİŞKENLER

isimler = []

sayilar = -

ÇIKTI

Kod İzleme: `get`, Jenerikler ve Kutulama



"Ali" indis 0'a eklenir.

Adım 2 / 10

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ahmet");
4 isimler.add("Mehmet");
5 String ilkIsim = isimler.get(0);
6 System.out.println(ilkIsim);
7 List<Integer> sayilar = new ArrayList<>();
8 sayilar.add(5);           // Integer.valueOf(5)
9 int x = sayilar.get(0);   // .intValue()
10 isimler.get(3);           // IndexOutOfBoundsException
```

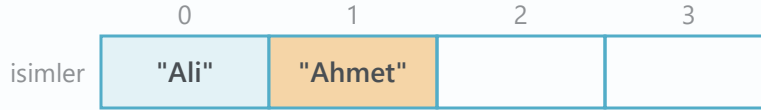
DEĞİŞKENLER

isimler = [Ali]

sayilar = -

ÇIKTI

Kod İzleme: `get`, Jenerikler ve Kutulama



"Ahmet" indis 1'e eklenir.

Adım 3 / 10

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 ▶ isimler.add("Ahmet");
4 isimler.add("Mehmet");
5 String ilkIsim = isimler.get(0);
6 System.out.println(ilkIsim);
7 List<Integer> sayilar = new ArrayList<>();
8 sayilar.add(5);           // Integer.valueOf(5)
9 int x = sayilar.get(0);   // .intValue()
10 isimler.get(3);          // IndexOutOfBoundsException
```

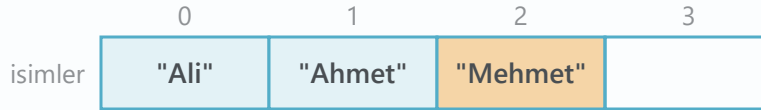
DEĞİŞKENLER

isimler = [Ali, Ahmet]

sayilar = -

ÇIKTI

Kod İzleme: `get`, Jenerikler ve Kutulama



"Mehmet" indis 2'ye eklenir.

Adım 4 / 10

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ahmet");
4 ► isimler.add("Mehmet");
5 String ilkIsim = isimler.get(0);
6 System.out.println(ilkIsim);
7 List<Integer> sayilar = new ArrayList<>();
8 sayilar.add(5);           // Integer.valueOf(5)
9 int x = sayilar.get(0);   // .intValue()
10 isimler.get(3);          // IndexOutOfBoundsException
```

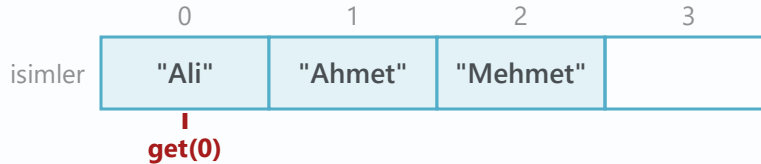
DEĞİŞKENLER

`isimler = [Ali, Ahmet, Mehmet]`

`sayilar = -`

ÇIKTI

Kod İzleme: `get`, Jenerikler ve Kutulama



`get(0)` : $0 < \text{size}(3)$ denetimi geçer, `dizi[0]` doğrudan okunur $\rightarrow O(1)$. Dönüşüm (cast) gerekmez.

Adım 5 / 10

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ahmet");
4 isimler.add("Mehmet");
5 String ilkIsim = isimler.get(0);
6 System.out.println(ilkIsim);
7 List<Integer> sayilar = new ArrayList<>();
8 sayilar.add(5);           // Integer.valueOf(5)
9 int x = sayilar.get(0);   // .intValue()
10 isimler.get(3);           // IndexOutOfBoundsException
```

DEĞİŞKENLER

`isimler = [Ali, Ahmet, Mehmet]`

`sayilar = -`

`ilkIsim = "Ali"`

ÇIKTI

Kod İzleme: `get`, Jenerikler ve Kutulama

	0	1	2	3
isimler	"Ali"	"Ahmet"	"Mehmet"	

Ali yazdırılır.

Adım 6 / 10

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ahmet");
4 isimler.add("Mehmet");
5 String ilkIsim = isimler.get(0);
6 System.out.println(ilkIsim);
7 List<Integer> sayilar = new ArrayList<>();
8 sayilar.add(5);           // Integer.valueOf(5)
9 int x = sayilar.get(0);   // .intValue()
10 isimler.get(3);           // IndexOutOfBoundsException
```

DEĞİŞKENLER

isimler = [Ali, Ahmet, Mehmet]

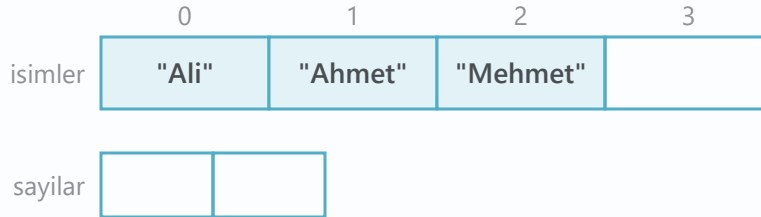
sayilar = -

ilkIsim = "Ali"

ÇIKTI

Ali

Kod İzleme: `get`, Jenerikler ve Kutulama



`List<int>` yazılamaz; sarmalayıcı (wrapper) sınıf `Integer` kullanılır.

Adım 7 / 10

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ahmet");
4 isimler.add("Mehmet");
5 String ilkIsim = isimler.get(0);
6 System.out.println(ilkIsim);
7 ► List<Integer> sayilar = new ArrayList<>();
8 sayilar.add(5);           // Integer.valueOf(5)
9 int x = sayilar.get(0);   // .intValue()
10 isimler.get(3);          // IndexOutOfBoundsException
```

DEĞİŞKENLER

`isimler = [Ali, Ahmet, Mehmet]`

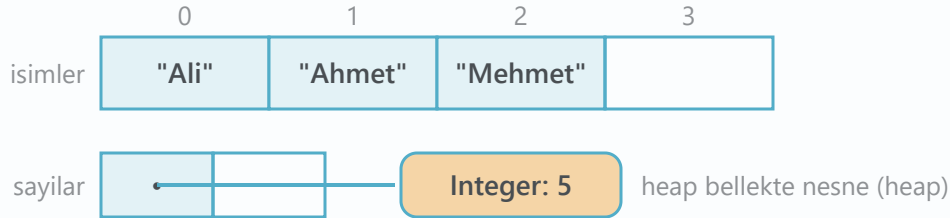
`sayilar = []`

`ilkIsim = "Ali"`

ÇIKTI

Ali

Kod İzleme: `get`, Jenerikler ve Kutulama



Otomatik kutulama (autoboxing): derleyici `Integer.valueOf(5)` yazar; listede nesneye referans durur.

Adım 8 / 10

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ahmet");
4 isimler.add("Mehmet");
5 String ilkIsim = isimler.get(0);
6 System.out.println(ilkIsim);
7 List<Integer> sayilar = new ArrayList<>();
8 sayilar.add(5); // Integer.valueOf(5)
9 int x = sayilar.get(0); // .intValue()
10 isimler.get(3); // IndexOutOfBoundsException
```

DEĞİŞKENLER

isimler = [Ali, Ahmet, Mehmet]

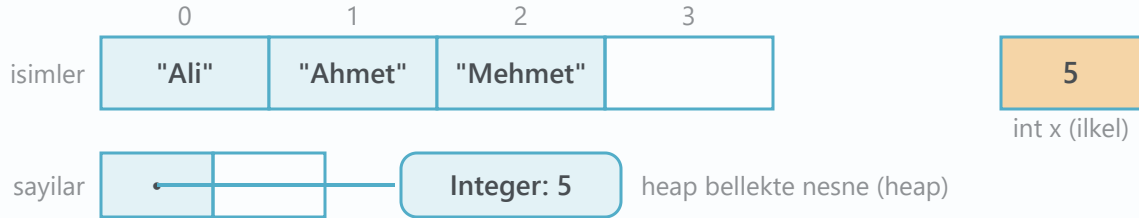
sayilar = [5]

ilkIsim = "Ali"

ÇIKTI

Ali

Kod İzleme: get, Jenerikler ve Kutulama



Kutudan çıkarma (unboxing): `get(0).intValue()` → ilkel 5. (Öğe null olsaydı NullPointerException.)

Adım 9 / 10

```

1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ahmet");
4 isimler.add("Mehmet");
5 String ilkIsim = isimler.get(0);
6 System.out.println(ilkIsim);
7 List<Integer> sayilar = new ArrayList<>();
8 sayilar.add(5);           // Integer.valueOf(5)
9 int x = sayilar.get(0);   // .intValue()
10 isimler.get(3);          // IndexOutOfBoundsException

```

DEĞİŞKENLER

isimler = [Ali, Ahmet, Mehmet]

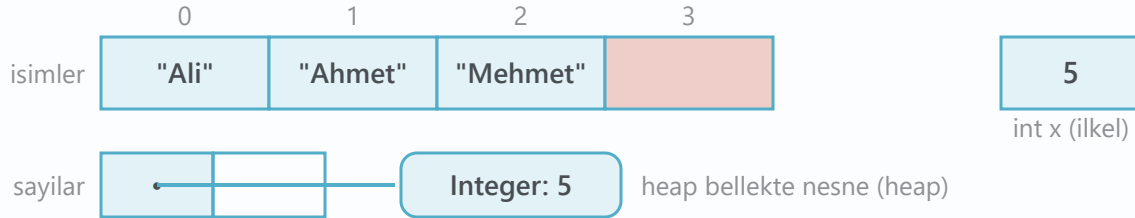
sayilar = [5] ilkIsim = "Ali"

x = 5

ÇIKTI

Ali

Kod İzleme: get, Jenerikler ve Kutulama



`get(3)` : $3 < \text{size}(3)$ değil → `IndexOutOfBoundsException` . Geçerli indisler $0 \dots \text{size} - 1$.

Adım 10 / 10

```

1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali");
3 isimler.add("Ahmet");
4 isimler.add("Mehmet");
5 String ilkIsim = isimler.get(0);
6 System.out.println(ilkIsim);
7 List<Integer> sayilar = new ArrayList<>();
8 sayilar.add(5);           // Integer.valueOf(5)
9 int x = sayilar.get(0);   // .intValue()
▶ isimler.get(3);           // IndexOutOfBoundsException

```

DEĞİŞKENLER

isimler = [Ali, Ahmet, Mehmet]

sayilar = [5] ilkIsim = "Ali"

x = 5

ÇIKTI

Ali

Spliterator: Bölünebilir Okuyucu

```
List<String> kelimeler = new ArrayList<>();
kelimeler.add("Java");
kelimeler.add("Python");
kelimeler.add("C++");
Spliterator<String> split =
    kelimeler.splitIterator();
while (split.tryAdvance(
    System.out::println)) {
    // her çağrı bir öğeyi işler
}
```

Java
Python
C++

- `tryAdvance(eylem)` : sıradaki öğe varsa eylemi uygular ve `true` döner; kalmadıysa `false`.
- `trySplit()` : kalan öğelerin bir kısmını yeni bir Spliterator'a devreder → iş parçacıkları arasında bölüştürme.
- Paralel akışlar (`parallelStream()`) arka planda Spliterator kullanır.

Kod İzleme: trySplit ve tryAdvance

	0	1	2	3
dizi	Java	Python	C++	Go

4 kelimelik liste.

Adım 1 / 9

```
► List<String> kelimeler = new ArrayList<>(  
►   List.of("Java", "Python", "C++", "Go"));  
3 Spliterator<String> a = kelimeler.spliterator();  
4 Spliterator<String> b = a.trySplit();  
5 while (b.tryAdvance(System.out::println)) { }  
6 while (a.tryAdvance(System.out::println)) { }
```

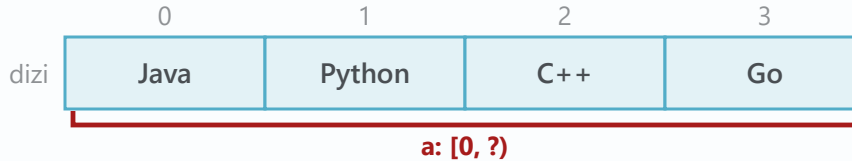
DEĞİŞKENLER

a = -

b = -

ÇIKTI

Kod İzleme: trySplit ve tryAdvance



`spliterator()` : indis 0, bitiş sınırı (fence) -1 → sınır, ilk kullanımda `size()` ile belirlenecek (tembel).

Adım 2 / 9

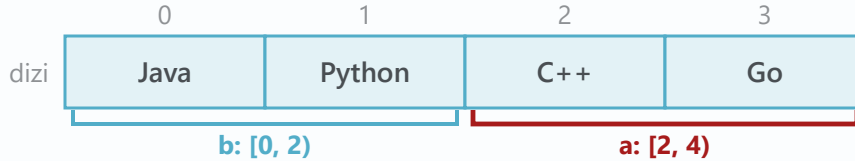
```
1 List<String> kelimeler = new ArrayList<>(  
2     List.of("Java", "Python", "C++", "Go"));  
▶ Spliterator<String> a = kelimeler.spliterator();  
4 Spliterator<String> b = a.trySplit();  
5 while (b.tryAdvance(System.out::println)) { }  
6 while (a.tryAdvance(System.out::println)) { }
```

DEĞİŞKENLER

a = index=0, fence=-1 b = -

ÇIKTI

Kod İzleme: trySplit ve tryAdvance



`trySplit()` : fence = 4, orta = $(0 + 4) / 2 = 2$. İlk yarı [0, 2) yeni `b` 'ye verilir; `a` [2, 4) ile kalır.

Adım 3 / 9

```
1 List<String> kelimeler = new ArrayList<>(  
2     List.of("Java", "Python", "C++", "Go"));  
3 Spliterator<String> a = kelimeler.splitIterator();  
▶ Spliterator<String> b = a.trySplit();  
5 while (b.tryAdvance(System.out::println)) { }  
6 while (a.tryAdvance(System.out::println)) { }
```

DEĞİŞKENLER

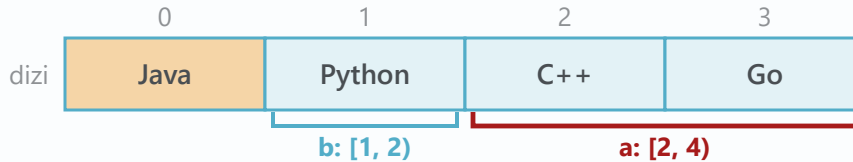
`a` = index=2, fence=4

`b` = index=0, fence=2

ÇIKTI



Kod İzleme: trySplit ve tryAdvance



b.tryAdvance : öge var → "Java" eyleme (println) verilir, index 1'e ilerler; true.

Adım 4 / 9

```
1 List<String> kelimeler = new ArrayList<>(  
2     List.of("Java", "Python", "C++", "Go"));  
3 Spliterator<String> a = kelimeler.spliterator();  
4 Spliterator<String> b = a.trySplit();  
▶ while (b.tryAdvance(System.out::println)) { }  
6 while (a.tryAdvance(System.out::println)) { }
```

DEĞİŞKENLER

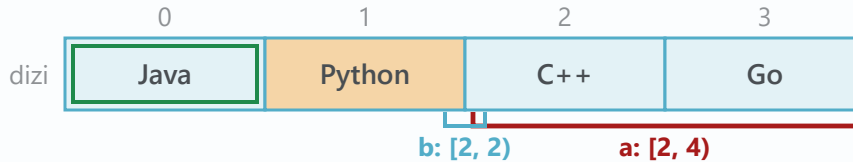
a = index=2, fence=4

b = index=1, fence=2

ÇIKTI

Java

Kod İzleme: trySplit ve tryAdvance



`b.tryAdvance` : öge var → "Python" eyleme (println) verilir, index 2'ye ilerler; true.

Adım 5 / 9

```

1 List<String> kelimeler = new ArrayList<>(
2     List.of("Java", "Python", "C++", "Go"));
3 Spliterator<String> a = kelimeler.spliterator();
4 Spliterator<String> b = a.trySplit();
5 ▶ while (b.tryAdvance(System.out::println)) { }
6 while (a.tryAdvance(System.out::println)) { }

```

DEĞİŞKENLER

`a = index=2, fence=4`

`b = index=2, fence=2`

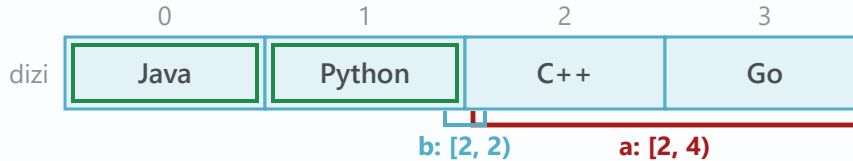
ÇIKTI

```

Java
Python

```

Kod İzleme: trySplit ve tryAdvance



`b.tryAdvance` : index = fence → öge yok, **false**; döngü biter.

Adım 6 / 9

```

1 List<String> kelimeler = new ArrayList<>(
2     List.of("Java", "Python", "C++", "Go"));
3 Spliterator<String> a = kelimeler.splitIterator();
4 Spliterator<String> b = a.trySplit();
▶ while (b.tryAdvance(System.out::println)) { }
6 while (a.tryAdvance(System.out::println)) { }

```

DEĞİŞKENLER

a = index=2, fence=4

b = index=2, fence=2

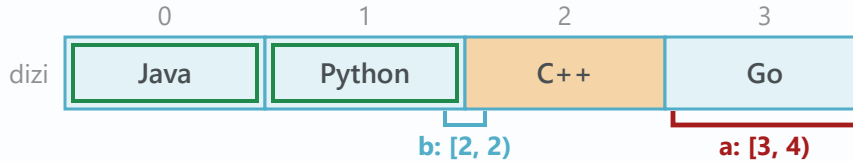
ÇIKTI

```

Java
Python

```

Kod İzleme: trySplit ve tryAdvance



`a.tryAdvance` : öge var → "C++" eyleme (println) verilir, index 3'e ilerler; true.

Adım 7 / 9

```
1 List<String> kelimeler = new ArrayList<>(  
2     List.of("Java", "Python", "C++", "Go"));  
3 Spliterator<String> a = kelimeler.spliterator();  
4 Spliterator<String> b = a.trySplit();  
5 while (b.tryAdvance(System.out::println)) { }  
▶ while (a.tryAdvance(System.out::println)) { }
```

DEĞİŞKENLER

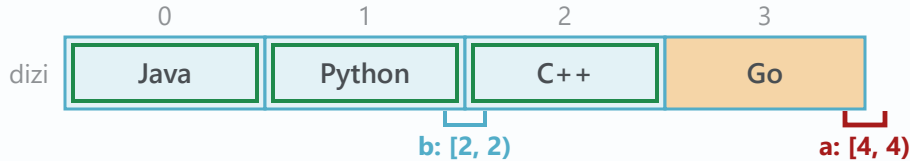
a = index=3, fence=4

b = index=2, fence=2

ÇIKTI

```
Java  
Python  
C++
```

Kod İzleme: trySplit ve tryAdvance



`a.tryAdvance` : öge var → "Go" eyleme (println) verilir, index 4'e ilerler; true.

Adım 8 / 9

```
1 List<String> kelimeler = new ArrayList<>(  
2     List.of("Java", "Python", "C++", "Go"));  
3 Spliterator<String> a = kelimeler.spliterator();  
4 Spliterator<String> b = a.trySplit();  
5 while (b.tryAdvance(System.out::println)) { }  
▶ while (a.tryAdvance(System.out::println)) { }
```

DEĞİŞKENLER

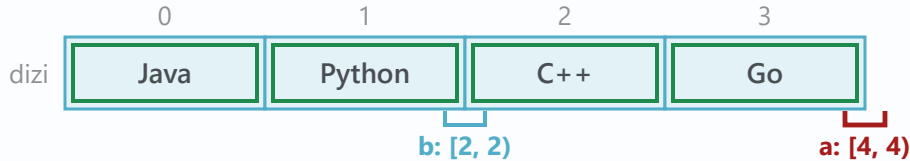
a = index=4, fence=4

b = index=2, fence=2

ÇIKTI

```
Java  
Python  
C++  
Go
```

Kod İzleme: trySplit ve tryAdvance



`a.tryAdvance` : `index = fence` → öge yok, **false**; döngü biter. İki parça iki ayrı iş parçasığında da işlenebilirdi.

Adım 9 / 9

```
1 List<String> kelimeler = new ArrayList<>(
2     List.of("Java", "Python", "C++", "Go"));
3 Spliterator<String> a = kelimeler.spliterator();
4 Spliterator<String> b = a.trySplit();
5 while (b.tryAdvance(System.out::println)) { }
▶ while (a.tryAdvance(System.out::println)) { }
```

DEĞİŞKENLER

`a = index=4, fence=4`

`b = index=2, fence=2`

ÇIKTI

```
Java
Python
C++
Go
```

Koleksiyonları Gezmenin Üç Yolu

1 · indisli for

```
int n = list.size();
for (int i = 0;
     i < n; i++) {
    String s =
        list.get(i);
    ...
}
```

Yalnızca `List` 'te; indis de lazımsa.

2 · forEach

```
list.forEach(
    System.out::println);

// ya da for-each:
for (String s : list) ...
```

Her `Iterable` 'da; en kısa yazım.

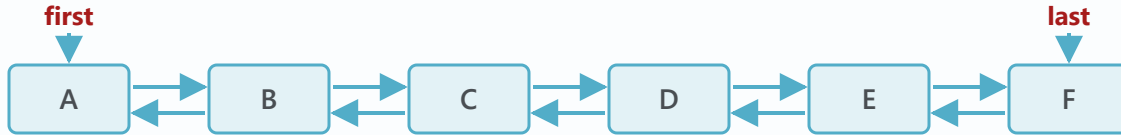
3 · Iterator

```
Iterator<String> it =
    list.iterator();
while (it.hasNext()) {
    String s = it.next();
}
```

Gezerken güvenle silmek gerekirse.

Dikkat: `LinkedList` 'te `get(i)` $O(n)$ olduğundan indisli döngü toplamda $O(n^2)$ sürer; `forEach` / `Iterator` her iki listede de $O(n)$ 'dir.

Kod İzleme: LinkedList'te İndisli Döngünün Bedeli



6 düğümlü çift yönlü bağlı liste.

Adım 1 / 9

```
▶ List<String> list = new LinkedList<>(  
▶   List.of("A", "B", "C", "D", "E", "F"));  
3 int n = list.size(), adim = 0;  
4 for (int i = 0; i < n; i++) {  
5   String s = list.get(i); // node(i) yürür  
6   adim += Math.min(i, n - 1 - i);  
7 }  
8 System.out.println(adim);
```

DEĞİŞKENLER

n = -

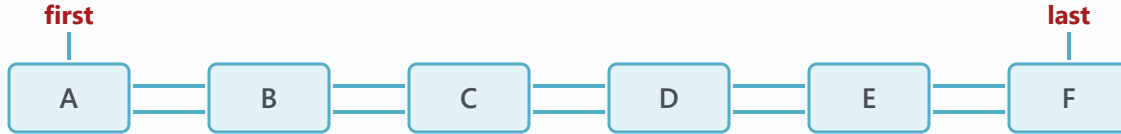
i = -

adim = -

s = -

ÇIKTI

Kod İzleme: LinkedList'te İndisli Döngünün Bedeli



`adim` : `get` 'lerin toplam kaç bağlantı yürüdüğünü sayacak.

Adım 2 / 9

```
1 List<String> list = new LinkedList<>(  
2     List.of("A", "B", "C", "D", "E", "F"));  
▶ int n = list.size(), adim = 0;  
4 for (int i = 0; i < n; i++) {  
5     String s = list.get(i); // node(i) yürür  
6     adim += Math.min(i, n - 1 - i);  
7 }  
8 System.out.println(adim);
```

DEĞİŞKENLER

`n = 6`

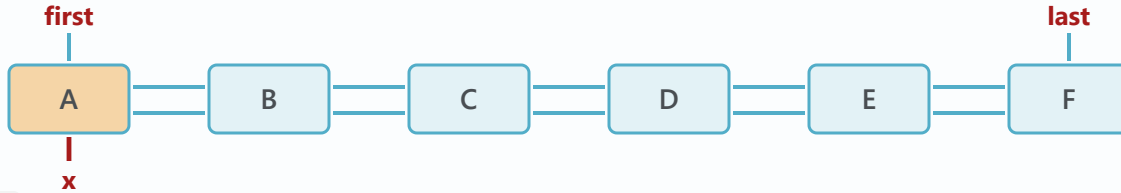
`i = -`

`adim = 0`

`s = -`

ÇIKTI

Kod İzleme: LinkedList'te İndisli Döngünün Bedeli



`get(0)` : $0 < n/2 = 3 \rightarrow$ baştan (first) yürür, **0** bağlantı. Toplam 0.

Adım 3 / 9

```
1 List<String> list = new LinkedList<>(  
2     List.of("A", "B", "C", "D", "E", "F"));  
3 int n = list.size(), adim = 0;  
4 for (int i = 0; i < n; i++) {  
5     String s = list.get(i); // node(i) yürür  
6     adim += Math.min(i, n - 1 - i);  
7 }  
8 System.out.println(adim);
```

DEĞİŞKENLER

`n = 6`

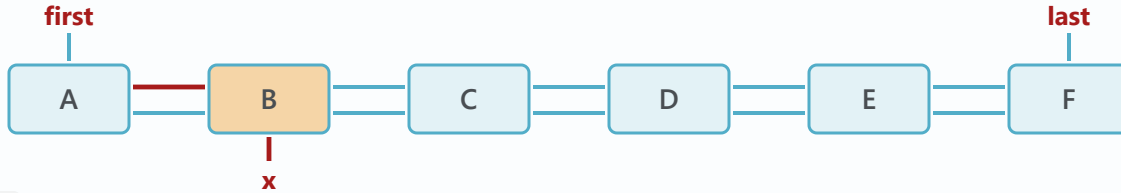
`i = 0`

`adim = 0`

`s = "A"`

ÇIKTI

Kod İzleme: LinkedList'te İndisli Döngünün Bedeli



`get(1)` : $1 < n/2 = 3 \rightarrow$ baştan (first) yürür, 1 bağlantı. Toplam 1.

Adım 4 / 9

```

1 List<String> list = new LinkedList<>(
2     List.of("A", "B", "C", "D", "E", "F"));
3 int n = list.size(), adim = 0;
4 for (int i = 0; i < n; i++) {
5     ▶ String s = list.get(i); // node(i) yürür
6     ▶ adim += Math.min(i, n - 1 - i);
7 }
8 System.out.println(adim);

```

DEĞİŞKENLER

`n = 6`

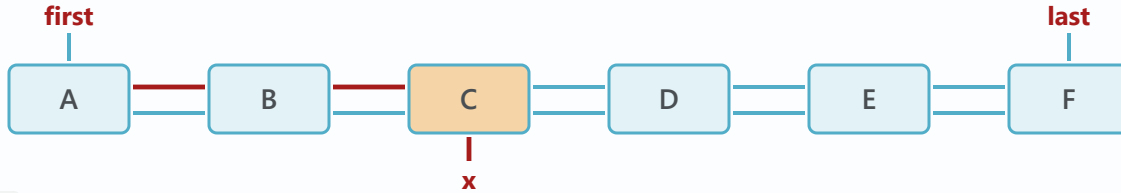
`i = 1`

`adim = 1`

`s = "B"`

ÇIKTI

Kod İzleme: LinkedList'te İndisli Döngünün Bedeli



`get(2)` : $2 < n/2 = 3 \rightarrow$ baştan (first) yürür, **2** bağlantı. Toplam 3.

Adım 5 / 9

```
1 List<String> list = new LinkedList<>(  
2     List.of("A", "B", "C", "D", "E", "F"));  
3 int n = list.size(), adim = 0;  
4 for (int i = 0; i < n; i++) {  
5     String s = list.get(i); // node(i) yürür  
6     adim += Math.min(i, n - 1 - i);  
7 }  
8 System.out.println(adim);
```

DEĞİŞKENLER

`n = 6`

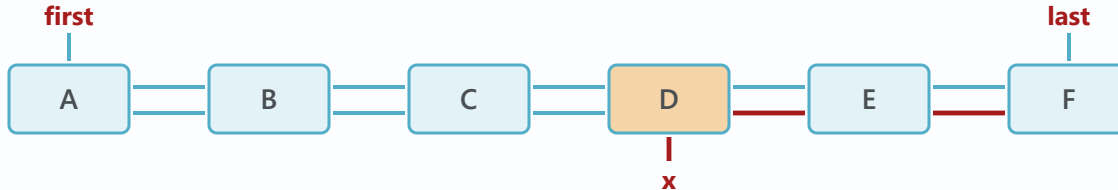
`i = 2`

`adim = 3`

`s = "C"`

ÇIKTI

Kod İzleme: LinkedList'te İndisli Döngünün Bedeli



`get(3)` : $3 \geq n/2 = 3 \rightarrow$ sondan (last) yürür, **2** bağlantı. Toplam 5.

Adım 6 / 9

```

1 List<String> list = new LinkedList<>(
2     List.of("A", "B", "C", "D", "E", "F"));
3 int n = list.size(), adim = 0;
4 for (int i = 0; i < n; i++) {
5     ▶ String s = list.get(i); // node(i) yürür
6     ▶ adim += Math.min(i, n - 1 - i);
7 }
8 System.out.println(adim);

```

DEĞİŞKENLER

`n = 6`

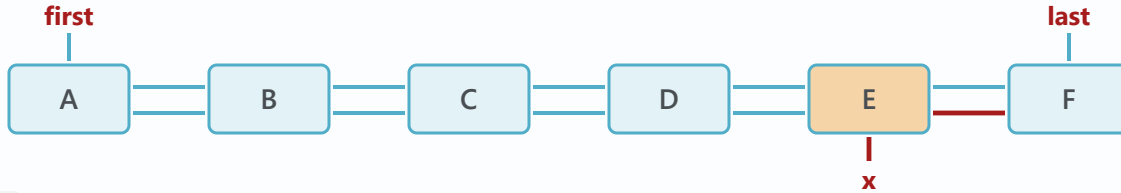
`i = 3`

`adim = 5`

`s = "D"`

ÇIKTI

Kod İzleme: LinkedList'te İndisli Döngünün Bedeli



`get(4)` : $4 \geq n/2 = 3 \rightarrow$ sondan (last) yürür, **1** bağlantı. Toplam 6.

Adım 7 / 9

```
1 List<String> list = new LinkedList<>(  
2     List.of("A", "B", "C", "D", "E", "F"));  
3 int n = list.size(), adim = 0;  
4 for (int i = 0; i < n; i++) {  
5     ▶ String s = list.get(i); // node(i) yürür  
6     ▶ adim += Math.min(i, n - 1 - i);  
7 }  
8 System.out.println(adim);
```

DEĞİŞKENLER

`n = 6`

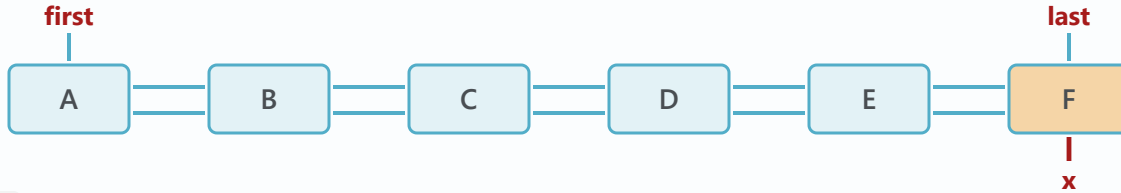
`i = 4`

`adim = 6`

`s = "E"`

ÇIKTI

Kod İzleme: LinkedList'te İndisli Döngünün Bedeli



`get(5)` : $5 \geq n/2 = 3 \rightarrow$ sondan (last) yürür, **0** bağlantı. Toplam 6.

Adım 8 / 9

```

1 List<String> list = new LinkedList<>(
2     List.of("A", "B", "C", "D", "E", "F"));
3 int n = list.size(), adim = 0;
4 for (int i = 0; i < n; i++) {
5     ▶ String s = list.get(i); // node(i) yürür
6     ▶ adim += Math.min(i, n - 1 - i);
7 }
8 System.out.println(adim);
  
```

DEĞİŞKENLER

`n = 6`

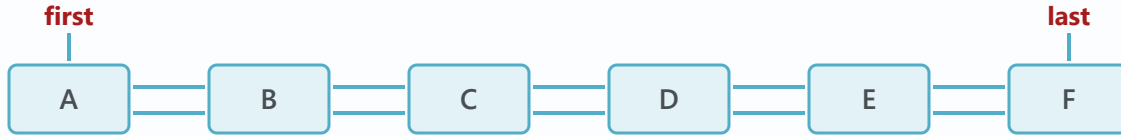
`i = 5`

`adim = 6`

`s = "F"`

ÇIKTI

Kod İzleme: LinkedList'te İndisli Döngünün Bedeli



Toplam $6 \approx n^2/4 \rightarrow O(n^2)$. Iterator ise her `next()` 'te 1 bağlantı: toplam $n - 1 = 5 \rightarrow O(n)$.

Adım 9 / 9

```
1 List<String> list = new LinkedList<>(  
2     List.of("A", "B", "C", "D", "E", "F"));  
3 int n = list.size(), adim = 0;  
4 for (int i = 0; i < n; i++) {  
5     String s = list.get(i); // node(i) yürür  
6     adim += Math.min(i, n - 1 - i);  
7 }  
▶ System.out.println(adim);
```

DEĞİŞKENLER

n = 6

i = 6

adim = 6

s = -

ÇIKTI

6

Iterator (Okuyucu) Arayüzü

hasNext()

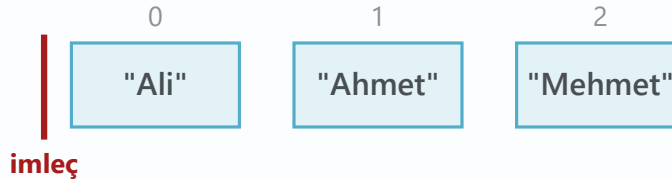
Gezilecek öğe kaldı mı?
Sona gelindiye `false`.

next()

Sıradaki öğeyi döndürür,
imleci ilerletir. Öğe
kalmadıysa
`NoSuchElementException`.

remove()

Son `next()` ile dönen
öğeyi siler (isteğe bağlı
işlem).



`iterator()` : imleç ilk öğenin **önünde**. `hasNext()` → true

Adım 1 / 5

```
while (iterator.hasNext()) { String isim = iterator.next(); System.out.println(isim); }
```

Iterator (Okuyucu) Arayüzü

hasNext()

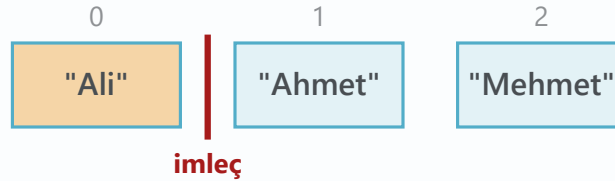
Gezilecek öğe kaldı mı?
Sona gelindiye `false`.

next()

Sıradaki öğeyi döndürür,
imleci ilerletir. Öğe
kalmadıysa
`NoSuchElementException`.

remove()

Son `next()` ile dönen
öğeyi siler (isteğe bağlı
işlem).



`next()` → "Ali" (imleç öğenin üzerinden atlar)

Adım 2 / 5

```
while (iterator.hasNext()) { String isim = iterator.next(); System.out.println(isim); }
```

Iterator (Okuyucu) Arayüzü

hasNext()

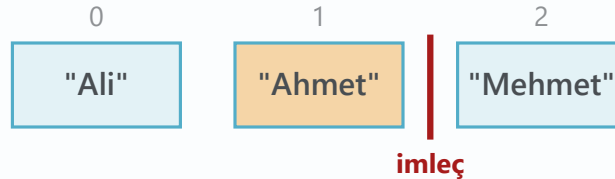
Gezilecek öğe kaldı mı?
Sona gelindiye `false`.

next()

Sıradaki öğeyi döndürür,
imleci ilerletir. Öğ
kalmadıysa
`NoSuchElementException`.

remove()

Son `next()` ile dönen
öğeyi siler (isteğe bağlı
işlem).



`next()` → "Ahmet"

Adım 3 / 5

```
while (iterator.hasNext()) { String isim = iterator.next(); System.out.println(isim); }
```

Iterator (Okuyucu) Arayüzü

hasNext()

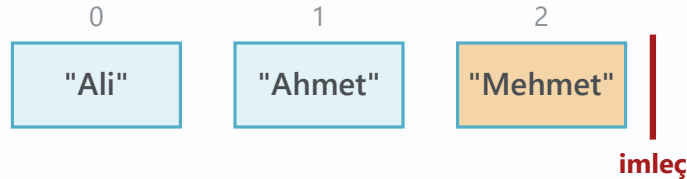
Gezilecek öğe kaldı mı?
Sona gelindiye `false`.

next()

Sıradaki öğeyi döndürür,
imleci ilerletir. Öğe
kalmadıysa
`NoSuchElementException`.

remove()

Son `next()` ile dönen
öğeyi siler (isteğe bağlı
işlem).



`next()` → "Mehmet"; artık `hasNext()` → `false`, döngü biter

Adım 4 / 5

```
while (iterator.hasNext()) { String isim = iterator.next(); System.out.println(isim); }
```

Iterator (Okuyucu) Arayüzü

hasNext()

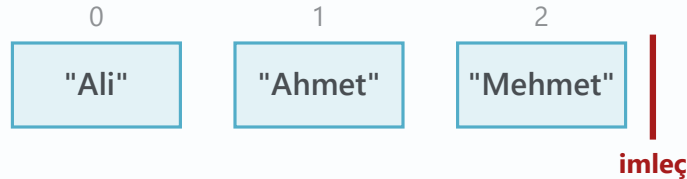
Gezilecek öğe kaldı mı?
Sona gelindiye `false`.

next()

Sıradaki öğeyi döndürür,
imleci ilerletir. Öğe
kalmadıysa
`NoSuchElementException`.

remove()

Son `next()` ile dönen
öğeyi siler (isteğe bağlı
işlem).

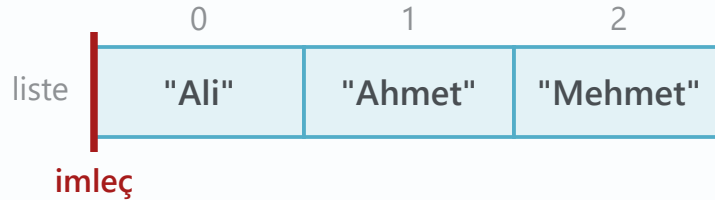


Bir kez daha `next()` çağrılırsa → `NoSuchElementException`

Adım 5 / 5

```
while (iterator.hasNext()) { String isim = iterator.next(); System.out.println(isim); }
```

Kod İzleme: Iterator ile Gezinme



`iterator()` : imleç ilk ögenin **önünde** (cursor = 0). Iterator liste üzerinde gezen ayrı bir nesnedir.

Adım 1 / 11

```
► Iterator<String> iterator = isimler.iterator();  
2 while (iterator.hasNext()) {  
3     String isim = iterator.next();  
4     System.out.println(isim);  
5 }
```

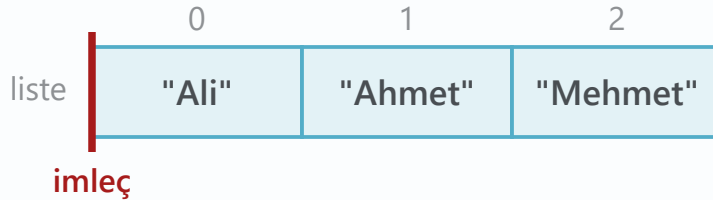
DEĞİŞKENLER

cursor = 0

isim = -

ÇIKTI

Kod İzleme: Iterator ile Gezinme



`hasNext()` : cursor 0 $\neq$ size 3 $\rightarrow$ **true**.

Adım 2 / 11

```
1 Iterator<String> iterator = isimler.iterator();  
▶ while (iterator.hasNext()) {  
3     String isim = iterator.next();  
4     System.out.println(isim);  
5 }
```

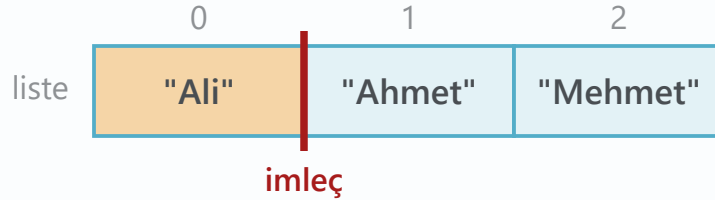
DEĞİŞKENLER

cursor = 0

isim = -

ÇIKTI

Kod İzleme: Iterator ile Gezinme



`next()` → **"Ali"**; imleç öğenin üzerinden atlar (cursor = 1).

Adım 3 / 11

```
1 Iterator<String> iterator = isimler.iterator();
2 while (iterator.hasNext()) {
3     ▶ String isim = iterator.next();
4     System.out.println(isim);
5 }
```

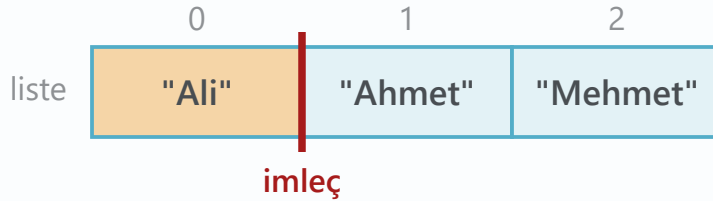
DEĞİŞKENLER

cursor = 1

isim = "Ali"

ÇIKTI

Kod İzleme: Iterator ile Gezinme



Ali yazdırılır.

Adım 4 / 11

```
1 Iterator<String> iterator = isimler.iterator();
2 while (iterator.hasNext()) {
3     String isim = iterator.next();
4     System.out.println(isim);
5 }
```

DEĞİŞKENLER

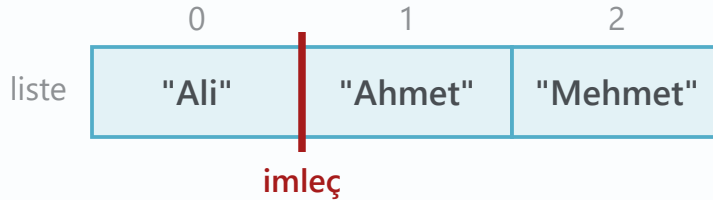
cursor = 1

isim = "Ali"

ÇIKTI

Ali

Kod İzleme: Iterator ile Gezinme



`hasNext()` : cursor 1 $\neq$ size 3 $\rightarrow$ **true**.

Adım 5 / 11

```
1 Iterator<String> iterator = isimler.iterator();
2 while (iterator.hasNext()) {
3     String isim = iterator.next();
4     System.out.println(isim);
5 }
```

DEĞİŞKENLER

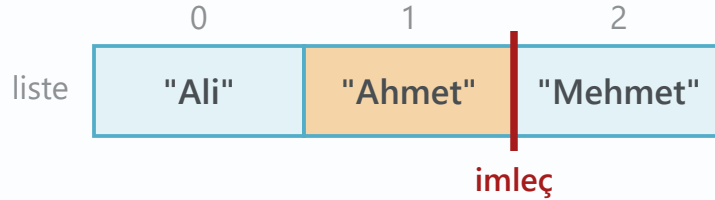
cursor = 1

isim = "Ali"

ÇIKTI

Ali

Kod İzleme: Iterator ile Gezinme



`next()` → **"Ahmet"**; imleç öğenin üzerinden atlar (cursor = 2).

Adım 6 / 11

```
1 Iterator<String> iterator = isimler.iterator();
2 while (iterator.hasNext()) {
3     ▶ String isim = iterator.next();
4     System.out.println(isim);
5 }
```

DEĞİŞKENLER

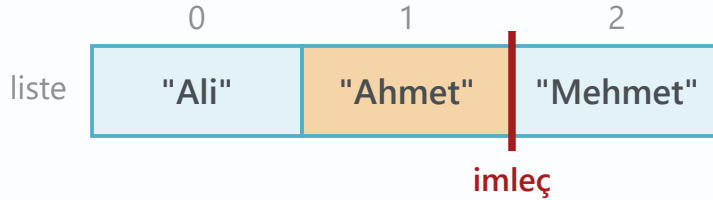
cursor = 2

isim = "Ahmet"

ÇIKTI

Ali

Kod İzleme: Iterator ile Gezinme



Ahmet yazdırılır.

Adım 7 / 11

```
1 Iterator<String> iterator = isimler.iterator();
2 while (iterator.hasNext()) {
3     String isim = iterator.next();
4     System.out.println(isim);
5 }
```

DEĞİŞKENLER

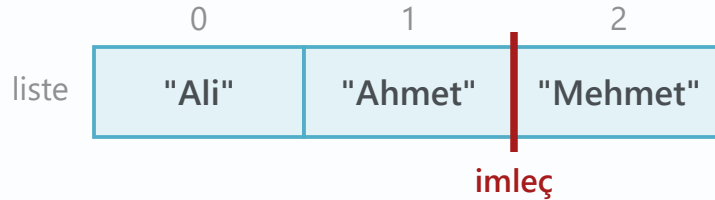
cursor = 2

isim = "Ahmet"

ÇIKTI

Ali
Ahmet

Kod İzleme: Iterator ile Gezinme



`hasNext()` : cursor 2 $\neq$ size 3 $\rightarrow$ **true**.

Adım 8 / 11

```
1 Iterator<String> iterator = isimler.iterator();  
▶ while (iterator.hasNext()) {  
3     String isim = iterator.next();  
4     System.out.println(isim);  
5 }
```

DEĞİŞKENLER

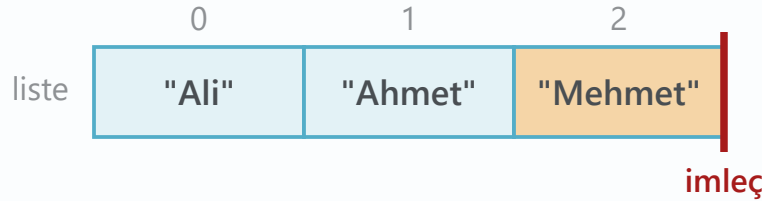
cursor = 2

isim = "Ahmet"

ÇIKTI

Ali
Ahmet

Kod İzleme: Iterator ile Gezinme



`next()` → **"Mehmet"**; imleç öğenin üzerinden atlar (cursor = 3).

Adım 9 / 11

```
1 Iterator<String> iterator = isimler.iterator();
2 while (iterator.hasNext()) {
3     ▶ String isim = iterator.next();
4     System.out.println(isim);
5 }
```

DEĞİŞKENLER

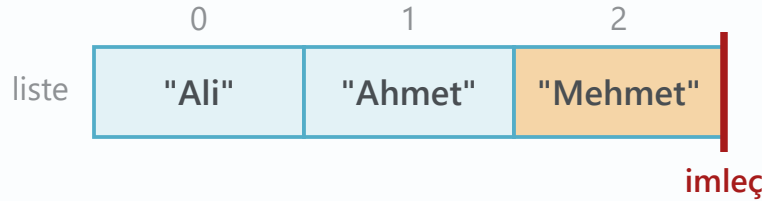
cursor = 3

isim = "Mehmet"

ÇIKTI

Ali
Ahmet

Kod İzleme: Iterator ile Gezinme



Mehmet yazdırılır.

Adım 10 / 11

```
1 Iterator<String> iterator = isimler.iterator();
2 while (iterator.hasNext()) {
3     String isim = iterator.next();
4     System.out.println(isim);
5 }
```

DEĞİŞKENLER

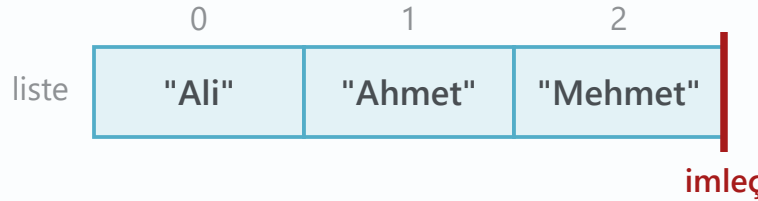
cursor = 3

isim = "Mehmet"

ÇIKTI

```
Ali
Ahmet
Mehmet
```

Kod İzleme: Iterator ile Gezinme



cursor 3 = size 3 → `hasNext()` **false**, döngü biter. Bir kez daha `next()` çağrılıyorsa → `NoSuchElementException`.

Adım 11 / 11

```
1 Iterator<String> iterator = isimler.iterator();  
2 ▶ while (iterator.hasNext()) {  
3     String isim = iterator.next();  
4     System.out.println(isim);  
5 }
```

DEĞİŞKENLER

cursor = 3

isim = "Mehmet"

ÇIKTI

```
Ali  
Ahmet  
Mehmet
```

Gezerken Silme: ConcurrentModificationException

Yanlış

```
for (String s : isimler)
    if (s.startsWith("A"))
        isimler.remove(s);
// ConcurrentModification-
// Exception (fail-fast)
```

Doğru

```
Iterator<String> it =
    isimler.iterator();
while (it.hasNext())
    if (it.next().startsWith("A"))
        it.remove();
// ya da kısaca:
isimler.removeIf(s -> s.startsWith("A"));
```

- for-each gizlice bir Iterator kullanır; liste okuyucu dışından değiştirilirse sonraki `next()` hatayı fark eder ve durur (**fail-fast**).
- `it.remove()`, `next()` 'ten sonra yalnızca **bir kez** çağrılabilir; aksi hâlde `IllegalStateException`.

Kod İzleme: ConcurrentModificationException

liste

modCount = 0

Boş ArrayList. Her yapısal değişiklik (ekleme/silme) listenin **modCount** sayacını artırır.

Adım 1 / 10

```
► List<String> isimler = new ArrayList<>();
2 isimler.add("Ali"); isimler.add("Can");
3 isimler.add("Ayşe");           // modCount = 3
4 // for (String s : isimler) ... derleyici:
5 Iterator<String> it = isimler.iterator();
6 while (it.hasNext()) {
7     String s = it.next();
8     if (s.startsWith("A"))
9         isimler.remove(s);           // it dışından!
10 }
```

DEĞİŞKENLER

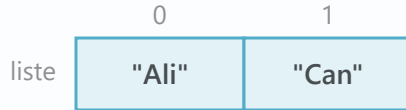
isimler = []

modCount = 0

ÇIKTI



Kod İzleme: ConcurrentModificationException



modCount = 2

İki `add` → modCount = 2.

Adım 2 / 10

```
1 List<String> isimler = new ArrayList<>();  
▶ isimler.add("Ali"); isimler.add("Can");  
3 isimler.add("Ayşe");           // modCount = 3  
4 // for (String s : isimler) ... derleyici:  
5 Iterator<String> it = isimler.iterator();  
6 while (it.hasNext()) {  
7     String s = it.next();  
8     if (s.startsWith("A"))  
9         isimler.remove(s);           // it dışından!  
10 }
```

DEĞİŞKENLER

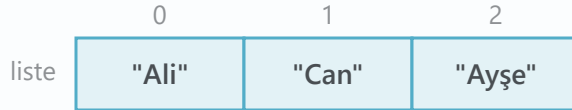
isimler = [Ali, Can]

modCount = 2

ÇIKTI



Kod İzleme: ConcurrentModificationException



modCount = 3

modCount = 3. Şimdi for-each döngüsünün derleyicinin ürettiği hâlini izleyelim.

Adım 3 / 10

```
1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali"); isimler.add("Can");
3 ▶ isimler.add("Ayşe"); // modCount = 3
4 // for (String s : isimler) ... derleyici:
5 Iterator<String> it = isimler.iterator();
6 while (it.hasNext()) {
7     String s = it.next();
8     if (s.startsWith("A"))
9         isimler.remove(s); // it dışından!
10 }
```

DEĞİŞKENLER

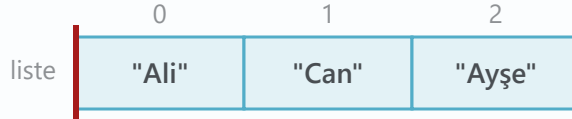
isimler = [Ali, Can, Ayşe]

modCount = 3

ÇIKTI



Kod İzleme: ConcurrentModificationException



$\text{modCount} = 3 \cdot \text{expectedModCount} = 3 \checkmark$

`iterator()` : imleç 0; okuyucu o anki sayacı **expectedModCount = 3** olarak saklar.

Adım 4 / 10

```

1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali"); isimler.add("Can");
3 isimler.add("Ayşe"); // modCount = 3
4 // for (String s : isimler) ... derleyici:
▶ Iterator<String> it = isimler.iterator();
5 while (it.hasNext()) {
6     String s = it.next();
7     if (s.startsWith("A"))
8         isimler.remove(s); // it dışından!
9 }
10 }
```

DEĞİŞKENLER

`isimler = [Ali, Can, Ayşe]`

`cursor = 0`

`s = -`

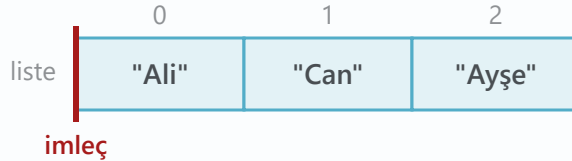
`modCount = 3`

`expectedModCount = 3`

ÇIKTI



Kod İzleme: ConcurrentModificationException



$\text{modCount} = 3 \cdot \text{expectedModCount} = 3 \checkmark$

`hasNext()` : cursor 0 $\neq$ size 3 $\rightarrow$ true.

Adım 5 / 10

```

1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali"); isimler.add("Can");
3 isimler.add("Ayşe");           // modCount = 3
4 // for (String s : isimler) ... derleyici:
5 Iterator<String> it = isimler.iterator();
6 ▶ while (it.hasNext()) {
7     String s = it.next();
8     if (s.startsWith("A"))
9         isimler.remove(s);           // it dışından!
10 }
```

DEĞİŞKENLER

`isimler = [Ali, Can, Ayşe]`

`cursor = 0`

`s = -`

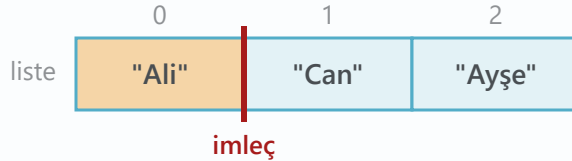
`modCount = 3`

`expectedModCount = 3`

ÇIKTI



Kod İzleme: ConcurrentModificationException



$\text{modCount} = 3 \cdot \text{expectedModCount} = 3 \checkmark$

`next()` : önce $\text{modCount} == \text{expectedModCount}$ mu? $3 == 3 \checkmark \rightarrow$ **"Ali"**, `cursor = 1`.

Adım 6 / 10

```

1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali"); isimler.add("Can");
3 isimler.add("Ayşe");           // modCount = 3
4 // for (String s : isimler) ... derleyici:
5 Iterator<String> it = isimler.iterator();
6 while (it.hasNext()) {
7     ▶ String s = it.next();
8     if (s.startsWith("A"))
9         isimler.remove(s);           // it dışından!
10 }

```

DEĞİŞKENLER

`isimler = [Ali, Can, Ayşe]`

`cursor = 1` `s = "Ali"`

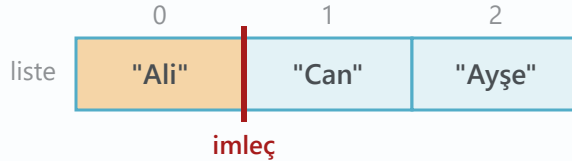
`modCount = 3`

`expectedModCount = 3`

ÇIKTI



Kod İzleme: ConcurrentModificationException



$\text{modCount} = 3 \cdot \text{expectedModCount} = 3 \checkmark$

"Ali" A ile başlıyor → true.

Adım 7 / 10

```

1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali"); isimler.add("Can");
3 isimler.add("Ayşe");           // modCount = 3
4 // for (String s : isimler) ... derleyici:
5 Iterator<String> it = isimler.iterator();
6 while (it.hasNext()) {
7     String s = it.next();
8     ▶ if (s.startsWith("A"))
9         isimler.remove(s);           // it dışından!
10 }

```

DEĞİŞKENLER

isimler = [Ali, Can, Ayşe]

cursor = 1 s = "Ali"

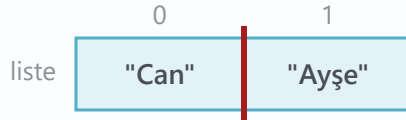
modCount = 3

expectedModCount = 3

ÇIKTI



Kod İzleme: ConcurrentModificationException



imleç

modCount = 4 · expectedModCount = 3 ✗

`isimler.remove(s)` : listeden **doğrudan** silinir → öğeler sola kayar, modCount = 4. Iterator bundan habersiz (expected hâlâ 3).

Adım 8 / 10

```

1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali"); isimler.add("Can");
3 isimler.add("Ayşe");           // modCount = 3
4 // for (String s : isimler) ... derleyici:
5 Iterator<String> it = isimler.iterator();
6 while (it.hasNext()) {
7     String s = it.next();
8     if (s.startsWith("A"))
9         isimler.remove(s);      // it dışından!
10 }

```

DEĞİŞKENLER

isimler = [Can, Ayşe]

cursor = 1

s = "Ali"

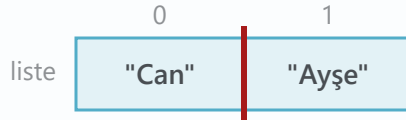
modCount = 4

expectedModCount = 3

ÇIKTI



Kod İzleme: ConcurrentModificationException



$\text{modCount} = 4 \cdot \text{expectedModCount} = 3$ ✗

`hasNext()` : cursor 1 $\neq$ size 2 $\rightarrow$ true. ("Can" imlecin gerisinde kaldı; hiç gezilmeyecekti!)

Adım 9 / 10

```

1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali"); isimler.add("Can");
3 isimler.add("Ayşe");           // modCount = 3
4 // for (String s : isimler) ... derleyici:
5 Iterator<String> it = isimler.iterator();
6 ► while (it.hasNext()) {
7     String s = it.next();
8     if (s.startsWith("A"))
9         isimler.remove(s);           // it dışından!
10 }

```

DEĞİŞKENLER

`isimler = [Can, Ayşe]`

`cursor = 1` `s = "Ali"`

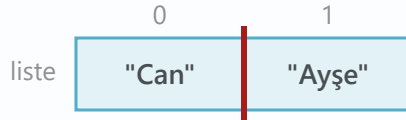
`modCount = 4`

`expectedModCount = 3`

ÇIKTI



Kod İzleme: ConcurrentModificationException



imleç

$$\text{modCount} = 4 \cdot \text{expectedModCount} = 3 \quad \times$$

`next()` : $\text{modCount } 4 \neq \text{expectedModCount } 3 \rightarrow$ **ConcurrentModificationException**. Iterator değişikliği hemen fark edip durur (**fail-fast**).

Adım 10 / 10

```

1 List<String> isimler = new ArrayList<>();
2 isimler.add("Ali"); isimler.add("Can");
3 isimler.add("Ayşe");           // modCount = 3
4 // for (String s : isimler) ... derleyici:
5 Iterator<String> it = isimler.iterator();
6 while (it.hasNext()) {
7     ▶ String s = it.next();
8     if (s.startsWith("A"))
9         isimler.remove(s);           // it dışından!
10 }

```

DEĞİŞKENLER

isimler = [Can, Ayşe]

cursor = 1 s = "Ali"

modCount = 4

expectedModCount = 3

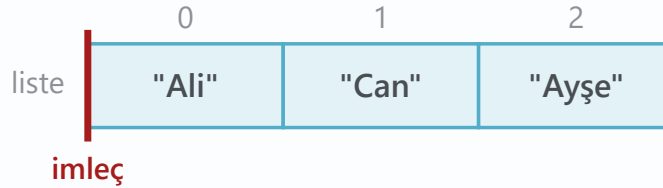
ÇIKTI

```

Exception in thread "main"
java.util.
ConcurrentModificationException

```

Kod İzleme: `it.remove()` ile Güvenli Silme



$\text{modCount} = 3 \cdot \text{expectedModCount} = 3 \checkmark$

Aynı liste [Ali, Can, Ayşe], $\text{modCount} = 3$. Iterator: $\text{cursor} = 0$, **lastRet = -1** (henüz öge döndürmedi).

Adım 1 / 14

```
► Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3   String s = it.next();
4   if (s.startsWith("A"))
5     it.remove(); // sayacıları eşitle
6 }
7 System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Ali, Can, Ayşe]`

`cursor = 0`

`lastRet = -1`

`s = -`

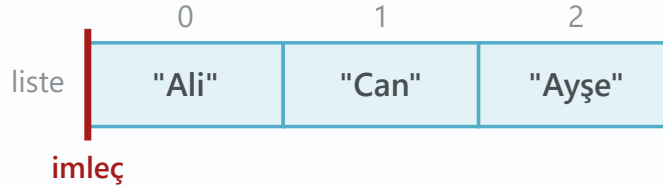
`modCount = 3`

`expected = 3`

ÇIKTI



Kod İzleme: `it.remove()` ile Güvenli Silme



$\text{modCount} = 3 \cdot \text{expectedModCount} = 3 \checkmark$

`hasNext()` : cursor 0 $\neq$ size 3 $\rightarrow$ **true**.

Adım 2 / 14

```
1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     String s = it.next();
4     if (s.startsWith("A"))
5         it.remove(); // sayacıları eşitler
6 }
7 System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Ali, Can, Ayşe]`

`cursor = 0`

`lastRet = -1`

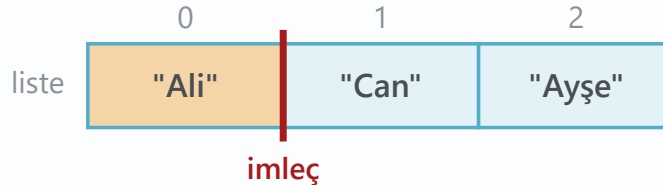
`s = -`

`modCount = 3`

`expected = 3`

ÇIKTI

Kod İzleme: `it.remove()` ile Güvenli Silme



`modCount = 3 · expectedModCount = 3 ✓`

`next()` : `3 == 3 ✓` → **"Ali"**; `cursor = 1`, `lastRet = 0`.

Adım 3 / 14

```
1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     ▶ String s = it.next();
4     if (s.startsWith("A"))
5         it.remove(); // sayacıları eşitler
6 }
7 System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Ali, Can, Ayşe]`

`cursor = 1`

`lastRet = 0`

`s = "Ali"`

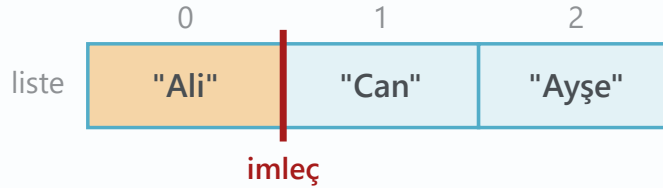
`modCount = 3`

`expected = 3`

ÇIKTI



Kod İzleme: `it.remove()` ile Güvenli Silme



`modCount = 3 · expectedModCount = 3 ✓`

"Ali" A ile başlıyor mu? **evet.**

Adım 4 / 14

```
1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     String s = it.next();
4     ▶ if (s.startsWith("A"))
5         it.remove(); // sayaçları eşitler
6 }
7 System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Ali, Can, Ayşe]`

`cursor = 1`

`lastRet = 0`

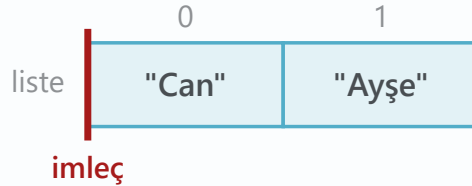
`s = "Ali"`

`modCount = 3`

`expected = 3`

ÇIKTI

Kod İzleme: `it.remove()` ile Güvenli Silme



$\text{modCount} = 4 \cdot \text{expectedModCount} = 4 \checkmark$

`it.remove()` : lastRet'teki öge silinir; **cursor = 0** (kayan öge atlanmaz), lastRet = -1, expectedModCount = modCount = 4.

Adım 5 / 14

```

1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     String s = it.next();
4     if (s.startsWith("A"))
5         it.remove(); // sayaçları eşitler
6 }
7 System.out.println(isimler);

```

DEĞİŞKENLER

isimler = [Can, Ayşe]

cursor = 0 lastRet = -1

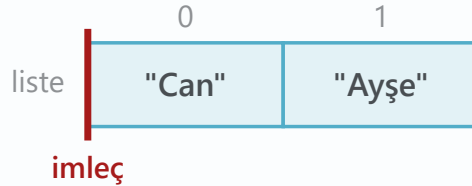
s = "Ali" modCount = 4

expected = 4

ÇIKTI



Kod İzleme: `it.remove()` ile Güvenli Silme



`modCount = 4` · `expectedModCount = 4` ✓

`hasNext()` : cursor 0 $\neq$ size 2 $\rightarrow$ **true**.

Adım 6 / 14

```
1 Iterator<String> it = isimler.iterator();  
2 while (it.hasNext()) {  
3     String s = it.next();  
4     if (s.startsWith("A"))  
5         it.remove(); // sayaçları eşitle  
6 }  
7 System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Can, Ayşe]`

`cursor = 0` `lastRet = -1`

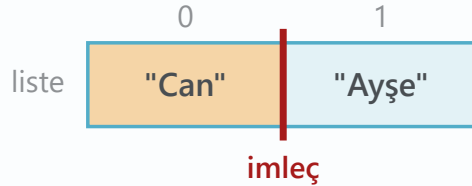
`s = "Ali"` `modCount = 4`

`expected = 4`

ÇIKTI

[Can, Ayşe]

Kod İzleme: `it.remove()` ile Güvenli Silme



`modCount = 4 · expectedModCount = 4 ✓`

`next()` : `4 == 4 ✓` → `"Can"`; `cursor = 1`, `lastRet = 0`.

Adım 7 / 14

```
1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     ▶ String s = it.next();
4     if (s.startsWith("A"))
5         it.remove(); // sayaçları eşitle
6 }
7 System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Can, Ayşe]`

`cursor = 1`

`lastRet = 0`

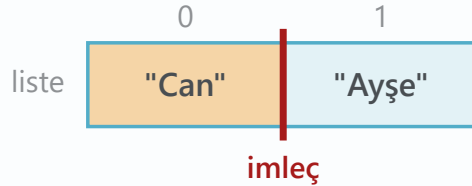
`s = "Can"`

`modCount = 4`

`expected = 4`

ÇIKTI

Kod İzleme: `it.remove()` ile Güvenli Silme



"Can" A ile başlıyor mu? **hayır**.

`modCount = 4 · expectedModCount = 4 ✓`

Adım 8 / 14

```
1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     String s = it.next();
4     if (s.startsWith("A"))
5         it.remove(); // sayaçları eşitler
6 }
7 System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Can, Ayşe]`

`cursor = 1`

`lastRet = 0`

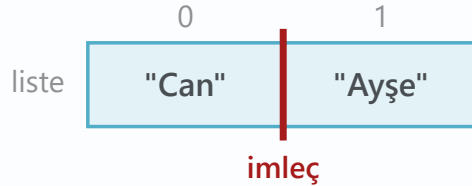
`s = "Can"`

`modCount = 4`

`expected = 4`

ÇIKTI

Kod İzleme: `it.remove()` ile Güvenli Silme



`modCount = 4 · expectedModCount = 4 ✓`

`hasNext()` : cursor 1 $\neq$ size 2 $\rightarrow$ **true**.

Adım 9 / 14

```
1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     String s = it.next();
4     if (s.startsWith("A"))
5         it.remove(); // sayaçları eşitle
6 }
7 System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Can, Ayşe]`

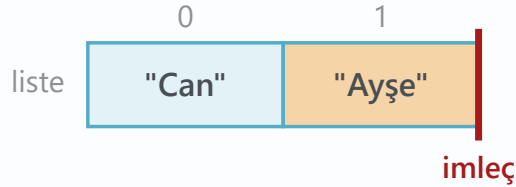
`cursor = 1` `lastRet = 0`

`s = "Can"` `modCount = 4`

`expected = 4`

ÇIKTI

Kod İzleme: `it.remove()` ile Güvenli Silme



$\text{modCount} = 4 \cdot \text{expectedModCount} = 4 \checkmark$

`next()` : $4 == 4 \checkmark \rightarrow$ **"Ayşe"**; `cursor = 2`, `lastRet = 1`.

Adım 10 / 14

```
1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     ▶ String s = it.next();
4     if (s.startsWith("A"))
5         it.remove(); // sayacıları eşitler
6 }
7 System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Can, Ayşe]`

`cursor = 2`

`lastRet = 1`

`s = "Ayşe"`

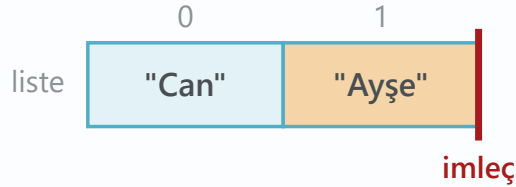
`modCount = 4`

`expected = 4`

ÇIKTI



Kod İzleme: `it.remove()` ile Güvenli Silme



`modCount = 4 · expectedModCount = 4 ✓`

"Ayşe" A ile başlıyor mu? **evet.**

Adım 11 / 14

```
1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     String s = it.next();
4     if (s.startsWith("A"))
5         it.remove(); // sayacı eşitle
6 }
7 System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Can, Ayşe]`

`cursor = 2`

`lastRet = 1`

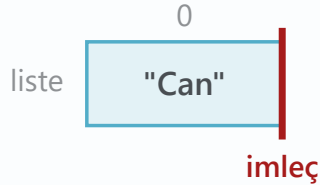
`s = "Ayşe"`

`modCount = 4`

`expected = 4`

ÇIKTI

Kod İzleme: `it.remove()` ile Güvenli Silme



$\text{modCount} = 5 \cdot \text{expectedModCount} = 5 \checkmark$

`it.remove()` : lastRet'teki öge silinir; **cursor = 1** (kayan öge atlanmaz), lastRet = -1, expectedModCount = modCount = 5.

Adım 12 / 14

```

1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     String s = it.next();
4     if (s.startsWith("A"))
5         it.remove(); // sayacıları eşitler
6 }
7 System.out.println(isimler);

```

DEĞİŞKENLER

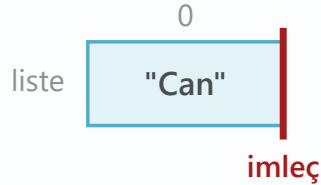
isimler = [Can] cursor = 1

lastRet = -1 s = "Ayşe"

modCount = 5 expected = 5

ÇIKTI

Kod İzleme: `it.remove()` ile Güvenli Silme



`modCount = 5` · `expectedModCount = 5` ✓

`hasNext()` : cursor 1 = size 1 → **false**. Döngü biter; hata yok.

Adım 13 / 14

```
1 Iterator<String> it = isimler.iterator();  
2 while (it.hasNext()) {  
3     String s = it.next();  
4     if (s.startsWith("A"))  
5         it.remove(); // sayaçları eşitle  
6 }  
7 System.out.println(isimler);
```

DEĞİŞKENLER

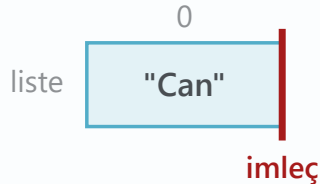
`isimler = [Can]` `cursor = 1`

`lastRet = -1` `s = "Ayşe"`

`modCount = 5` `expected = 5`

ÇIKTI

Kod İzleme: `it.remove()` ile Güvenli Silme



`modCount = 5` · `expectedModCount = 5` ✓

Sonuç **[Can]**: A ile başlayanlar güvenli silindi. İkinci bir `it.remove()` (araya next olmadan) → `IllegalStateException`.

Adım 14 / 14

```
1 Iterator<String> it = isimler.iterator();
2 while (it.hasNext()) {
3     String s = it.next();
4     if (s.startsWith("A"))
5         it.remove(); // sayacıları eşitle
6 }
▶ System.out.println(isimler);
```

DEĞİŞKENLER

`isimler = [Can]` `cursor = 1`

`lastRet = -1` `s = "Ayşe"`

`modCount = 5` `expected = 5`

ÇIKTI

[Can]

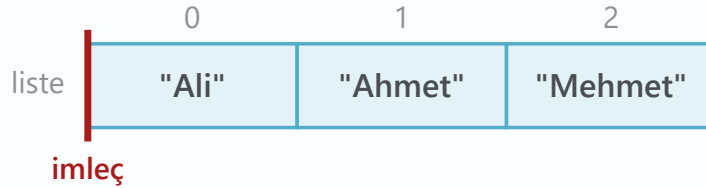
ListIterator Arayüzü

Yalnızca `List` 'lerde bulunur (`list.listIterator()`). Iterator'a ek olarak **iki yönde** gezinir ve gezinirken ekleme, silme, güncelleme yapar. İmleç (cursor) öğelerin **arasında** durur.

Metot	Görevi
<code>hasNext()</code> / <code>next()</code>	ileride öğe var mı? / sonraki öğeyi döndür, imleci ileri al
<code>hasPrevious()</code> / <code>previous()</code>	geride öğe var mı? / önceki öğeyi döndür, imleci geri al
<code>nextIndex()</code> / <code>previousIndex()</code>	<code>next()</code> / <code>previous()</code> çağrılrsa dönecek öğenin indisi (başta 0 / -1)
<code>add(e)</code>	öğeyi imlecin önüne ekler (sonraki <code>next()</code> etkilenmez)
<code>remove()</code>	son <code>next()</code> / <code>previous()</code> ile dönen öğeyi siler
<code>set(e)</code>	son <code>next()</code> / <code>previous()</code> ile dönen öğeyi değiştirir

`add` veya `remove` 'dan hemen sonra, araya yeni bir `next` / `previous` girmeden `remove` / `set` çağrılırsa → `IllegalStateException` .

Kod İzleme: ListIterator Adım Adım



[Ali, Ahmet, Mehmet]

`listIterator()` : imleç başta; `nextIndex` = 0, `previousIndex` = -1. Turuncu: son döndürülen (`lastRet`) öge.

Adım 1 / 7

```
► ListIterator<String> it = isimler.listIterator();
2 it.next();           // "Ali"
3 it.next();           // "Ahmet"
4 it.set("AHMET");     // [Ali, AHMET, Mehmet]
5 it.add("Ayşe");      // [Ali, AHMET, Ayşe, Mehmet]
6 it.previous();       // "Ayşe"
7 it.remove();         // [Ali, AHMET, Mehmet]
```

DEĞİŞKENLER

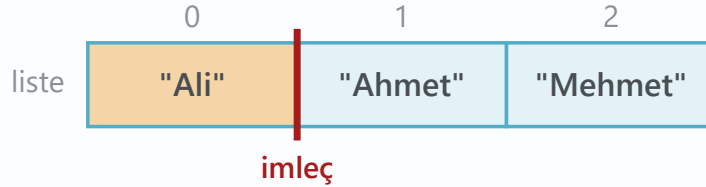
`nextIndex` = 0

`previousIndex` = -1

`lastRet` = -1

`dönen` = -

Kod İzleme: ListIterator Adım Adım



[Ali, Ahmet, Mehmet]

`next()` → "Ali"; imleç bir ileri, lastRet = 0.

Adım 2 / 7

```
1 ListIterator<String> it = isimler.listIterator();
2 it.next();           // "Ali"
3 it.next();           // "Ahmet"
4 it.set("AHMET");     // [Ali, AHMET, Mehmet]
5 it.add("Ayşe");      // [Ali, AHMET, Ayşe, Mehmet]
6 it.previous();       // "Ayşe"
7 it.remove();         // [Ali, AHMET, Mehmet]
```

DEĞİŞKENLER

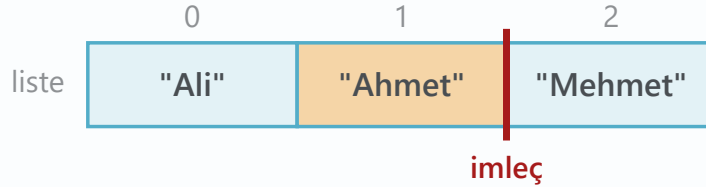
nextIndex = 1

previousIndex = 0

lastRet = 0

dönen = "Ali"

Kod İzleme: ListIterator Adım Adım



[Ali, Ahmet, Mehmet]

`next()` → "Ahmet"; lastRet = 1.

Adım 3 / 7

```
1 ListIterator<String> it = isimler.listIterator();
2 it.next();           // "Ali"
3 ► it.next();         // "Ahmet"
4 it.set("AHMET");     // [Ali, AHMET, Mehmet]
5 it.add("Ayşe");      // [Ali, AHMET, Ayşe, Mehmet]
6 it.previous();       // "Ayşe"
7 it.remove();         // [Ali, AHMET, Mehmet]
```

DEĞİŞKENLER

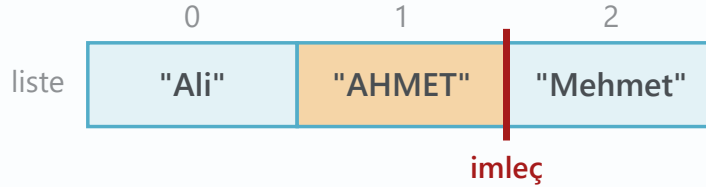
nextIndex = 2

previousIndex = 1

lastRet = 1

dönen = "Ahmet"

Kod İzleme: ListIterator Adım Adım



[Ali, AHMET, Mehmet]

`set("AHMET")` : **lastRet**'teki öge ("Ahmet") değişir; imleç yerinde kalır.

Adım 4 / 7

```
1 ListIterator<String> it = isimler.listIterator();
2 it.next();           // "Ali"
3 it.next();           // "Ahmet"
4 ► it.set("AHMET");    // [Ali, AHMET, Mehmet]
5 it.add("Ayşe");      // [Ali, AHMET, Ayşe, Mehmet]
6 it.previous();        // "Ayşe"
7 it.remove();          // [Ali, AHMET, Mehmet]
```

DEĞİŞKENLER

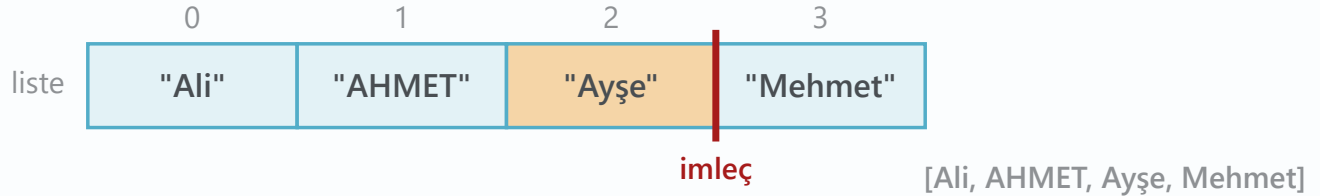
nextIndex = 2

previousIndex = 1

lastRet = 1

dönen = -

Kod İzleme: ListIterator Adım Adım



`add("Ayşe")` : **imlecin önüne** eklenir, imleç eklenenin arkasına geçer (sonraki next yine "Mehmet");
`lastRet = -1`.

Adım 5 / 7

```
1 ListIterator<String> it = isimler.listIterator();
2 it.next();           // "Ali"
3 it.next();           // "Ahmet"
4 it.set("AHMET");     // [Ali, AHMET, Mehmet]
5 ► it.add("Ayşe");    // [Ali, AHMET, Ayşe, Mehmet]
6 it.previous();       // "Ayşe"
7 it.remove();         // [Ali, AHMET, Mehmet]
```

DEĞİŞKENLER

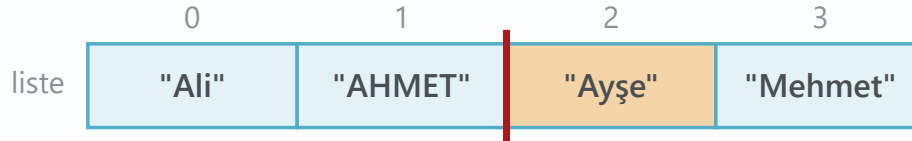
`nextIndex = 3`

`previousIndex = 2`

`lastRet = -1`

`dönen = -`

Kod İzleme: ListIterator Adım Adım



imleç

[Ali, AHMET, Ayşe, Mehmet]

`previous()` → "Ayşe": geri yönde; imleç bir geri, lastRet = 2.

Adım 6 / 7

```
1 ListIterator<String> it = isimler.listIterator();
2 it.next();           // "Ali"
3 it.next();           // "Ahmet"
4 it.set("AHMET");     // [Ali, AHMET, Mehmet]
5 it.add("Ayşe");      // [Ali, AHMET, Ayşe, Mehmet]
6 ► it.previous();     // "Ayşe"
7 it.remove();         // [Ali, AHMET, Mehmet]
```

DEĞİŞKENLER

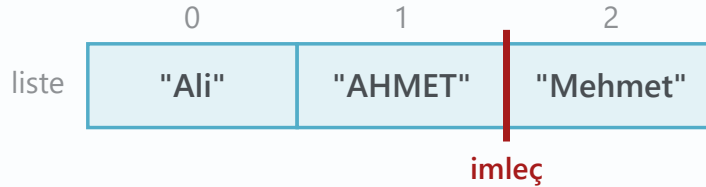
nextIndex = 2

previousIndex = 1

lastRet = 2

dönen = "Ayşe"

Kod İzleme: ListIterator Adım Adım



[Ali, AHMET, Mehmet]

`remove()` : lastRet'teki "Ayşe" silinir; lastRet = -1 (hemen ikinci remove/set → IllegalStateException).

Adım 7 / 7

```
1 ListIterator<String> it = isimler.listIterator();
2 it.next();           // "Ali"
3 it.next();           // "Ahmet"
4 it.set("AHMET");     // [Ali, AHMET, Mehmet]
5 it.add("Ayşe");      // [Ali, AHMET, Ayşe, Mehmet]
6 it.previous();       // "Ayşe"
▶ it.remove();         // [Ali, AHMET, Mehmet]
```

DEĞİŞKENLER

nextIndex = 2

previousIndex = 1

lastRet = -1

dönen = -

ListIterator Demo: İleri ve Geri

```
ListIterator<String> iterator =  
    isimler.listIterator();  
// listeyi ileri yönde gez  
while (iterator.hasNext()) {  
    String isim = iterator.next();  
    System.out.println(isim);  
}  
// imleç artık sonda: ters yönde gez  
while (iterator.hasPrevious()) {  
    String isim = iterator.previous();  
    System.out.println(isim);  
}
```

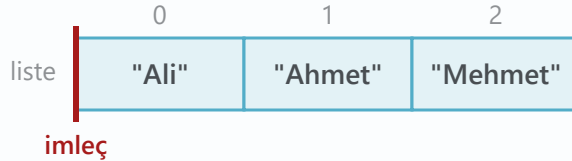
Ali
Ahmet
Mehmet
Mehmet
Ahmet
Ali

İkinci döngü aynı okuyucuyla çalışır: ilk döngü bittiğinde imleç listenin sonundadır.

Doğrudan sondan başlamak için:

```
isimler.listIterator(isimler.size())
```

Kod İzleme: İleri ve Geri Gezinme



`listIterator()` : imleç başta.

Adım 1 / 15

```
► ListIterator<String> iterator =  
► isimler.listIterator();  
3 // listeyi ileri yönde gez  
4 while (iterator.hasNext()) {  
5     String isim = iterator.next();  
6     System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
9 while (iterator.hasPrevious()) {  
10    String isim = iterator.previous();  
11    System.out.println(isim);  
12 }
```

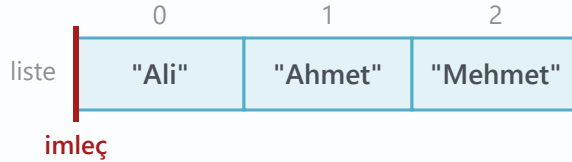
DEĞİŞKENLER

cursor = 0

isim = -

ÇIKTI

Kod İzleme: İleri ve Geri Gezinme



`hasNext()` → true (cursor 0 < 3).

Adım 2 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
▶ while (iterator.hasNext()) {  
4     String isim = iterator.next();  
5     System.out.println(isim);  
6 }  
7 // imleç artık sonda: ters yönde gez  
8 while (iterator.hasPrevious()) {  
9     String isim = iterator.previous();  
10    System.out.println(isim);  
11 }  
12 }
```

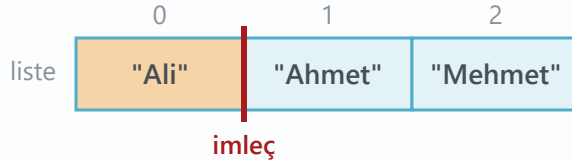
DEĞİŞKENLER

cursor = 0

isim = -

ÇIKTI

Kod İzleme: İleri ve Geri Gezinme



`next()` → "Ali", yazdırılır; imleç ileri.

Adım 3 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
4 while (iterator.hasNext()) {  
5     ▶ String isim = iterator.next();  
6     ▶ System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
9 while (iterator.hasPrevious()) {  
10    String isim = iterator.previous();  
11    System.out.println(isim);  
12 }
```

DEĞİŞKENLER

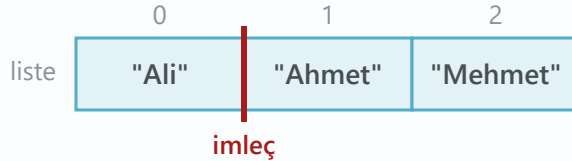
cursor = 1

isim = "Ali"

ÇIKTI

Ali

Kod İzleme: İleri ve Geri Gezinme



`hasNext()` → true (cursor 1 < 3).

Adım 4 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
4  
5 while (iterator.hasNext()) {  
6     String isim = iterator.next();  
7     System.out.println(isim);  
8 }  
9 // imleç artık sonda: ters yönde gez  
10 while (iterator.hasPrevious()) {  
11     String isim = iterator.previous();  
12     System.out.println(isim);  
13 }
```

DEĞİŞKENLER

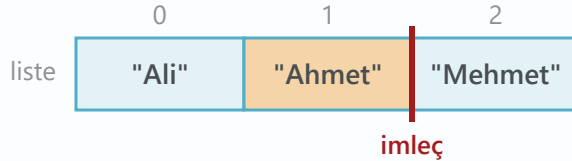
cursor = 1

isim = "Ali"

ÇIKTI

Ali

Kod İzleme: İleri ve Geri Gezinme



`next()` → "Ahmet", yazdırılır; imleç ileri.

Adım 5 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
4 while (iterator.hasNext()) {  
5     ▶ String isim = iterator.next();  
6     ▶ System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
9 while (iterator.hasPrevious()) {  
10    String isim = iterator.previous();  
11    System.out.println(isim);  
12 }
```

DEĞİŞKENLER

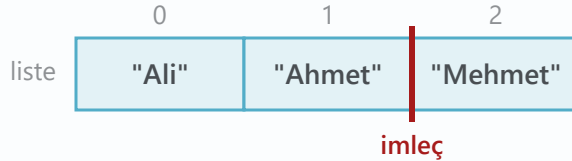
`cursor = 2`

`isim = "Ahmet"`

ÇIKTI

```
Ali  
Ahmet
```

Kod İzleme: İleri ve Geri Gezinme



`hasNext()` → true (cursor 2 < 3).

Adım 6 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
▶ while (iterator.hasNext()) {  
5     String isim = iterator.next();  
6     System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
9 while (iterator.hasPrevious()) {  
10    String isim = iterator.previous();  
11    System.out.println(isim);  
12 }
```

DEĞİŞKENLER

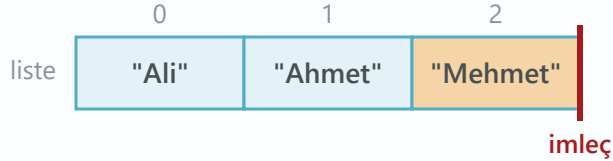
cursor = 2

isim = "Ahmet"

ÇIKTI

Ali
Ahmet

Kod İzleme: İleri ve Geri Gezinme



`next()` → "Mehmet", yazdırılır; imleç ileri.

Adım 7 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
4 while (iterator.hasNext()) {  
5     ▶ String isim = iterator.next();  
6     ▶ System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
9 while (iterator.hasPrevious()) {  
10    String isim = iterator.previous();  
11    System.out.println(isim);  
12 }
```

DEĞİŞKENLER

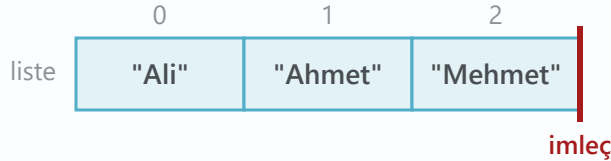
`cursor = 3`

`isim = "Mehmet"`

ÇIKTI

```
Ali  
Ahmet  
Mehmet
```

Kod İzleme: İleri ve Geri Gezinme



`hasNext()` → false: imleç sonda. İlk döngü biter; **aynı iterator** ile devam.

Adım 8 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
▶ while (iterator.hasNext()) {  
5     String isim = iterator.next();  
6     System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
9 while (iterator.hasPrevious()) {  
10    String isim = iterator.previous();  
11    System.out.println(isim);  
12 }
```

DEĞİŞKENLER

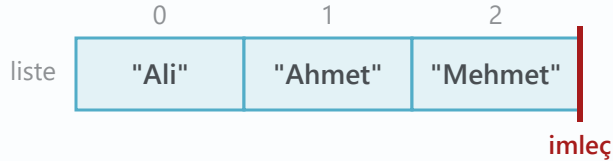
cursor = 3

isim = "Mehmet"

ÇIKTI

```
Ali  
Ahmet  
Mehmet
```

Kod İzleme: İleri ve Geri Gezinme



`hasPrevious()` → true (cursor 3 > 0).

Adım 9 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
4 while (iterator.hasNext()) {  
5     String isim = iterator.next();  
6     System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
9 while (iterator.hasPrevious()) {  
10    String isim = iterator.previous();  
11    System.out.println(isim);  
12 }
```

DEĞİŞKENLER

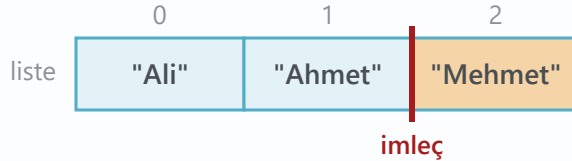
cursor = 3

isim = "Mehmet"

ÇIKTI

```
Ali  
Ahmet  
Mehmet
```

Kod İzleme: İleri ve Geri Gezinme



`previous()` → "Mehmet", yazdırılır; imleç geri.

Adım 10 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
4 while (iterator.hasNext()) {  
5     String isim = iterator.next();  
6     System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
9 while (iterator.hasPrevious()) {  
10     ▶ String isim = iterator.previous();  
11     ▶ System.out.println(isim);  
12 }
```

DEĞİŞKENLER

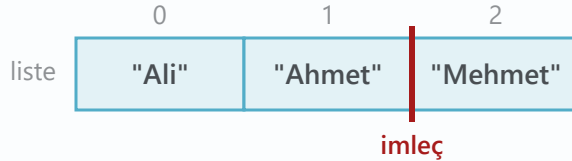
`cursor = 2`

`isim = "Mehmet"`

ÇIKTI

```
Ali  
Ahmet  
Mehmet  
Mehmet
```

Kod İzleme: İleri ve Geri Gezinme



`hasPrevious()` → true (cursor 2 > 0).

Adım 11 / 15

```
1 ListIterator<String> iterator =
2     isimler.listIterator();
3 // listeyi ileri yönde gez
4 while (iterator.hasNext()) {
5     String isim = iterator.next();
6     System.out.println(isim);
7 }
8 // imleç artık sonda: ters yönde gez
9 ► while (iterator.hasPrevious()) {
10     String isim = iterator.previous();
11     System.out.println(isim);
12 }
```

DEĞİŞKENLER

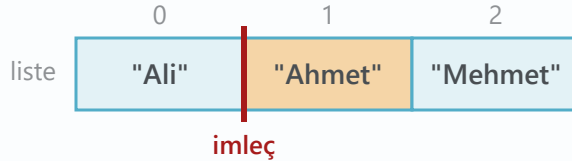
`cursor = 2`

`isim = "Mehmet"`

ÇIKTI

```
Ali
Ahmet
Mehmet
Mehmet
```

Kod İzleme: İleri ve Geri Gezinme



`previous()` → "Ahmet", yazdırılır; imleç geri.

Adım 12 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
4 while (iterator.hasNext()) {  
5     String isim = iterator.next();  
6     System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
9 while (iterator.hasPrevious()) {  
10     ▶ String isim = iterator.previous();  
11     ▶ System.out.println(isim);  
12 }
```

DEĞİŞKENLER

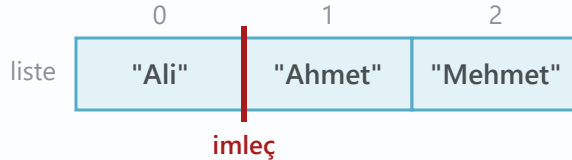
`cursor = 1`

`isim = "Ahmet"`

ÇIKTI

```
Ali  
Ahmet  
Mehmet  
Mehmet  
Ahmet
```

Kod İzleme: İleri ve Geri Gezinme



hasPrevious() → true (cursor 1 > 0).

Adım 13 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
4 while (iterator.hasNext()) {  
5     String isim = iterator.next();  
6     System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
▶ while (iterator.hasPrevious()) {  
10     String isim = iterator.previous();  
11     System.out.println(isim);  
12 }
```

DEĞİŞKENLER

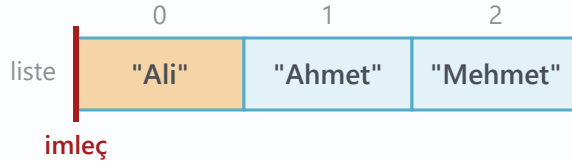
cursor = 1

isim = "Ahmet"

ÇIKTI

```
Ali  
Ahmet  
Mehmet  
Mehmet  
Ahmet
```

Kod İzleme: İleri ve Geri Gezinme



`previous()` → "Ali", yazdırılır; imleç geri.

Adım 14 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
4 while (iterator.hasNext()) {  
5     String isim = iterator.next();  
6     System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
9 while (iterator.hasPrevious()) {  
10     ▶ String isim = iterator.previous();  
11     ▶ System.out.println(isim);  
12 }
```

DEĞİŞKENLER

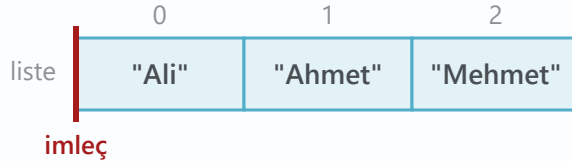
`cursor = 0`

`isim = "Ali"`

ÇIKTI

```
Ali  
Ahmet  
Mehmet  
Mehmet  
Ahmet  
Ali
```

Kod İzleme: İleri ve Geri Gezinme



`hasPrevious()` → false: imleç başa döndü. Çıktı: ileri 3 + geri 3 satır.

Adım 15 / 15

```
1 ListIterator<String> iterator =  
2     isimler.listIterator();  
3 // listeyi ileri yönde gez  
4 while (iterator.hasNext()) {  
5     String isim = iterator.next();  
6     System.out.println(isim);  
7 }  
8 // imleç artık sonda: ters yönde gez  
▶ while (iterator.hasPrevious()) {  
10     String isim = iterator.previous();  
11     System.out.println(isim);  
12 }
```

DEĞİŞKENLER

`cursor = 0`

`isim = "Ali"`

ÇIKTI

```
Ali  
Ahmet  
Mehmet  
Mehmet  
Ahmet  
Ali
```

List Arayüzünü Uygulayan Sınıflar

ArrayList

Büyüyeabilen dizi. Hızlı indisle erişim; genel amaçlı varsayılan seçim.

LinkedList

Çift yönlü bağlı liste. Uçlarda hızlı ekleme/silme; indisle erişim yavaş.

Vector

ArrayList'in eski, metotları senkronize (synchronized) sürümü.

Stack

Vector'dan türeyen yığın: push, pop, peek (LIFO).

CopyOnWriteArrayList

Her yazmada kopyalayan, çok iş parçacıklı okumaya uygun liste.

Arrays.asList()

Bir diziyi sabit boyutlu bir List görünümüne çevirir.

ArrayList: Büyüeyebilen Dizi

- Öğeleri içeride bir dizide tutar. **Kapasite** (dizinin boyu) $\geq$ **size** (öge sayısı).
- Dizi dolunca $\sim 1,5$ kat büyük yeni bir dizi oluşturulur ve öğeler kopyalanır ($O(n)$). Bu seyrek olduğundan sona ekleme **amortize $O(1)$** 'dir.
- Silmede kendiliğinden küçülmez; `trimToSize()` ile kapasite size'a indirilebilir.

`add(x)``remove(size - 1)``trimToSize()`

sıfırla



size = 3 · kapasite = 4 · toplam kopya = 0 (0 eklemede)

Gerçek Java: boş `new ArrayList<>()` ilk eklemede 10 kapasite alır; yeni kapasite = eski + eski/2. Demoda 4'ten başlanıyor.

Kod İzleme: ArrayList `add(e)` ve Büyüme



Kapasite 4, boyut 3 (demo; gerçek Java boş listede ilk eklemede 10 ile başlar). Sırayla D, E, F, G eklenecek.

Adım 1 / 13

```
▶ public boolean add(E e) {  
2   if (boyut == dizi.length) {  
3       int yeni = dizi.length + dizi.length / 2;  
4       dizi = Arrays.copyOf(dizi, yeni);  
5   }  
6   dizi[boyut++] = e;  
7   return true;  
8 }
```

DEĞİŞKENLER

e = -

boyut = 3

dizi.length = 4

kopyalanan = 0

ÇAĞRI YIĞINI

main()

Kod İzleme: ArrayList `add(e)` ve Büyüme



`add(D)` : boyut 3 == kapasite 4? **hayır**.

Adım 2 / 13

```
1 public boolean add(E e) {  
2     if (boyut == dizi.length) {  
3         int yeni = dizi.length + dizi.length / 2;  
4         dizi = Arrays.copyOf(dizi, yeni);  
5     }  
6     dizi[boyut++] = e;  
7     return true;  
8 }
```

DEĞİŞKENLER

`e = D`

`boyut = 3`

`dizi.length = 4`

`yeni = -`

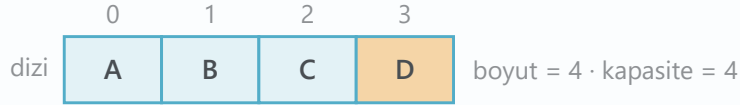
`kopyalanan = 0`

ÇAĞRI YİĞİNİ

`add(D)`

`main()`

Kod İzleme: ArrayList `add(e)` ve Büyüme



`dizi[3] = D`, boyut = 4; true döner.

Adım 3 / 13

```
1 public boolean add(E e) {  
2     if (boyut == dizi.length) {  
3         int yeni = dizi.length + dizi.length / 2;  
4         dizi = Arrays.copyOf(dizi, yeni);  
5     }  
6     ▶ dizi[boyut++] = e;  
7     ▶ return true;  
8 }
```

DEĞİŞKENLER

e = D

boyut = 4

dizi.length = 4

yeni = -

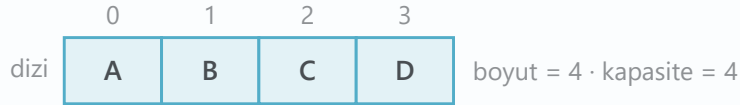
kopyalanan = 0

ÇAĞRI YİĞİNİ

add(D)

main()

Kod İzleme: ArrayList `add(e)` ve Büyüme



`add(E)` : boyut 4 == kapasite 4? **evet** → **dizi dolu**.

Adım 4 / 13

```
1 public boolean add(E e) {  
2     if (boyut == dizi.length) {  
3         int yeni = dizi.length + dizi.length / 2;  
4         dizi = Arrays.copyOf(dizi, yeni);  
5     }  
6     dizi[boyut++] = e;  
7     return true;  
8 }
```

DEĞİŞKENLER

`e = E`

`boyut = 4`

`dizi.length = 4`

`yeni = -`

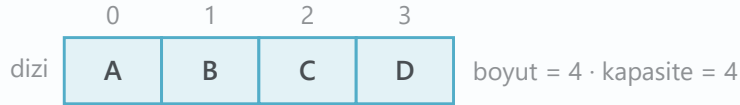
`kopyalanan = 0`

ÇAĞRI YİĞİNİ

`add(E)`

`main()`

Kod İzleme: ArrayList `add(e)` ve Büyüme



yeni = 4 + 4 / 2 = 6: yaklaşık 1,5 kat (Java: `eski + eski >> 1`).

Adım 5 / 13

```
1 public boolean add(E e) {  
2   if (boyut == dizi.length) {  
3     ▶ int yeni = dizi.length + dizi.length / 2;  
4     dizi = Arrays.copyOf(dizi, yeni);  
5   }  
6   dizi[boyut++] = e;  
7   return true;  
8 }
```

DEĞİŞKENLER

e = E

boyut = 4

dizi.length = 4

yeni = 6

kopyalanan = 0

ÇAĞRI YİĞİNİ

add(E)

main()

Kod İzleme: ArrayList add(e) ve Büyüme



`Arrays.copyOf` : 6 yuvalı yeni dizi, 4 öge tek tek kopyalanır → bu ekleme **O(n)**. Eski dizi çöpe gider.

Adım 6 / 13

```

1 public boolean add(E e) {
2     if (boyut == dizi.length) {
3         int yeni = dizi.length + dizi.length / 2;
4         dizi = Arrays.copyOf(dizi, yeni);
5     }
6     dizi[boyut++] = e;
7     return true;
8 }

```

DEĞİŞKENLER

e = E

boyut = 4

dizi.length = 4

yeni = 6

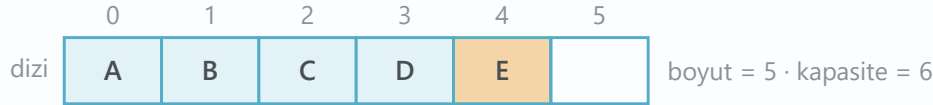
kopyalanan = 4

ÇAĞRI YİĞİNİ

add(E)

main()

Kod İzleme: ArrayList add(e) ve Büyüme



dizi[4] = E, boyut = 5; true döner.

Adım 7 / 13

```
1 public boolean add(E e) {  
2     if (boyut == dizi.length) {  
3         int yeni = dizi.length + dizi.length / 2;  
4         dizi = Arrays.copyOf(dizi, yeni);  
5     }  
6     ▶ dizi[boyut++] = e;  
7     ▶ return true;  
8 }
```

DEĞİŞKENLER

e = E

boyut = 5

dizi.length = 6

yeni = -

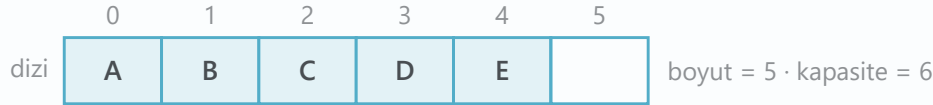
kopyalanan = 4

ÇAĞRI YIĞINI

add(E)

main()

Kod İzleme: ArrayList `add(e)` ve Büyüme



`add(F)` : boyut 5 == kapasite 6? **hayır**.

Adım 8 / 13

```
1 public boolean add(E e) {  
2     if (boyut == dizi.length) {  
3         int yeni = dizi.length + dizi.length / 2;  
4         dizi = Arrays.copyOf(dizi, yeni);  
5     }  
6     dizi[boyut++] = e;  
7     return true;  
8 }
```

DEĞİŞKENLER

`e = F`

`boyut = 5`

`dizi.length = 6`

`yeni = -`

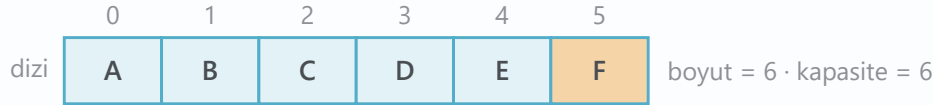
`kopyalanan = 4`

ÇAĞRI YİĞİNİ

`add(F)`

`main()`

Kod İzleme: ArrayList add(e) ve Büyüme



dizi[5] = F, boyut = 6; true döner.

Adım 9 / 13

```
1 public boolean add(E e) {  
2     if (boyut == dizi.length) {  
3         int yeni = dizi.length + dizi.length / 2;  
4         dizi = Arrays.copyOf(dizi, yeni);  
5     }  
6     ▶ dizi[boyut++] = e;  
7     ▶ return true;  
8 }
```

DEĞİŞKENLER

e = F

boyut = 6

dizi.length = 6

yeni = -

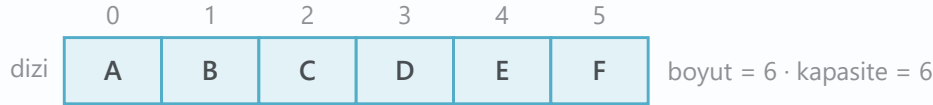
kopyalanan = 4

ÇAĞRI YIĞINI

add(F)

main()

Kod İzleme: ArrayList `add(e)` ve Büyüme



`add(G)` : boyut 6 == kapasite 6? **evet** → **dizi dolu**.

Adım 10 / 13

```
1 public boolean add(E e) {  
2     if (boyut == dizi.length) {  
3         int yeni = dizi.length + dizi.length / 2;  
4         dizi = Arrays.copyOf(dizi, yeni);  
5     }  
6     dizi[boyut++] = e;  
7     return true;  
8 }
```

DEĞİŞKENLER

`e = G`

`boyut = 6`

`dizi.length = 6`

`yeni = -`

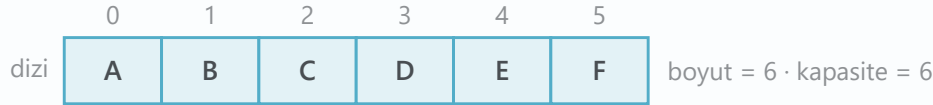
`kopyalanan = 4`

ÇAĞRI YIĞINI

`add(G)`

`main()`

Kod İzleme: ArrayList `add(e)` ve Büyüme



yeni = 6 + 6 / 2 = 9: yaklaşık 1,5 kat (Java: `eski + eski >> 1`).

Adım 11 / 13

```
1 public boolean add(E e) {  
2     if (boyut == dizi.length) {  
3         ▶ int yeni = dizi.length + dizi.length / 2;  
4         dizi = Arrays.copyOf(dizi, yeni);  
5     }  
6     dizi[boyut++] = e;  
7     return true;  
8 }
```

DEĞİŞKENLER

e = G

boyut = 6

dizi.length = 6

yeni = 9

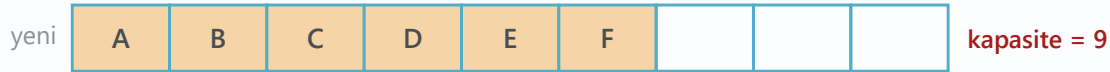
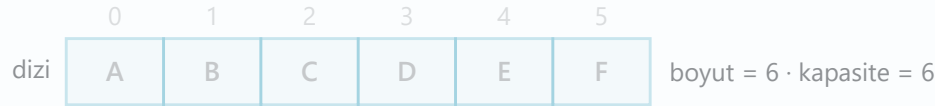
kopyalanan = 4

ÇAĞRI YİĞİNİ

add(G)

main()

Kod İzleme: ArrayList add(e) ve Büyüme



`Arrays.copyOf` : 9 yuvalı yeni dizi, 6 öge tek tek kopyalanır → bu ekleme **O(n)**. Eski dizi çöpe gider.

Adım 12 / 13

```

1 public boolean add(E e) {
2     if (boyut == dizi.length) {
3         int yeni = dizi.length + dizi.length / 2;
4         dizi = Arrays.copyOf(dizi, yeni);
5     }
6     dizi[boyut++] = e;
7     return true;
8 }

```

DEĞİŞKENLER

e = G

boyut = 6

dizi.length = 6

yeni = 9

kopyalanan = 10

ÇAĞRI YİĞİNİ

add(G)

main()

Kod İzleme: ArrayList add(e) ve Büyüme



`dizi[6] = G`, boyut = 7; true döner. Kapasite her seferinde 1,5 katına çıktığından büyütme giderek seyrekleşir: n eklemede toplam kopya $O(n)$ → ekleme **amortize $O(1)$** .

Adım 13 / 13

```

1 public boolean add(E e) {
2     if (boyut == dizi.length) {
3         int yeni = dizi.length + dizi.length / 2;
4         dizi = Arrays.copyOf(dizi, yeni);
5     }
6     ▶ dizi[boyut++] = e;
7     ▶ return true;
8 }

```

DEĞİŞKENLER

e = G

boyut = 7

dizi.length = 9

yeni = -

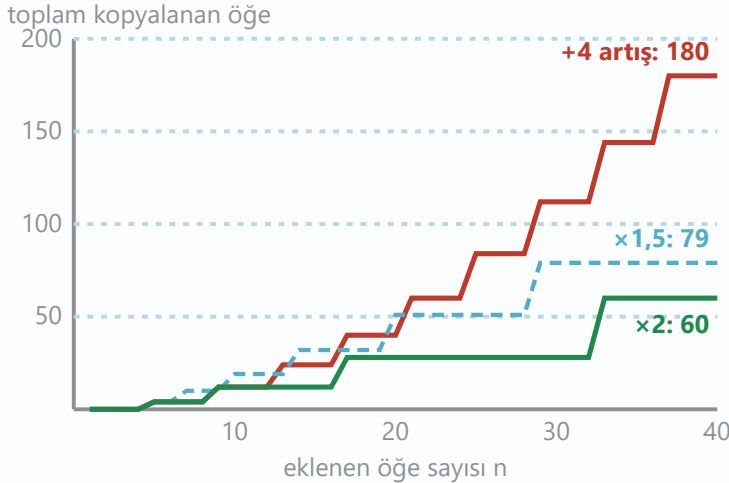
kopyalanan = 10

ÇAĞRI YIĞINI

add(G)

main()

Amortize Analiz: Neden Katlayarak Büyütülür?



Grafik, kapasite 4'ten başlayan bir dinamik dizide 40 ekleme için gerçekten sayılmış kopyalardır.

- **Katlayarak büyütme ($\times 2$):** n eklemede kopyalar $4 + 8 + 16 + \dots < 2n \rightarrow$ işlem başına **amortize $O(1)$** .
- Java'nın $\times 1,5$ 'i de geometrik büyümedir: toplam $\leq 3n$, yine $O(1)$ amortize.
- **Sabit artış ($+c$):** kopyalar $c + 2c + 3c + \dots \approx n^2/(2c) \rightarrow$ işlem başına **$O(n)$** .
- Goodrich–Tamassia: "bankacı" yöntemi: her ucuz eklemeye birkaç "jeton" yatırılır, pahalı büyütme o jetonlar öder.

ArrayList Kullanımı ve Maliyetleri

```
ArrayList<String> liste =  
    new ArrayList<>();  
liste.add("Ali");        // [Ali]  
liste.add("Veli");       // [Ali, Veli]  
liste.add(0, "Can");     // [Can, Ali, Veli]  
liste.get(1);            // "Ali"  
liste.remove(0);         // [Ali, Veli]  
liste.size();            // 2
```

İşlem	Maliyet
<code>get(i)</code> , <code>set(i, e)</code>	$O(1)$
<code>add(e)</code> (sona)	amortize $O(1)$
<code>add(i, e)</code> , <code>remove(i)</code>	$O(n)$
<code>contains</code> , <code>indexOf</code>	$O(n)$
<code>size()</code>	$O(1)$

Diziye göre artıları

Boyut kendiliğinden büyür; ekleme, silme, arama, sıralama hazır metotlarla.

Diziye göre eksileri

Yalnızca nesne tutar (kutulama maliyeti); her erişimde sınır denetimi ve metot çağrısı yükü.

Kod İzleme: ArrayList İşlemlerinin Maliyeti

boş dizi {}

Boş `new ArrayList<>()` : iç dizi başta boş; ilk eklemede 10 kapasiteli dizi ayrılır.

Adım 1 / 9

```
▶ ArrayList<String> liste = new ArrayList<>();
```

```
2 liste.add("Ali");  
3 liste.add("Veli");  
4 liste.add(0, "Can");  
5 String s = liste.get(1);  
6 liste.remove(0);  
7 int n = liste.size();
```

DEĞİŞKENLER

liste = []

size = 0

kapasite = 0

kaydırma = 0

Kod İzleme: ArrayList İşlemlerinin Maliyeti



İlk `add` : 10'luk dizi oluşturulur, Ali indis 0'a. Sona ekleme: kaydırma yok.

Adım 2 / 9

```
1 ArrayList<String> liste = new ArrayList<>();  
2 ► liste.add("Ali");  
3 liste.add("Veli");  
4 liste.add(0, "Can");  
5 String s = liste.get(1);  
6 liste.remove(0);  
7 int n = liste.size();
```

DEĞİŞKENLER

liste = [Ali]

size = 1

kapasite = 10

kaydırma = 0

Kod İzleme: ArrayList İşlemlerinin Maliyeti



Veli indis 1'e (amortize $O(1)$).

Adım 3 / 9

```
1 ArrayList<String> liste = new ArrayList<>();
2 liste.add("Ali");
3 ► liste.add("Veli");
4 liste.add(0, "Can");
5 String s = liste.get(1);
6 liste.remove(0);
7 int n = liste.size();
```

DEĞİŞKENLER

liste = [Ali, Veli] size = 2

kapasite = 10

kaydırma = 0

Kod İzleme: ArrayList İşlemlerinin Maliyeti



`add(0, ...)` : önce indis 0'dan sonraki 2 öge birer sağa kaydırılır (sağdan başlayarak).

Adım 4 / 9

```
1 ArrayList<String> liste = new ArrayList<>();
2 liste.add("Ali");
3 liste.add("Veli");
4 ► liste.add(0, "Can");
5 String s = liste.get(1);
6 liste.remove(0);
7 int n = liste.size();
```

DEĞİŞKENLER

liste = [Ali, Veli] size = 2

kapasite = 10 kaydırma = 2

Kod İzleme: ArrayList İşlemlerinin Maliyeti



Boşalan indis 0'a Can yazılır. Başa ekleme n kaydırma → **O(n)**.

Adım 5 / 9

```
1 ArrayList<String> liste = new ArrayList<>();
2 liste.add("Ali");
3 liste.add("Veli");
4 ► liste.add(0, "Can");
5 String s = liste.get(1);
6 liste.remove(0);
7 int n = liste.size();
```

DEĞİŞKENLER

liste = [Can, Ali, Veli]

size = 3 kapasite = 10

kaydırma = 2

Kod İzleme: ArrayList İşlemlerinin Maliyeti



`get(1)` : adres hesabıyla doğrudan erişim → **O(1)**, kaydırma yok.

Adım 6 / 9

```
1 ArrayList<String> liste = new ArrayList<>();
2 liste.add("Ali");
3 liste.add("Veli");
4 liste.add(0, "Can");
5 ▶ String s = liste.get(1);
6 liste.remove(0);
7 int n = liste.size();
```

DEĞİŞKENLER

liste = [Can, Ali, Veli]

size = 3 kapasite = 10

kaydırma = 2 s = "Ali"

Kod İzleme: ArrayList İşlemlerinin Maliyeti



`remove(0)` : Can silinecek...

Adım 7 / 9

```
1 ArrayList<String> liste = new ArrayList<>();
2 liste.add("Ali");
3 liste.add("Veli");
4 liste.add(0, "Can");
5 String s = liste.get(1);
▶ liste.remove(0);
7 int n = liste.size();
```

DEĞİŞKENLER

liste = [Can, Ali, Veli]

size = 3 kapasite = 10

kaydırma = 2 s = "Ali"

Kod İzleme: ArrayList İşlemlerinin Maliyeti



...sonraki 2 öge sola kayar, son hücre null yapılır. Kapasite küçülmez.

Adım 8 / 9

```
1 ArrayList<String> liste = new ArrayList<>();
2 liste.add("Ali");
3 liste.add("Veli");
4 liste.add(0, "Can");
5 String s = liste.get(1);
▶ liste.remove(0);
7 int n = liste.size();
```

DEĞİŞKENLER

liste = [Ali, Veli] size = 2

kapasite = 10

kaydırma = 4

s = "Ali"

Kod İzleme: ArrayList İşlemlerinin Maliyeti



`size()` = 2: sayaç alanı okunur, $O(1)$. Toplam 4 kaydırmanın hepsi baştaki işlemlerden.

Adım 9 / 9

```
1 ArrayList<String> liste = new ArrayList<>();
2 liste.add("Ali");
3 liste.add("Veli");
4 liste.add(0, "Can");
5 String s = liste.get(1);
6 liste.remove(0);
▶ int n = liste.size();
```

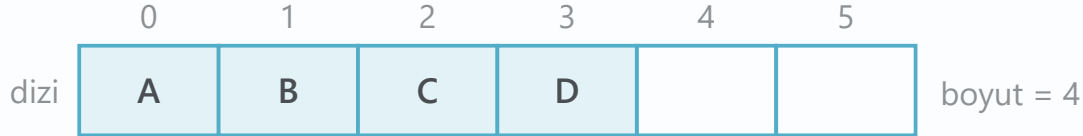
DEĞİŞKENLER

liste = [Ali, Veli] size = 2

kapasite = 10 kaydırma = 4

s = "Ali" n = 2

Kod İzleme: ArrayList add(i, e)



[A, B, C, D], kapasite 6. `add(1, "X")` : X, indis 1'e girecek; B, C, D bir sağa kaymalı.

Adım 1 / 9

```
► public void add(int i, E e) {  
2  Objects.checkIndex(i, boyut + 1);  
3  if (boyut == dizi.length) buyut();  
4  for (int k = boyut; k > i; k--)  
5      dizi[k] = dizi[k - 1];  
6  dizi[i] = e;  
7  boyut++;  
8 }
```

DEĞİŞKENLER

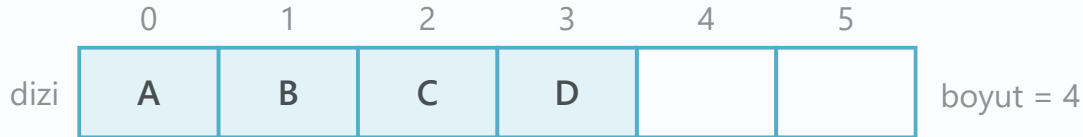
i = 1 e = X k = - boyut = 4

dizi = [A, B, C, D, null, null]

ÇAĞRI YIĞINI

main()

Kod İzleme: ArrayList add(i, e)



$0 \leq i < \text{boyut}$ (4) → geçerli indis (yoksa IndexOutOfBoundsException).

Adım 2 / 9

```
1 public void add(int i, E e) {  
2     Objects.checkIndex(i, boyut + 1);  
3     if (boyut == dizi.length) boyut();  
4     for (int k = boyut; k > i; k--)  
5         dizi[k] = dizi[k - 1];  
6     dizi[i] = e;  
7     boyut++;  
8 }
```

DEĞİŞKENLER

i = 1

e = X

k = -

boyut = 4

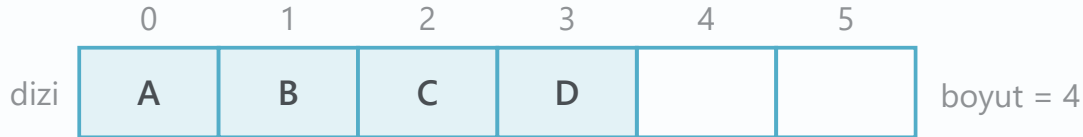
dizi = [A, B, C, D, null, null]

ÇAĞRI YIĞINI

add(1, "X")

main()

Kod İzleme: ArrayList add(i, e)



boyut 4 < kapasite 6 → büyötmeye gerek yok.

Adım 3 / 9

```
1 public void add(int i, E e) {  
2   Objects.checkIndex(i, boyut + 1);  
3   if (boyut == dizi.length) buyut();  
4   for (int k = boyut; k > i; k--)  
5     dizi[k] = dizi[k - 1];  
6   dizi[i] = e;  
7   boyut++;  
8 }
```

DEĞİŞKENLER

i = 1 e = X k = - boyut = 4

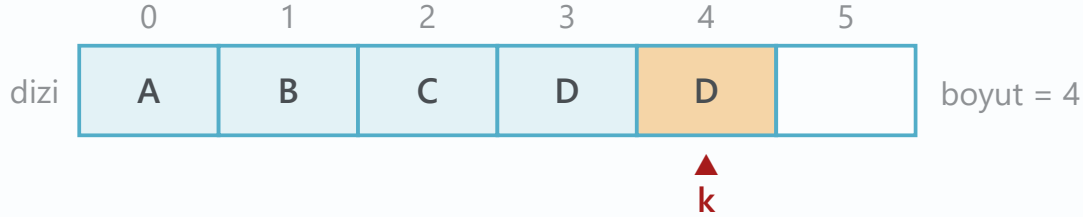
dizi = [A, B, C, D, null, null]

ÇAĞRI YIĞINI

add(1, "X")

main()

Kod İzleme: ArrayList add(i, e)



$k = 4 > 1$: dizi[4] = dizi[3] → D sağa kopyalanır. **Sağdan** başlanır ki üzerine yazılsın.

Adım 4 / 9

```

1 public void add(int i, E e) {
2   Objects.checkIndex(i, boyut + 1);
3   if (boyut == dizi.length) buyut();
4   ▶ for (int k = boyut; k > i; k--)
5     ▶   dizi[k] = dizi[k - 1];
6   dizi[i] = e;
7   boyut++;
8 }

```

DEĞİŞKENLER

i = 1

e = X

k = 4

boyut = 4

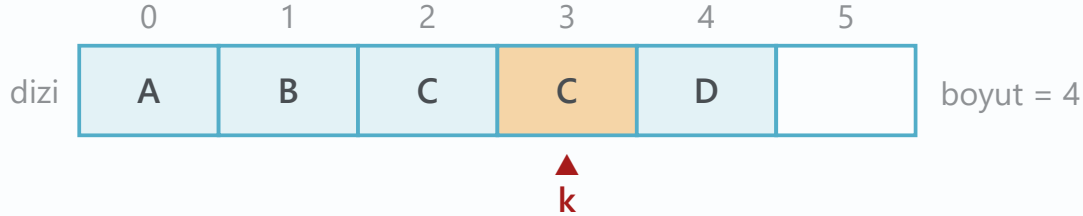
dizi = [A, B, C, D, D, null]

ÇAĞRI YIĞINI

add(1, "X")

main()

Kod İzleme: ArrayList add(i, e)



$k = 3 > 1$: dizi[3] = dizi[2] → C sağa kopyalanır. **Sağdan** başlanır ki üzerine yazılsın.

Adım 5 / 9

```
1 public void add(int i, E e) {  
2     Objects.checkIndex(i, boyut + 1);  
3     if (boyut == dizi.length) buyut();  
4     for (int k = boyut; k > i; k--)  
5         dizi[k] = dizi[k - 1];  
6     dizi[i] = e;  
7     boyut++;  
8 }
```

DEĞİŞKENLER

i = 1

e = X

k = 3

boyut = 4

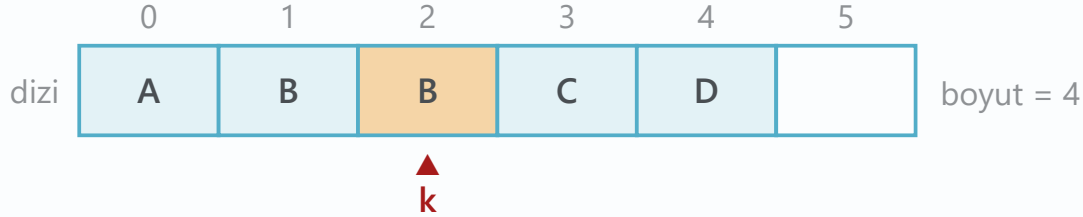
dizi = [A, B, C, C, D, null]

ÇAĞRI YIĞINI

add(1, "X")

main()

Kod İzleme: ArrayList add(i, e)



$k = 2 > 1$: dizi[2] = dizi[1] → B sağa kopyalanır. **Sağdan** başlanır ki üzerine yazılsın.

Adım 6 / 9

```

1 public void add(int i, E e) {
2   Objects.checkIndex(i, boyut + 1);
3   if (boyut == dizi.length) buyut();
4   ▶ for (int k = boyut; k > i; k--)
5     ▶   dizi[k] = dizi[k - 1];
6   dizi[i] = e;
7   boyut++;
8 }

```

DEĞİŞKENLER

i = 1

e = X

k = 2

boyut = 4

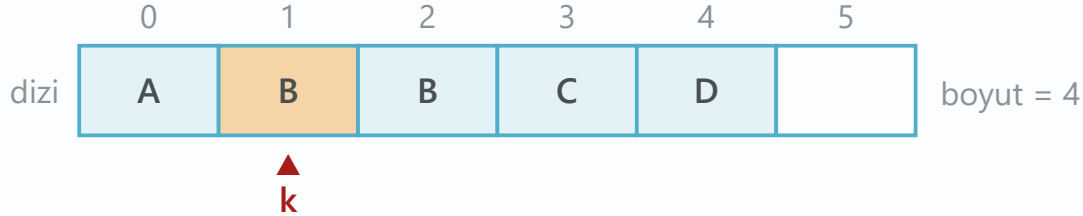
dizi = [A, B, B, C, D, null]

ÇAĞRI YIĞINI

add(1, "X")

main()

Kod İzleme: ArrayList add(i, e)



$k = 1 > 1$ değil → döngü biter. dizi[1]'de B'nin eski kopyası duruyor.

Adım 7 / 9

```

1 public void add(int i, E e) {
2   Objects.checkIndex(i, boyut + 1);
3   if (boyut == dizi.length) buyut();
4   ► for (int k = boyut; k > i; k--)
5     dizi[k] = dizi[k - 1];
6   dizi[i] = e;
7   boyut++;
8 }

```

DEĞİŞKENLER

i = 1

e = X

k = 1

boyut = 4

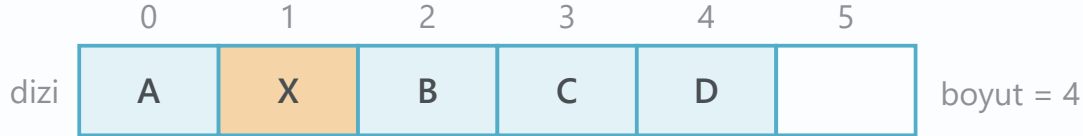
dizi = [A, B, B, C, D, null]

ÇAĞRI YIĞINI

add(1, "X")

main()

Kod İzleme: ArrayList add(i, e)



`dizi[1] = X` : açılan yere yazılır.

Adım 8 / 9

```
1 public void add(int i, E e) {  
2     Objects.checkIndex(i, boyut + 1);  
3     if (boyut == dizi.length) buyut();  
4     for (int k = boyut; k > i; k--)  
5         dizi[k] = dizi[k - 1];  
6     ► dizi[i] = e;  
7     boyut++;  
8 }
```

DEĞİŞKENLER

i = 1

e = X

k = 1

boyut = 4

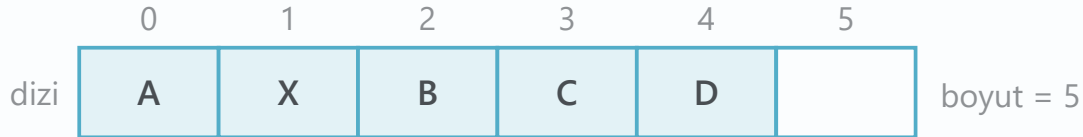
dizi = [A, X, B, C, D, null]

ÇAĞRI YIĞINI

add(1, "X")

main()

Kod İzleme: ArrayList add(i, e)



boyut = 5. $n - i = 3$ kaydırma → **O(n)**; başa eklemede tüm liste kayar.

Adım 9 / 9

```
1 public void add(int i, E e) {  
2     Objects.checkIndex(i, boyut + 1);  
3     if (boyut == dizi.length) buyut();  
4     for (int k = boyut; k > i; k--)  
5         dizi[k] = dizi[k - 1];  
6     dizi[i] = e;  
7     ▶ boyut++;  
8 }
```

DEĞİŞKENLER

i = 1

e = X

k = 1

boyut = 5

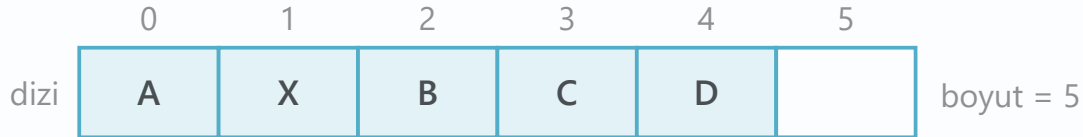
dizi = [A, X, B, C, D, null]

ÇAĞRI YIĞINI

add(1, "X")

main()

Kod İzleme: ArrayList `remove(i)`



Önceki slaytın sonucu [A, X, B, C, D]. `remove(1)` : X silinecek; B, C, D sola kaymalı.

Adım 1 / 9

```
► public E remove(int i) {  
2  Objects.checkIndex(i, boyut);  
3  E eski = dizi[i];  
4  for (int k = i; k < boyut - 1; k++)  
5    dizi[k] = dizi[k + 1];  
6  dizi[--boyut] = null;  
7  return eski;  
8 }
```

DEĞİŞKENLER

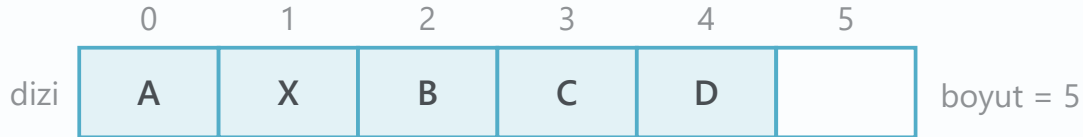
i = 1 eski = - k = - boyut = 5

dizi = [A, X, B, C, D, null]

ÇAĞRI YIĞINI

main()

Kod İzleme: ArrayList `remove(i)`



$0 \leq 1 < \text{boyut} (5) \rightarrow$ geçerli indis.

Adım 2 / 9

```
1 public E remove(int i) {  
2     ▶ Objects.checkIndex(i, boyut);  
3     E eski = dizi[i];  
4     for (int k = i; k < boyut - 1; k++)  
5         dizi[k] = dizi[k + 1];  
6     dizi[--boyut] = null;  
7     return eski;  
8 }
```

DEĞİŞKENLER

i = 1 eski = - k = - boyut = 5

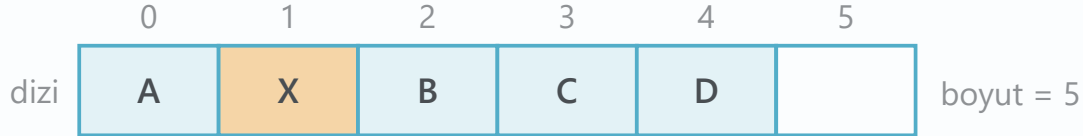
dizi = [A, X, B, C, D, null]

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: ArrayList `remove(i)`



eski = dizi[1] = X (döndürülecek).

Adım 3 / 9

```
1 public E remove(int i) {  
2     Objects.checkIndex(i, boyut);  
3     E eski = dizi[i];  
4     for (int k = i; k < boyut - 1; k++)  
5         dizi[k] = dizi[k + 1];  
6     dizi[--boyut] = null;  
7     return eski;  
8 }
```

DEĞİŞKENLER

i = 1 eski = X k = - boyut = 5

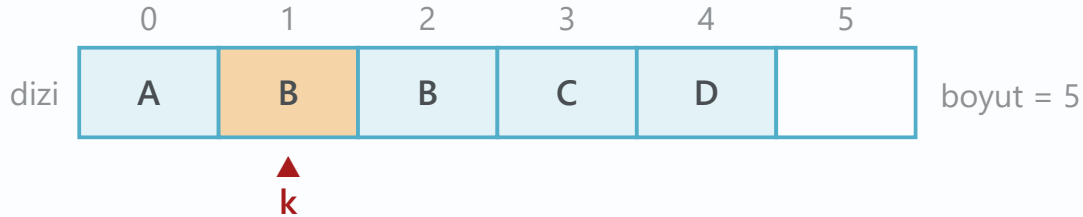
dizi = [A, X, B, C, D, null]

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: ArrayList remove(i)



$k = 1 < 4$: dizi[1] = dizi[2] → B sola kayar. Bu kez **soldan** başlanır.

Adım 4 / 9

```

1 public E remove(int i) {
2     Objects.checkIndex(i, boyut);
3     E eski = dizi[i];
4     for (int k = i; k < boyut - 1; k++)
5         dizi[k] = dizi[k + 1];
6     dizi[--boyut] = null;
7     return eski;
8 }

```

DEĞİŞKENLER

i = 1 eski = X k = 1 boyut = 5

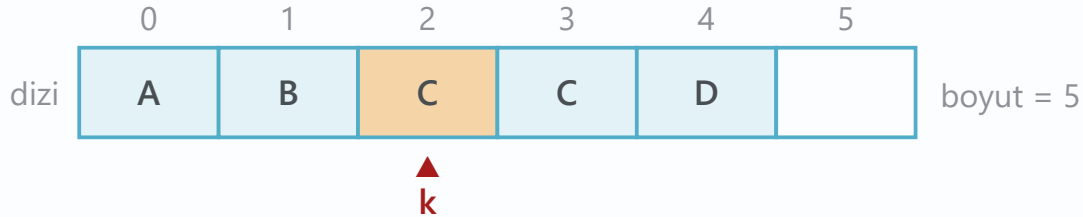
dizi = [A, B, B, C, D, null]

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: ArrayList remove(i)



$k = 2 < 4$: dizi[2] = dizi[3] → C sola kayar. Bu kez **soldan** başlanır.

Adım 5 / 9

```

1 public E remove(int i) {
2     Objects.checkIndex(i, boyut);
3     E eski = dizi[i];
4     ▶ for (int k = i; k < boyut - 1; k++)
5     ▶     dizi[k] = dizi[k + 1];
6     dizi[--boyut] = null;
7     return eski;
8 }

```

DEĞİŞKENLER

i = 1 eski = X k = 2 boyut = 5

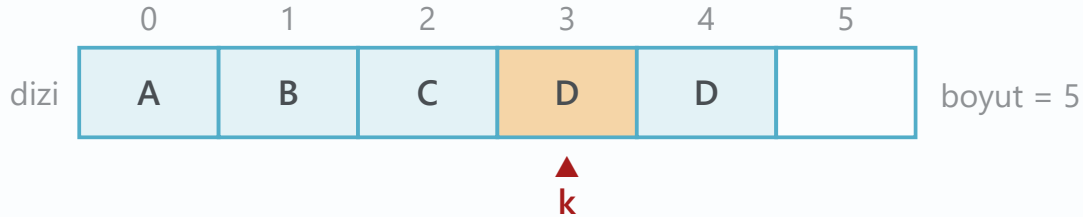
dizi = [A, B, C, C, D, null]

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: ArrayList remove(i)



$k = 3 < 4$: dizi[3] = dizi[4] → D sola kayar. Bu kez **soldan** başlanır.

Adım 6 / 9

```
1 public E remove(int i) {  
2     Objects.checkIndex(i, boyut);  
3     E eski = dizi[i];  
4     for (int k = i; k < boyut - 1; k++)  
5         dizi[k] = dizi[k + 1];  
6     dizi[--boyut] = null;  
7     return eski;  
8 }
```

DEĞİŞKENLER

i = 1 eski = X k = 3 boyut = 5

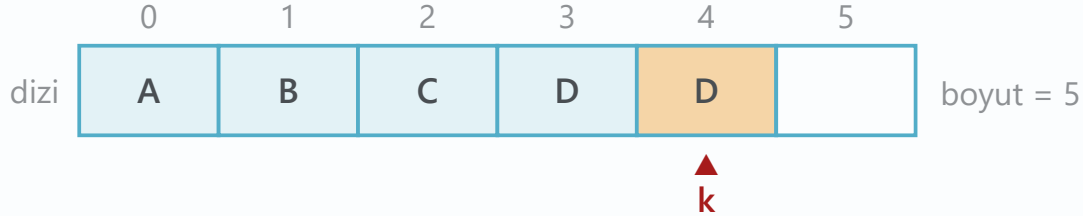
dizi = [A, B, C, D, D, null]

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: ArrayList remove(i)



$k = 4 < 4$ değil → döngü biter. Son yuvada D iki kez var.

Adım 7 / 9

```
1 public E remove(int i) {  
2     Objects.checkIndex(i, boyut);  
3     E eski = dizi[i];  
4     for (int k = i; k < boyut - 1; k++)  
5         dizi[k] = dizi[k + 1];  
6     dizi[--boyut] = null;  
7     return eski;  
8 }
```

DEĞİŞKENLER

`i = 1` `eski = X` **`k = 4`** `boyut = 5`

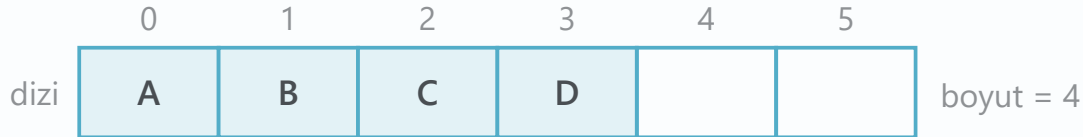
`dizi = [A, B, C, D, D, null]`

ÇAĞRI YIĞINI

`remove(1)`

`main()`

Kod İzleme: ArrayList remove(i)



`dizi[--boyut] = null` : boyut 4, fazla referans temizlenir (bellek sızıntısını önler). Kapasite küçülmez.

Adım 8 / 9

```
1 public E remove(int i) {  
2     Objects.checkIndex(i, boyut);  
3     E eski = dizi[i];  
4     for (int k = i; k < boyut - 1; k++)  
5         dizi[k] = dizi[k + 1];  
6     dizi[--boyut] = null;  
7     return eski;  
8 }
```

DEĞİŞKENLER

i = 1 eski = X k = - boyut = 4

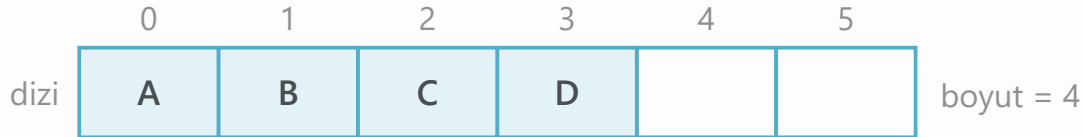
dizi = [A, B, C, D, null, null]

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: ArrayList remove(i)



return X. $n - i - 1 = 3$ kaydırma → **O(n)**. Sondan silmek ($i = \text{boyut} - 1$) O(1).

Adım 9 / 9

```

1 public E remove(int i) {
2     Objects.checkIndex(i, boyut);
3     E eski = dizi[i];
4     for (int k = i; k < boyut - 1; k++)
5         dizi[k] = dizi[k + 1];
6     dizi[--boyut] = null;
7     return eski;
8 }
```

DEĞİŞKENLER

i = 1 eski = X k = - boyut = 4

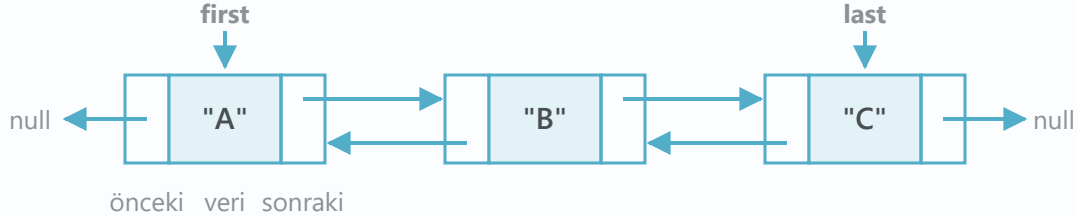
dizi = [A, B, C, D, null, null]

ÇAĞRI YIĞINI

remove(1)

main()

LinkedList: Çift Yönlü Bağlı Liste



- Öğeler bellekte ardışık değildir: her öge ayrı bir **düğüm** (node) nesnesidir.
- Her düğüm veriyi ve **önceki** / **sonraki** düğümlerin referansını tutar; liste **first** ve **last** 'ı bilir.
- **List** 'in yanında **Deque** arayüzünü de uygular: yığın ve kuyruk olarak kullanılabilir (Bölüm 4–5).
- Rastgele erişim yavaştır: **get(i)** düğümleri tek tek yürür (yakın uçtan başlar) $\rightarrow O(n)$.

LinkedList Kullanımı ve Maliyetleri

```
LinkedList<String> liste =  
    new LinkedList<>();  
liste.add("B");           // [B]  
liste.addFirst("A");      // [A, B]  
liste.addLast("C");       // [A, B, C]  
liste.get(1);             // "B" (yürüyerek)  
liste.removeFirst();      // [B, C]  
liste.size();             // 2
```

İşlem	Maliyet
baş/sona ekleme, baştan/sondan silme	O(1)
get(i) , set(i, e)	O(n)
add(i, e) , remove(i)	O(n)*
iterator ile ekleme/silme	O(1)

* Bağlantıyı değiştirmek O(1)'dir, ama önce **i. düğüme yürümek** O(n) sürer. "LinkedList'te ekleme hızlıdır" ancak konum zaten elimizdeyse (uçlar ya da ListIterator) doğrudur.

Kod İzleme: LinkedList İşlemlerinin Maliyeti

first = null · last = null

Boş LinkedList: hiç düğüm yok, **first** ve **last** null.

Adım 1 / 9

```
► LinkedList<String> liste = new LinkedList<>();  
2 liste.add("B");  
3 liste.addFirst("A");  
4 liste.addLast("C");  
5 String s = liste.get(1);  
6 liste.removeFirst();  
7 int n = liste.size();
```

DEĞİŞKENLER

liste = []

size = 0

first = null

last = null

Kod İzleme: LinkedList İşlemlerinin Maliyeti



`add` = `addLast` : tek düğüm; first ve last ikisi de onu gösterir.

Adım 2 / 9

```
1 LinkedList<String> liste = new LinkedList<>();  
2 ► liste.add("B");  
3 liste.addFirst("A");  
4 liste.addLast("C");  
5 String s = liste.get(1);  
6 liste.removeFirst();  
7 int n = liste.size();
```

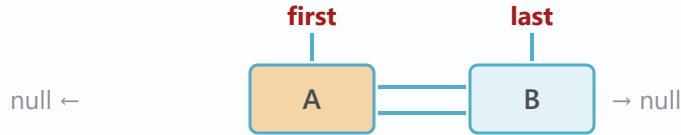
DEĞİŞKENLER

liste = [B] size = 1

first = düğüm(B)

last = düğüm(B)

Kod İzleme: LinkedList İşlemlerinin Maliyeti



addFirst : yeni düğüm, A.sonraki = B, B.önceki = A, first = A → **O(1)**, kaydırma yok.

Adım 3 / 9

```
1 LinkedList<String> liste = new LinkedList<>();
2 liste.add("B");
3 ► liste.addFirst("A");
4 liste.addLast("C");
5 String s = liste.get(1);
6 liste.removeFirst();
7 int n = liste.size();
```

DEĞİŞKENLER

liste = [A, B] size = 2

first = düğüm(A)

last = düğüm(B)

Kod İzleme: LinkedList İşlemlerinin Maliyeti



addLast : last sayesinde sona yürümeden eklenir → $O(1)$.

Adım 4 / 9

```
1 LinkedList<String> liste = new LinkedList<>();
2 liste.add("B");
3 liste.addFirst("A");
4 ► liste.addLast("C");
5 String s = liste.get(1);
6 liste.removeFirst();
7 int n = liste.size();
```

DEĞİŞKENLER

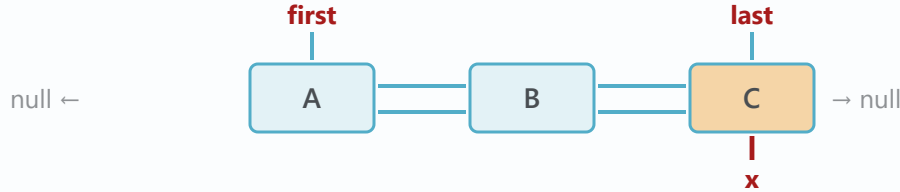
liste = [A, B, C]

size = 3

first = düğüm(A)

last = düğüm(C)

Kod İzleme: LinkedList İşlemlerinin Maliyeti



`get(1)` → `node(1)` : $1 < \text{size}/2 = 1$ değil → **sondan** başla: $x = \text{last (C)}$.

Adım 5 / 9

```
1 LinkedList<String> liste = new LinkedList<>();
2 liste.add("B");
3 liste.addFirst("A");
4 liste.addLast("C");
5 ► String s = liste.get(1);
6 liste.removeFirst();
7 int n = liste.size();
```

DEĞİŞKENLER

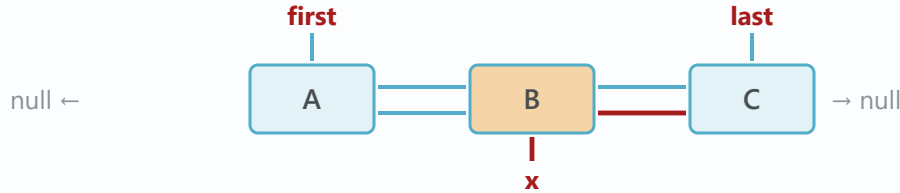
`liste = [A, B, C]`

`size = 3`

`first = düğüm(A)`

`last = düğüm(C)`

Kod İzleme: LinkedList İşlemlerinin Maliyeti



$x = x.onceki \rightarrow B$. Aranana düğüm; 1 bağlantı yüründü. Genelde $O(n)$.

Adım 6 / 9

```
1 LinkedList<String> liste = new LinkedList<>();
2 liste.add("B");
3 liste.addFirst("A");
4 liste.addLast("C");
5 ▶ String s = liste.get(1);
6 liste.removeFirst();
7 int n = liste.size();
```

DEĞİŞKENLER

liste = [A, B, C]

size = 3

first = düğüm(A)

last = düğüm(C)

s = "B"

Kod İzleme: LinkedList İşlemlerinin Maliyeti



removeFirst : A ayrılacak.

Adım 7 / 9

```
1 LinkedList<String> liste = new LinkedList<>();
2 liste.add("B");
3 liste.addFirst("A");
4 liste.addLast("C");
5 String s = liste.get(1);
6 ► liste.removeFirst();
7 int n = liste.size();
```

DEĞİŞKENLER

liste = [A, B, C]

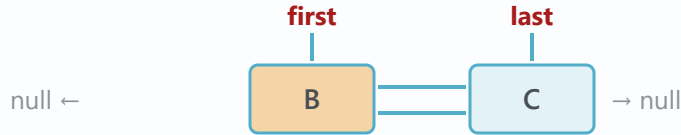
size = 3

first = düğüm(A)

last = düğüm(C)

s = "B"

Kod İzleme: LinkedList İşlemlerinin Maliyeti



first = B, B.onceki = null; A'ya referans kalmaz, çöp toplayıcı temizler → O(1).

Adım 8 / 9

```
1 LinkedList<String> liste = new LinkedList<>();
2 liste.add("B");
3 liste.addFirst("A");
4 liste.addLast("C");
5 String s = liste.get(1);
▶ liste.removeFirst();
7 int n = liste.size();
```

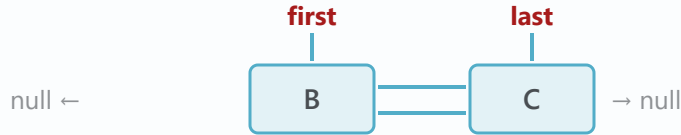
DEĞİŞKENLER

liste = [B, C] size = 2

first = düğüm(B)

last = düğüm(C) s = "B"

Kod İzleme: LinkedList İşlemlerinin Maliyeti



`size()` sayaç alanından: $O(1)$.

Adım 9 / 9

```
1 LinkedList<String> liste = new LinkedList<>();
2 liste.add("B");
3 liste.addFirst("A");
4 liste.addLast("C");
5 String s = liste.get(1);
6 liste.removeFirst();
▶ int n = liste.size();
```

DEĞİŞKENLER

liste = [B, C] size = 2

first = düğüm(B)

last = düğüm(C) s = "B"

n = 2

Kod İzleme: LinkedList addLast(e)

first = null · last = null · size = 0

Boş LinkedList. `addLast("A")`, sonra `addLast("B")` (Java'daki `linkLast` ile aynı mantık).

Adım 1 / 12

```
► void addLast(E e) {  
2   Dugum<E> l = last;  
3   Dugum<E> d = new Dugum<>(l, e, null);  
4   last = d;  
5   if (l == null) first = d;  
6   else l.sonraki = d;  
7   size++;  
8 }
```

DEĞİŞKENLER

e = - l = - d = - first = null

last = null size = 0

ÇAĞRI YIĞINI

main()

Kod İzleme: LinkedList addLast(e)

first = null · last = null · size = 0

l = last = null: eski son düğüm saklanır.

Adım 2 / 12

```
1 void addLast(E e) {  
2     ▶ Dugum<E> l = last;  
3     Dugum<E> d = new Dugum<>(l, e, null);  
4     last = d;  
5     if (l == null) first = d;  
6     else l.sonraki = d;  
7     size++;  
8 }
```

DEĞİŞKENLER

e = A

l = null

d = -

first = null

last = null

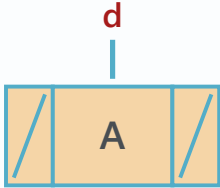
size = 0

ÇAĞRI YIĞINI

addLast("A")

main()

Kod İzleme: LinkedList addLast(e)



`first = null · last = null · size = 0`

Yeni düğüm: önceki = `l` (`null`), sonraki = `null`.

Adım 3 / 12

```
1 void addLast(E e) {  
2   Dugum<E> l = last;  
3   ► Dugum<E> d = new Dugum<>(l, e, null);  
4   last = d;  
5   if (l == null) first = d;  
6   else l.sonraki = d;  
7   size++;  
8 }
```

DEĞİŞKENLER

`e = A`

`l = null`

`d = düğüm(A)`

`first = null`

`last = null`

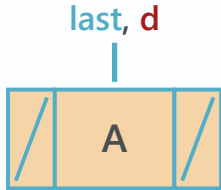
`size = 0`

ÇAĞRI YIĞINI

`addLast("A")`

`main()`

Kod İzleme: LinkedList addLast(e)



first = null · size = 0

last = d .

Adım 4 / 12

```
1 void addLast(E e) {  
2   Dugum<E> l = last;  
3   Dugum<E> d = new Dugum<>(l, e, null);  
4   last = d;  
5   if (l == null) first = d;  
6   else l.sonraki = d;  
7   size++;  
8 }
```

DEĞİŞKENLER

e = A l = null d = düğüm(A)

first = null last = düğüm(A)

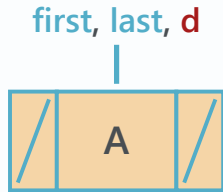
size = 0

ÇAĞRI YIĞINI

addLast("A")

main()

Kod İzleme: LinkedList addLast(e)



size = 0

l == null (liste boştu) → `first = d` : tek düğüm hem ilk hem son.

Adım 5 / 12

```

1 void addLast(E e) {
2     Dugum<E> l = last;
3     Dugum<E> d = new Dugum<>(l, e, null);
4     last = d;
5     ► if (l == null) first = d;
6     else l.sonraki = d;
7     size++;
8 }

```

DEĞİŞKENLER

e = A

l = null

d = düğüm(A)

first = düğüm(A)

last = düğüm(A)

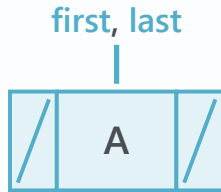
size = 0

ÇAĞRI YIĞINI

addLast("A")

main()

Kod İzleme: LinkedList addLast(e)



size = 1

size = 1.

Adım 6 / 12

```
1 void addLast(E e) {
2     Dugum<E> l = last;
3     Dugum<E> d = new Dugum<>(l, e, null);
4     last = d;
5     if (l == null) first = d;
6     else l.sonraki = d;
7     size++;
8 }
```

DEĞİŞKENLER

e = A

l = null

d = düğüm(A)

first = düğüm(A)

last = düğüm(A)

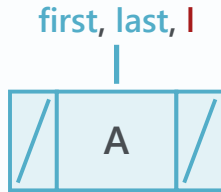
size = 1

ÇAĞRI YIĞINI

addLast("A")

main()

Kod İzleme: LinkedList addLast(e)



size = 1

l = last = düğüm(A): eski son düğüm saklanır.

Adım 7 / 12

```

1 void addLast(E e) {
2     ▶ Dugum<E> l = last;
3     Dugum<E> d = new Dugum<>(l, e, null);
4     last = d;
5     if (l == null) first = d;
6     else l.sonraki = d;
7     size++;
8 }

```

DEĞİŞKENLER

e = B

l = düğüm(A)

d = -

first = düğüm(A)

last = düğüm(A)

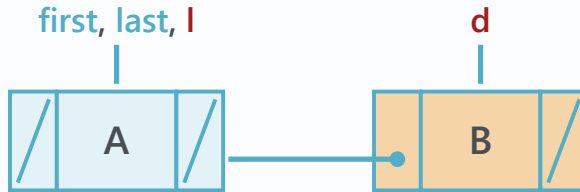
size = 1

ÇAĞRI YIĞINI

addLast("B")

main()

Kod İzleme: LinkedList addLast(e)



size = 1

Yeni düğüm: önceki = l (düğüm(A)), sonraki = null. Geri bağlantı kurulurken hazır.

Adım 8 / 12

```

1 void addLast(E e) {
2   Dugum<E> l = last;
3   ► Dugum<E> d = new Dugum<>(l, e, null);
4   last = d;
5   if (l == null) first = d;
6   else l.sonraki = d;
7   size++;
8 }

```

DEĞİŞKENLER

e = B

l = düğüm(A)

d = düğüm(B)

first = düğüm(A)

last = düğüm(A)

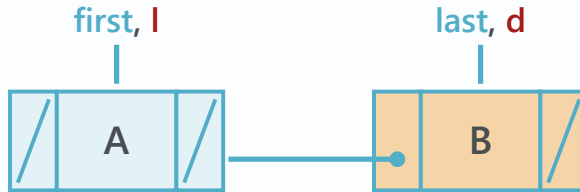
size = 1

ÇAĞRI YIĞINI

addLast("B")

main()

Kod İzleme: LinkedList addLast(e)



size = 1

`last = d .`

Adım 9 / 12

```
1 void addLast(E e) {  
2   Dugum<E> l = last;  
3   Dugum<E> d = new Dugum<>(l, e, null);  
4   last = d;  
5   if (l == null) first = d;  
6   else l.sonraki = d;  
7   size++;  
8 }
```

DEĞİŞKENLER

e = B

l = düğüm(A)

d = düğüm(B)

first = düğüm(A)

last = düğüm(B)

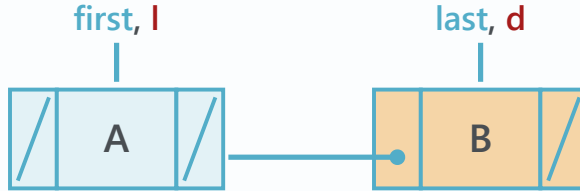
size = 1

ÇAĞRI YIĞINI

addLast("B")

main()

Kod İzleme: LinkedList addLast(e)



size = 1

`l == null` değil → else.

Adım 10 / 12

```
1 void addLast(E e) {  
2     Dugum<E> l = last;  
3     Dugum<E> d = new Dugum<>(l, e, null);  
4     last = d;  
5     ► if (l == null) first = d;  
6     else l.sonraki = d;  
7     size++;  
8 }
```

DEĞİŞKENLER

e = B

l = düğüm(A)

d = düğüm(B)

first = düğüm(A)

last = düğüm(B)

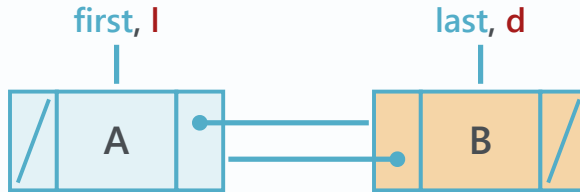
size = 1

ÇAĞRI YIĞINI

addLast("B")

main()

Kod İzleme: LinkedList addLast(e)



size = 1

`l.sonraki = d` : A → B ileri bağlantısı.

Adım 11 / 12

```
1 void addLast(E e) {  
2   Dugum<E> l = last;  
3   Dugum<E> d = new Dugum<>(l, e, null);  
4   last = d;  
5   if (l == null) first = d;  
6   else l.sonraki = d;  
7   size++;  
8 }
```

DEĞİŞKENLER

e = B

l = düğüm(A)

d = düğüm(B)

first = düğüm(A)

last = düğüm(B)

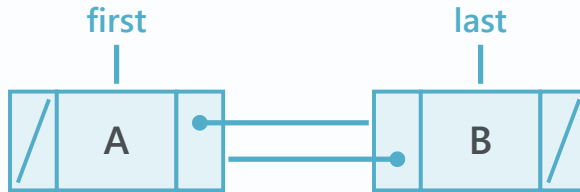
size = 1

ÇAĞRI YIĞINI

addLast("B")

main()

Kod İzleme: LinkedList addLast(e)



size = 2

size = 2. Gezinme yok: sona ekleme **O(1)**.

Adım 12 / 12

```
1 void addLast(E e) {  
2     Dugum<E> l = last;  
3     Dugum<E> d = new Dugum<>(l, e, null);  
4     last = d;  
5     if (l == null) first = d;  
6     else l.sonraki = d;  
7     size++;  
8 }
```

DEĞİŞKENLER

e = B

l = düğüm(A)

d = düğüm(B)

first = düğüm(A)

last = düğüm(B)

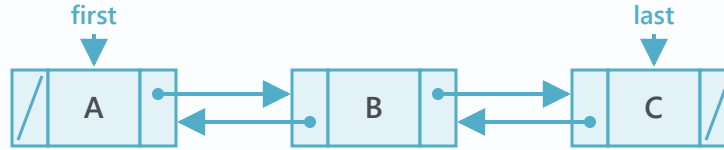
size = 2

ÇAĞRI YIĞINI

addLast("B")

main()

Kod İzleme: LinkedList `remove(i)`



size = 3

[A, B, C]. `remove(1)` : B'yi bulmak için önce **yürümek** gerekir.

Adım 1 / 10

```

▶ E remove(int i) {
2  Objects.checkIndex(i, size);
3  Dugum<E> x = first;
4  for (int k = 0; k < i; k++) x = x.sonraki;
5  Dugum<E> o = x.onceki, s = x.sonraki;
6  if (o == null) first = s; else o.sonraki = s;
7  if (s == null) last = o; else s.onceki = o;
8  size--;
9  return x.veri;
10 }
  
```

DEĞİŞKENLER

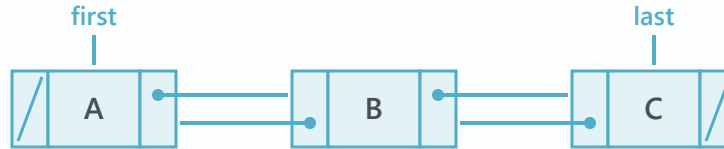
i = 1 k = - x = - o = -

s = - size = 3

ÇAĞRI YIĞINI

main()

Kod İzleme: LinkedList `remove(i)`



size = 3

 $0 \leq 1 < 3 \rightarrow$ geçerli indis.

Adım 2 / 10

```
1 E remove(int i) {
2   ► Objects.checkIndex(i, size);
3   Dugum<E> x = first;
4   for (int k = 0; k < i; k++) x = x.sonraki;
5   Dugum<E> o = x.onceki, s = x.sonraki;
6   if (o == null) first = s; else o.sonraki = s;
7   if (s == null) last = o; else s.onceki = o;
8   size--;
9   return x.veri;
10 }
```

DEĞİŞKENLER

i = 1	k = -	x = -	o = -
-------	-------	-------	-------

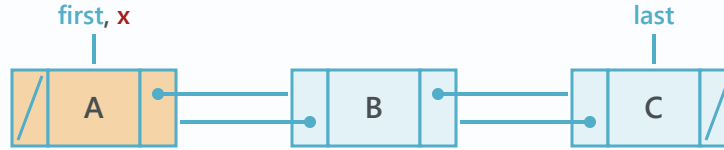
s = -	size = 3
-------	----------

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: LinkedList `remove(i)`



size = 3

x = first (A). Dizideki gibi doğrudan indisle erişim yok.

Adım 3 / 10

```

1 E remove(int i) {
2   Objects.checkIndex(i, size);
3   ▶ Dugum<E> x = first;
4   for (int k = 0; k < i; k++) x = x.sonraki;
5   Dugum<E> o = x.onceki, s = x.sonraki;
6   if (o == null) first = s; else o.sonraki = s;
7   if (s == null) last = o; else s.onceki = o;
8   size--;
9   return x.veri;
10 }

```

DEĞİŞKENLER

i = 1

k = -

x = düğüm(A)

o = -

s = -

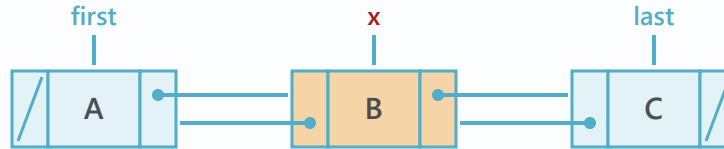
size = 3

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: LinkedList remove(i)



size = 3

$k = 0 < 1$: $x = x.sonraki \rightarrow B$.

Adım 4 / 10

```

1 E remove(int i) {
2   Objects.checkIndex(i, size);
3   Dugum<E> x = first;
4   ▶ for (int k = 0; k < i; k++) x = x.sonraki;
5   Dugum<E> o = x.onceki, s = x.sonraki;
6   if (o == null) first = s; else o.sonraki = s;
7   if (s == null) last = o; else s.onceki = o;
8   size--;
9   return x.veri;
10 }

```

DEĞİŞKENLER

i = 1

k = 0

x = düğüm(B)

o = -

s = -

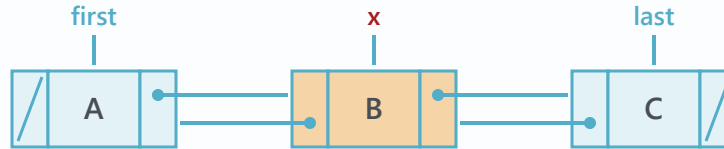
size = 3

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: LinkedList remove(i)



size = 3

$k = 1 < 1$ değil $\rightarrow$ yürüme biter. i adım yürüdük: **$O(i)$** . (Java, $i \geq \text{size}/2$ ise last'tan geriye yürür.)

Adım 5 / 10

```

1 E remove(int i) {
2   Objects.checkIndex(i, size);
3   Dugum<E> x = first;
4   ► for (int k = 0; k < i; k++) x = x.sonraki;
5   Dugum<E> o = x.onceki, s = x.sonraki;
6   if (o == null) first = s; else o.sonraki = s;
7   if (s == null) last = o; else s.onceki = o;
8   size--;
9   return x.veri;
10 }

```

DEĞİŞKENLER

$i = 1$ $k = 1$ $x = \text{düğüm}(B)$

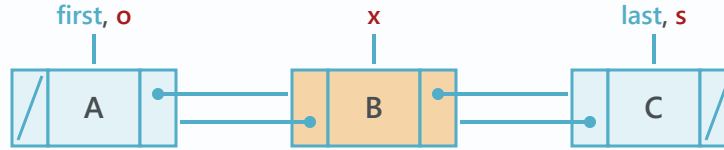
$o = -$ $s = -$ $\text{size} = 3$

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: LinkedList remove(i)



size = 3

o = x.onceki = A, **s = x.sonraki** = C: silinecek düğümün iki komşusu.

Adım 6 / 10

```

1 E remove(int i) {
2   Objects.checkIndex(i, size);
3   Dugum<E> x = first;
4   for (int k = 0; k < i; k++) x = x.sonraki;
5   ► Dugum<E> o = x.onceki, s = x.sonraki;
6   if (o == null) first = s; else o.sonraki = s;
7   if (s == null) last = o; else s.onceki = o;
8   size--;
9   return x.veri;
10 }

```

DEĞİŞKENLER

i = 1 k = 1 x = düğüm(B)

o = düğüm(A) s = düğüm(C)

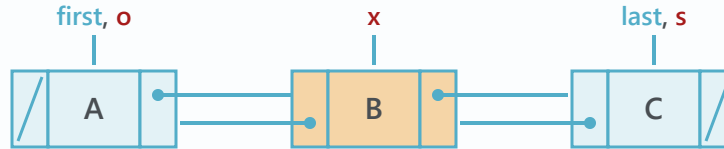
size = 3

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: LinkedList `remove(i)`



size = 3

o null değil → `o.sonraki = s` : A artık doğrudan C'yi gösterir (B atlanır).

Adım 7 / 10

```

1 E remove(int i) {
2   Objects.checkIndex(i, size);
3   Dugum<E> x = first;
4   for (int k = 0; k < i; k++) x = x.sonraki;
5   Dugum<E> o = x.onceki, s = x.sonraki;
6   ► if (o == null) first = s; else o.sonraki = s;
7   if (s == null) last = o; else s.onceki = o;
8   size--;
9   return x.veri;
10 }

```

DEĞİŞKENLER

`i = 1` `k = 1` `x = düğüm(B)`

`o = düğüm(A)` `s = düğüm(C)`

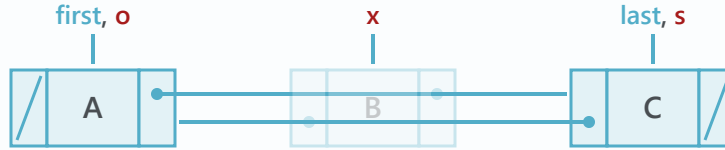
`size = 3`

ÇAĞRI YIĞINI

`remove(1)`

`main()`

Kod İzleme: LinkedList remove(i)



size = 3

s null değil → `s.onceki = o` : C geriye A'yı gösterir. B iki yönden de koptu.

Adım 8 / 10

```

1 E remove(int i) {
2   Objects.checkIndex(i, size);
3   Dugum<E> x = first;
4   for (int k = 0; k < i; k++) x = x.sonraki;
5   Dugum<E> o = x.onceki, s = x.sonraki;
6   if (o == null) first = s; else o.sonraki = s;
7   if (s == null) last = o; else s.onceki = o;
8   size--;
9   return x.veri;
10 }

```

DEĞİŞKENLER

`i = 1` `k = 1` `x = düğüm(B)`

`o = düğüm(A)` `s = düğüm(C)`

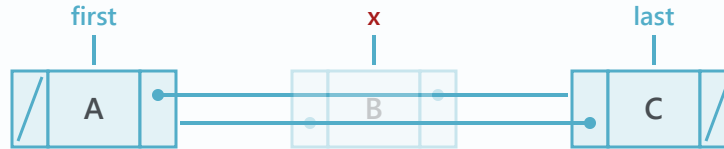
`size = 3`

ÇAĞRI YIĞINI

`remove(1)`

`main()`

Kod İzleme: LinkedList remove(i)



size = 2

size = 2.

Adım 9 / 10

```

1 E remove(int i) {
2   Objects.checkIndex(i, size);
3   Dugum<E> x = first;
4   for (int k = 0; k < i; k++) x = x.sonraki;
5   Dugum<E> o = x.onceki, s = x.sonraki;
6   if (o == null) first = s; else o.sonraki = s;
7   if (s == null) last = o; else s.onceki = o;
8   ▶ size--;
9   return x.veri;
10 }

```

DEĞİŞKENLER

i = 1 k = 1 x = düğüm(B)

o = düğüm(A) s = düğüm(C)

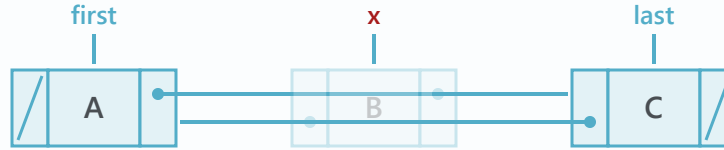
size = 2

ÇAĞRI YIĞINI

remove(1)

main()

Kod İzleme: LinkedList remove(i)



size = 2

return "B". Bağ değiştirmek $O(1)$, ama yürümek $O(n) \rightarrow$ `remove(i)` toplamda **$O(n)$** .

Adım 10 / 10

```

1 E remove(int i) {
2   Objects.checkIndex(i, size);
3   Dugum<E> x = first;
4   for (int k = 0; k < i; k++) x = x.sonraki;
5   Dugum<E> o = x.onceki, s = x.sonraki;
6   if (o == null) first = s; else o.sonraki = s;
7   if (s == null) last = o; else s.onceki = o;
8   size--;
9   return x.veri;
10 }
```

DEĞİŞKENLER

`i = 1` `k = 1` `x = düğüm(B)`

`o = düğüm(A)` `s = düğüm(C)`

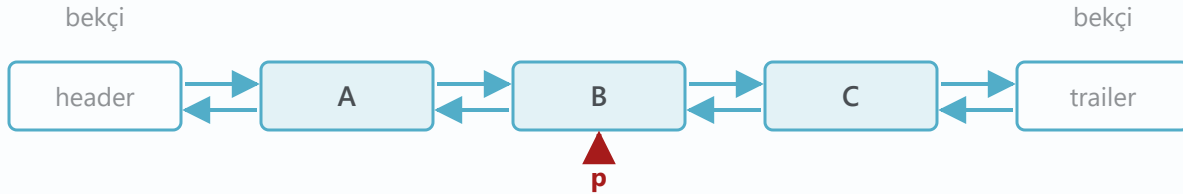
`size = 2`

ÇAĞRI YIĞINI

`remove(1)`

`main()`

Konumsal Liste (Positional List) ADT



after(p) = C'nin konumu · before(p) = A'nın konumu

- İndisler ekleme/silmede **kayar**; **konum** (position) ise öğeye bağlı kalır: metin editöründeki imleç gibi.
- Goodrich–Tamassia: `Position<E>` yalnızca `getElement()` sunar; bağlı gerçekleştirimde konum = düğüm.
- Bekçi** (sentinel) header/trailer sayesinde her ekleme iki gerçek düğüm arasına yapılır: null denetimi yok.

Metot	Maliyet
<code>first()</code> , <code>last()</code> , <code>before(p)</code> , <code>after(p)</code>	O(1)
<code>addFirst(e)</code> , <code>addLast(e)</code>	O(1)
<code>addBefore(p, e)</code> , <code>addAfter(p, e)</code>	O(1)
<code>set(p, e)</code> , <code>remove(p)</code>	O(1)

Java'da doğrudan karşılığı yok; en yakını `ListIterator` 'dır (imlecin yanında O(1) ekleme/silme).

Kod İzleme: Konumsal Liste `addAfter(p, e)`



size = 2

header $\rightleftharpoons$ A $\rightleftharpoons$ C $\rightleftharpoons$ trailer; p, A'nın konumu. `addAfter(p, "B")` çağrılıyor.

Adım 1 / 8

```

1 ▶ Position<E> addAfter(Position<E> p, E e) {
2   Dugum<E> d = validate(p);
3   return addBetween(e, d, d.sonraki);
4 }
5 Position<E> addBetween(E e, Dugum<E> o, Dugum<E> s) {
6   Dugum<E> yeni = new Dugum<>(o, e, s);
7   o.sonraki = yeni;
8   s.onceki = yeni;
9   size++;
10  return yeni;
11 }
  
```

DEĞİŞKENLER

p = düğüm(A)

e = "B"

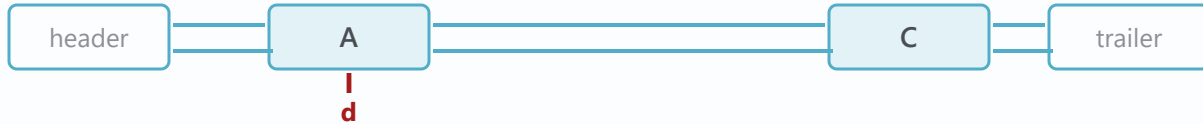
size = 2

ÇAĞRI YIĞINI

addAfter(p, "B")

main(): liste.addAfter(p, "B")

Kod İzleme: Konumsal Liste `addAfter(p, e)`



size = 2

`validate(p)` : p bu listenin geçerli bir düğümü mü? Evet → Dugum'e dönüştürülür.

Adım 2 / 8

```

1 Position<E> addAfter(Position<E> p, E e) {
2     ▶ Dugum<E> d = validate(p);
3     return addBetween(e, d, d.sonraki);
4 }
5 Position<E> addBetween(E e, Dugum<E> o, Dugum<E> s) {
6     Dugum<E> yeni = new Dugum<>(o, e, s);
7     o.sonraki = yeni;
8     s.onceki = yeni;
9     size++;
10    return yeni;
11 }

```

DEĞİŞKENLER

p = düğüm(A)

e = "B"

size = 2

d = düğüm(A)

ÇAĞRI YIĞINI

addAfter(p, "B")

main(): liste.addAfter(p, "B")

Kod İzleme: Konumsal Liste `addAfter(p, e)`



Komşular belli: $o = A$, $s = A.sonraki = C$. İş `addBetween` 'e devredilir.

Adım 3 / 8

```

1 Position<E> addAfter(Position<E> p, E e) {
2   Dugum<E> d = validate(p);
3   ► return addBetween(e, d, d.sonraki);
4 }
5 Position<E> addBetween(E e, Dugum<E> o, Dugum<E> s) {
6   Dugum<E> yeni = new Dugum<>(o, e, s);
7   o.sonraki = yeni;
8   s.onceki = yeni;
9   size++;
10  return yeni;
11 }

```

DEĞİŞKENLER

$p = \text{dügüm}(A)$

$e = "B"$

$size = 2$

$d = \text{dügüm}(A)$

$o = \text{dügüm}(A)$

$s = \text{dügüm}(C)$

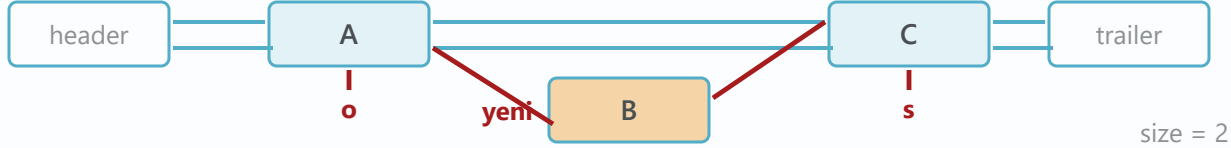
ÇAĞRI YIĞINI

`addBetween("B", A, C)`

`addAfter(p, "B")`

`main(): liste.addAfter(p, "B")`

Kod İzleme: Konumsal Liste `addAfter(p, e)`



Yeni düğüm iki komşusunu gösterir; ama henüz kimse onu göstermiyor.

Adım 4 / 8

```

1 Position<E> addAfter(Position<E> p, E e) {
2   Dugum<E> d = validate(p);
3   return addBetween(e, d, d.sonraki);
4 }
5 Position<E> addBetween(E e, Dugum<E> o, Dugum<E> s) {
6   ▶ Dugum<E> yeni = new Dugum<>(o, e, s);
7   o.sonraki = yeni;
8   s.onceki = yeni;
9   size++;
10  return yeni;
11 }

```

DEĞİŞKENLER

p = düğüm(A)

e = "B"

size = 2

o = düğüm(A)

s = düğüm(C)

yeni = düğüm(B)

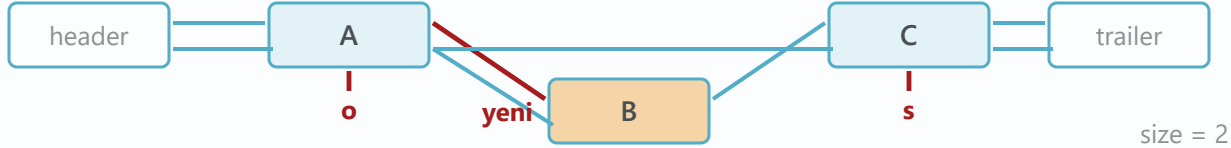
ÇAĞRI YIĞINI

addBetween("B", A, C)

addAfter(p, "B")

main(): liste.addAfter(p, "B")

Kod İzleme: Konumsal Liste `addAfter(p, e)`



`o.sonraki = yeni`: ileri yönde $A \rightarrow B \rightarrow C$.

Adım 5 / 8

```

1 Position<E> addAfter(Position<E> p, E e) {
2   Dugum<E> d = validate(p);
3   return addBetween(e, d, d.sonraki);
4 }
5 Position<E> addBetween(E e, Dugum<E> o, Dugum<E> s) {
6   Dugum<E> yeni = new Dugum<>(o, e, s);
7   o.sonraki = yeni;
8   s.onceki = yeni;
9   size++;
10  return yeni;
11 }

```

DEĞİŞKENLER

`p = düğüm(A)`

`e = "B"`

`size = 2`

`o = düğüm(A)`

`s = düğüm(C)`

`yeni = düğüm(B)`

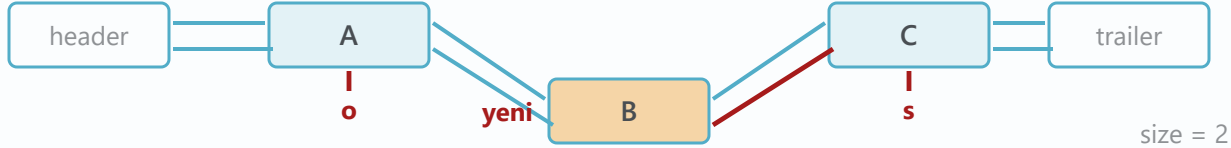
ÇAĞRI YIĞINI

`addBetween("B", A, C)`

`addAfter(p, "B")`

`main(): liste.addAfter(p, "B")`

Kod İzleme: Konumsal Liste `addAfter(p, e)`



s.onceki = yeni: geri yönde $C \rightarrow B \rightarrow A$. Eski $A \rightleftharpoons C$ bağlantıları kalmadı.

Adım 6 / 8

```

1 Position<E> addAfter(Position<E> p, E e) {
2   Dugum<E> d = validate(p);
3   return addBetween(e, d, d.sonraki);
4 }
5 Position<E> addBetween(E e, Dugum<E> o, Dugum<E> s) {
6   Dugum<E> yeni = new Dugum<>(o, e, s);
7   o.sonraki = yeni;
8   s.onceki = yeni;
9   size++;
10  return yeni;
11 }

```

DEĞİŞKENLER

p = düğüm(A)

e = "B"

size = 2

o = düğüm(A)

s = düğüm(C)

yeni = düğüm(B)

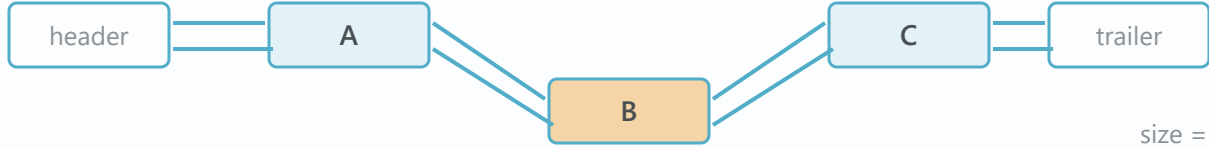
ÇAĞRI YIĞINI

addBetween("B", A, C)

addAfter(p, "B")

main(): liste.addAfter(p, "B")

Kod İzleme: Konumsal Liste `addAfter(p, e)`



size++. Yürüme, kaydırma yok: 4 atama → **O(1)**.

Adım 7 / 8

```

1 Position<E> addAfter(Position<E> p, E e) {
2   Dugum<E> d = validate(p);
3   return addBetween(e, d, d.sonraki);
4 }
5 Position<E> addBetween(E e, Dugum<E> o, Dugum<E> s) {
6   Dugum<E> yeni = new Dugum<>(o, e, s);
7   o.sonraki = yeni;
8   s.onceki = yeni;
9   ▶ size++;
10  return yeni;
11 }
  
```

DEĞİŞKENLER

p = düğüm(A)

e = "B"

size = 3

o = düğüm(A)

s = düğüm(C)

yeni = düğüm(B)

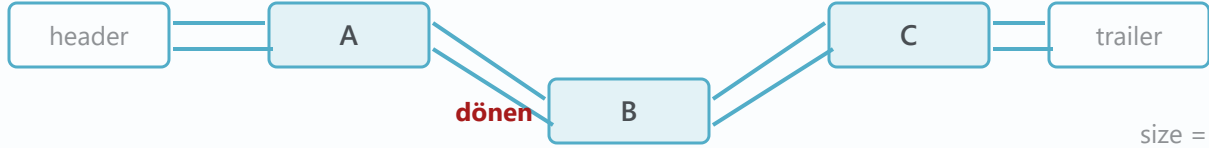
ÇAĞRI YIĞINI

addBetween("B", A, C)

addAfter(p, "B")

main(): liste.addAfter(p, "B")

Kod İzleme: Konumsal Liste `addAfter(p, e)`



Yeni öğenin **konumu** döner; ileride `addBefore` , `remove` için kullanılabilir.

Adım 8 / 8

```

1 Position<E> addAfter(Position<E> p, E e) {
2   Dugum<E> d = validate(p);
3   return addBetween(e, d, d.sonraki);
4 }
5 Position<E> addBetween(E e, Dugum<E> o, Dugum<E> s) {
6   Dugum<E> yeni = new Dugum<>(o, e, s);
7   o.sonraki = yeni;
8   s.onceki = yeni;
9   size++;
10  return yeni;
11 }
  
```

DEĞİŞKENLER

p = düğüm(A)

e = "B"

size = 3

o = -

s = -

yeni = düğüm(B)

ÇAĞRI YIĞINI

main(): liste.addAfter(p, "B")

ArrayList mi, LinkedList mi?

	ArrayList	LinkedList
İç yapı	büyüyebilen dizi	çift yönlü bağlı düğümler
get(i)	O(1)	O(n)
Sona ekleme	amortize O(1)	O(1)
Başa ekleme / baştan silme	O(n)	O(1)
Öğeye başına bellek	1 referans (+ boş kapasite)	düğüm nesnesi (3 referans)
Önbellek (cache)	ardışık, hızlı	dağınık, yavaş

Varsayılan: ArrayList

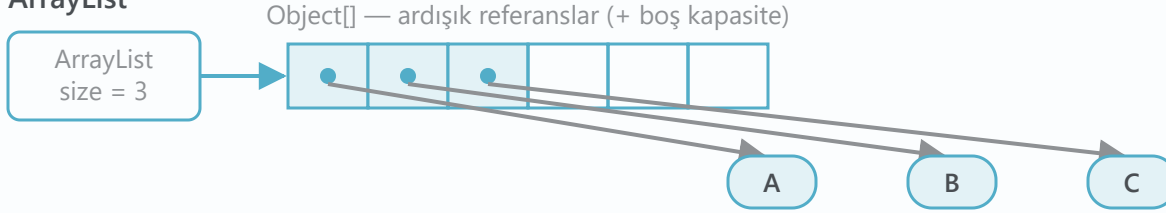
Çoğu senaryoda (okuma ağırlıklı, sona ekleme) daha hızlıdır.

LinkedList / ArrayDeque

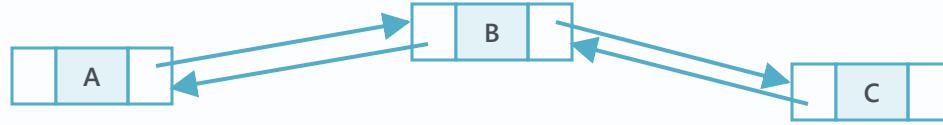
İki uçta sık ekleme-silme (kuyruk, çift uçlu kuyruk) gerekiyorsa.

Bellekte ArrayList ve LinkedList

ArrayList



LinkedList



her öge ayrı düğüm nesnesi: veri + önceki + sonraki; bellekte dağınık

ArrayList

Öge başına ~1 referans (4–8 bayt) + boş kapasite. Ardışık dizi: işlemci ön belleği (cache) sıradaki öğeleri önceden getirir.

LinkedList

Öge başına bir düğüm nesnesi: nesne başlığı + 3 referans (~24–40 bayt). Her **sonraki** izlemesi ön bellek ıskası olabilir.

Vector

- ArrayList gibi büyüyeabilen bir dizidir; öğelere indisle erişilir, ihtiyaca göre büyür (varsayılan: kapasite **iki katına** çıkar).
- Metotları `synchronized` 'dır: tek tek işlemler çok iş parçacıklı ortamda güvenlidir.
- Bedeli: tek iş parçacığında bile her çağrıda kilit maliyeti.
- "Kontrol et, sonra yap" gibi **bileşik** işlemler yine de atomik değildir.

```
Vector<Integer> v = new Vector<>();
v.add(10);           // [10]
v.add(20);           // [10, 20]
v.get(0);            // 10
v.remove(0);         // [20]
v.size();            // 1
v.capacity();        // 10

// bileşik işlem: dışarıdan kilit gerekir
synchronized (v) {
    if (!v.isEmpty()) v.remove(0);
}
```

Vector Java 1.0'dan kalma eski (legacy) bir sınıftır. Yeni kodda tek iş parçacığı için `ArrayList`; çok iş parçacığı için `Collections.synchronizedList` ya da `CopyOnWriteArrayList` tercih edilir.

Vector: "Kontrol Et, Sonra Yap" Yarışı

Vector'ın her metodu `synchronized` : iki çağrının **arası** ise korunmasızdır. İki iş parçacığının olası bir sıralanışı:



Başlangıç: $v = [20]$. İki iş parçacığı da `if (!v.isEmpty()) v.remove(0);` çalıştıracak.

Vector: "Kontrol Et, Sonra Yap" Yarıışı

Vector'ın her metodu `synchronized` : iki çağrının **arası** ise korunmasızdır. İki iş parçacığının olası bir sıralanışı:



T1: `isEmpty()` kilidi alır, false döner, kilidi **bırakır**.

Vector: "Kontrol Et, Sonra Yap" Yarıışı

Vector'ın her metodu `synchronized` : iki çağrının **arası** ise korunmasızdır. İki iş parçacığının olası bir sıralanışı:



T2 araya girer: v hâlâ 1 öğeli, o da false görür. İkisi de silebileceğini sanıyor.

Vector: "Kontrol Et, Sonra Yap" Yarıışı

Vector'ın her metodu `synchronized` : iki çağrının **arası** ise korunmasızdır. İki iş parçacığının olası bir sıralanışı:

İş parçacığı T1

`isEmpty()` → false

`remove(0)` → 20

Vector v



size = 0

İş parçacığı T2

`isEmpty()` → false

T1: `remove(0)` → 20. v artık boş.

Adım 4 / 7

Vector: "Kontrol Et, Sonra Yap" Yarıışı

Vector'ın her metodu `synchronized` : iki çağrının **arası** ise korunmasızdır. İki iş parçacığının olası bir sıralanışı:

İş parçacığı T1

`isEmpty()` → false
`remove(0)` → 20

Vector v

boş

size = 0

İş parçacığı T2

`isEmpty()` → false
`remove(0)` → HATA

T2: `remove(0)` boş vektörde → `ArrayIndexOutOfBoundsException` . Her metot tek tek senkronize, ama **ikisi birlikte** atomik değil.

Vector: "Kontrol Et, Sonra Yap" Yarıışı

Vector'ın her metodu `synchronized` : iki çağrının **arası** ise korunmasızdır. İki iş parçacığının olası bir sıralanışı:

İş parçacığı T1

```
synchronized (v) {  
  isEmpty → false  
  remove(0) → 20 }
```

Vector v



size = 0

kilit T1'de: T2 bloğa giremez

İş parçacığı T2

```
synchronized (v) {  
bekler...
```

Çözüm: bileşik işlemi `synchronized (v) { ... }` ile tek kilide al. T2, T1 bloğu bitirene kadar bekler.

Vector: "Kontrol Et, Sonra Yap" Yarıışı

Vector'ın her metodu `synchronized` : iki çağrının **arası** ise korunmasızdır. İki iş parçacığının olası bir sıralanışı:

İş parçacığı T1

```
synchronized (v) {  
  isEmpty → false  
  remove(0) → 20 }
```

Vector v



size = 0

İş parçacığı T2

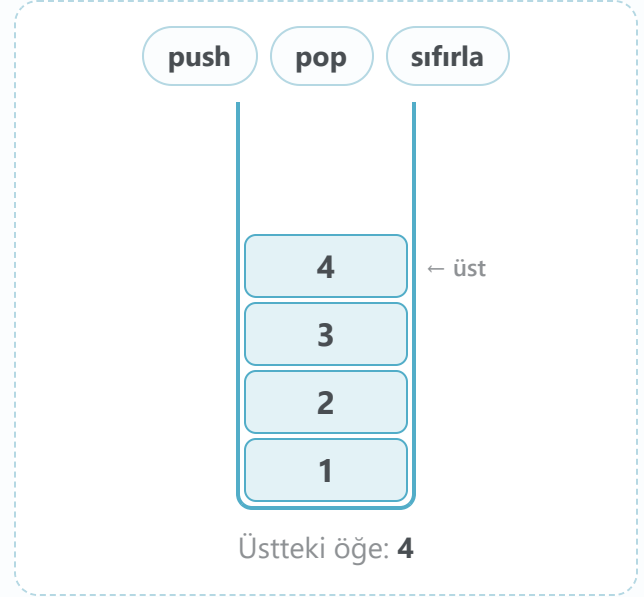
```
synchronized (v) {  
  isEmpty → true  
  } (silmez)
```

T2 kilidi alınca v'yi boş görür ve silmez: "kontrol et, sonra yap" artık güvenli.

Stack Sınıfı

- Vector'ı genişletir; **son giren ilk çıkar** (LIFO) ilkesiyle çalışan yığını uygular.
- İşlemler yığının **üstündeki** öğeyi etkiler; geri al / yinele (undo/redo) için kullanışlıdır.

<code>push(e)</code>	üste ekler
<code>pop()</code>	üsttekini çıkarır ve döndürür
<code>peek()</code>	üsttekini döndürür, çıkarmaz
<code>empty()</code>	yığın boş mu?
<code>search(o)</code>	üstten 1'den başlayan uzaklık; yoksa -1

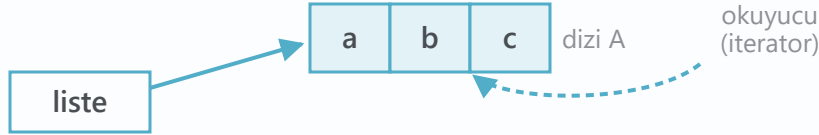


Boş yığında `pop` / `peek` → `EmptyStackException` .

Stack, Vector'dan türediği için `get(i)` , `add(i, e)` gibi LIFO'yu bozan metotları da taşır ve senkronizasyon yükü vardır. Java belgeleri yığın için `Deque<E> s = new ArrayDeque<>();` önerir.

CopyOnWriteArrayList: Yazarken Kopyala

- Değişiklik (add, set, remove) yapılacağında iç dizi **kopyalanır**, işlem kopyada yapılır, sonra liste yeni diziyi gösterir.
- Okumalar kilitsizdir ve çok hızlıdır; yazmalar kısa bir kilitle sıraya girer ve **O(n)** kopya maliyeti taşır.



Okuyucular iç diziyi kilitsiz okur; her okuyucu, oluşturulduğu andaki diziyi (anlık görüntü) gezer.

Adım 1 / 3

Uygun

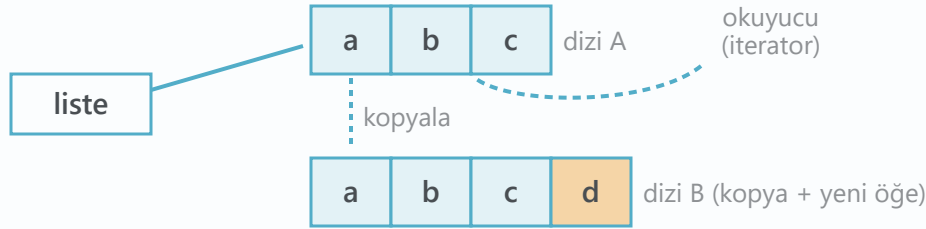
Sık gezilen, nadiren değişen listeler: olay dinleyicileri (listener), yapılandırma.

Uygun değil

Büyük ve sık değişen listeler: her yazma tüm diziyi kopyalar. `it.remove()` desteklenmez.

CopyOnWriteArrayList: Yazarken Kopyala

- Değişiklik (add, set, remove) yapılacağında iç dizi **kopyalanır**, işlem kopyada yapılır, sonra liste yeni diziyi gösterir.
- Okumalar kilitsizdir ve çok hızlıdır; yazmalar kısa bir kilitle sıraya girer ve **O(n)** kopya maliyeti taşır.



`add("d")` : yazar kilidi alır, diziyi **kopyalar** ve yeni öğeyi kopyaya ekler → O(n)

Adım 2 / 3

Uygun

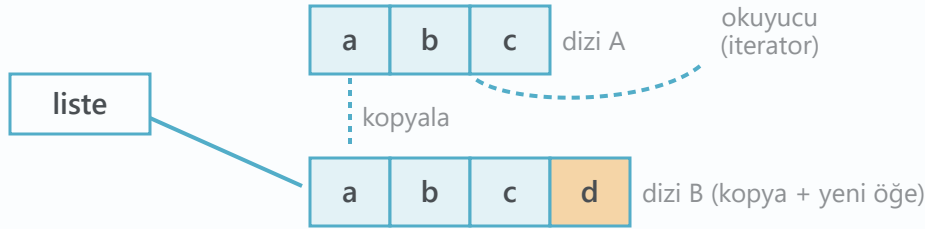
Sık gezilen, nadiren değişen listeler: olay dinleyicileri (listener), yapılandırma.

Uygun değil

Büyük ve sık değişen listeler: her yazma tüm diziyi kopyalar. `it.remove()` desteklenmez.

CopyOnWriteArrayList: Yazarken Kopyala

- Değişiklik (add, set, remove) yapılacağında iç dizi **kopyalanır**, işlem kopyada yapılır, sonra liste yeni diziyi gösterir.
- Okumalar kilitsizdir ve çok hızlıdır; yazmalar kısa bir kilitle sıraya girer ve **O(n)** kopya maliyeti taşır.



Liste referansı yeni diziyeye döner. Eski okuyucu A'yı gezmeye devam eder:

`ConcurrentModificationException` olmaz.

Adım 3 / 3

Uygun

Sık gezilen, nadiren değişen listeler: olay dinleyicileri (listener), yapılandırma.

Uygun değil

Büyük ve sık değişen listeler: her yazma tüm diziyi kopyalar. `it.remove()` desteklenmez.

Kod İzleme: CopyOnWriteArrayList add(e)



Liste A = [a, b, c]'yi gösteriyor. Bir okuyucu (iterator) A üzerinde gezinmiş, "a"yı okumuş (cursor 1). Şimdi `add("d")`.

Adım 1 / 10

```
▶ public boolean add(E e) {  
  2   synchronized (kilit) {  
  3     Object[] eski = dizi;  
  4     int n = eski.length;  
  5     Object[] yeni = Arrays.copyOf(eski, n + 1);  
  6     yeni[n] = e;  
  7     dizi = yeni;          // tek referans ataması  
  8     return true;  
  9   }  
 10 }
```

DEĞİŞKENLER

e = -

eski = -

n = -

yeni = -

`liste.dizi = A`

ÇAĞRI YIĞINI

`main()`

Kod İzleme: CopyOnWriteArrayList add(e)

kilit: yazarda



Yazarlar kilidi alır: aynı anda iki yazar birbirinin kopyasını ezemez. **Okuyucular kilit beklemez.**

Adım 2 / 10

```
1 public boolean add(E e) {  
2     synchronized (kilit) {  
3         Object[] eski = dizi;  
4         int n = eski.length;  
5         Object[] yeni = Arrays.copyOf(eski, n + 1);  
6         yeni[n] = e;  
7         dizi = yeni;        // tek referans ataması  
8         return true;  
9     }  
10 }
```

DEĞİŞKENLER

e = d

eski = -

n = -

yeni = -

liste.dizi = A

ÇAĞRI YİĞİNİ

add("d")

main()

Kod İzleme: CopyOnWriteArrayList add(e)

kilit: yazarda



eski = dizi: o anki dizi A.

Adım 3 / 10

```
1 public boolean add(E e) {  
2     synchronized (kilit) {  
3         Object[] eski = dizi;  
4         int n = eski.length;  
5         Object[] yeni = Arrays.copyOf(eski, n + 1);  
6         yeni[n] = e;  
7         dizi = yeni;        // tek referans ataması  
8         return true;  
9     }  
10 }
```

DEĞİŞKENLER

e = d

eski = A

n = -

yeni = -

liste.dizi = A

ÇAĞRI YIĞINI

add("d")

main()

Kod İzleme: CopyOnWriteArrayList add(e)

kilit: yazarda



n = 3.

Adım 4 / 10

```
1 public boolean add(E e) {  
2     synchronized (kilit) {  
3         Object[] eski = dizi;  
4         int n = eski.length;  
5         Object[] yeni = Arrays.copyOf(eski, n + 1);  
6         yeni[n] = e;  
7         dizi = yeni;        // tek referans ataması  
8         return true;  
9     }  
10 }
```

DEĞİŞKENLER

e = d

eski = A

n = 3

yeni = -

liste.dizi = A

ÇAĞRI YIĞINI

add("d")

main()

Kod İzleme: CopyOnWriteArrayList add(e)

kilit: yazarda



`Arrays.copyOf(eski, 4)` : tüm öğeler **yeni** diziye kopyalanır → O(n). A'ya dokunulmaz.

Adım 5 / 10

```

1 public boolean add(E e) {
2     synchronized (kilit) {
3         Object[] eski = dizi;
4         int n = eski.length;
5         ▶ Object[] yeni = Arrays.copyOf(eski, n + 1);
6         yeni[n] = e;
7         dizi = yeni;           // tek referans ataması
8         return true;
9     }
10 }

```

DEĞİŞKENLER

e = d

eski = A

n = 3

yeni = B

liste.dizi = A

ÇAĞRI YIĞINI

add("d")

main()

Kod İzleme: CopyOnWriteArrayList add(e)

kilit: yazarda



yeni[3] = "d" : değişiklik yalnızca kopyada yapılır.

Adım 6 / 10

```
1 public boolean add(E e) {  
2     synchronized (kilit) {  
3         Object[] eski = dizi;  
4         int n = eski.length;  
5         Object[] yeni = Arrays.copyOf(eski, n + 1);  
6         ► yeni[n] = e;  
7         dizi = yeni;        // tek referans ataması  
8         return true;  
9     }  
10 }
```

DEĞİŞKENLER

e = d

eski = A

n = 3

yeni = B

liste.dizi = A

ÇAĞRI YİĞİNİ

add("d")

main()

Kod İzleme: CopyOnWriteArrayList add(e)

kilit: yazarda

A

a	b	c
---	---	---

 ← okuyucu it (cursor 1)

B

a	b	c	d
---	---	---	---

 ← liste.dizi

`dizi = yeni` : tek bir referans ataması; liste artık B'yi gösterir. Okuyucu hâlâ A'da.

Adım 7 / 10

```
1 public boolean add(E e) {  
2     synchronized (kilit) {  
3         Object[] eski = dizi;  
4         int n = eski.length;  
5         Object[] yeni = Arrays.copyOf(eski, n + 1);  
6         yeni[n] = e;  
7         dizi = yeni; // tek referans ataması  
8         return true;  
9     }  
10 }
```

DEĞİŞKENLER

e = d

eski = A

n = 3

yeni = B

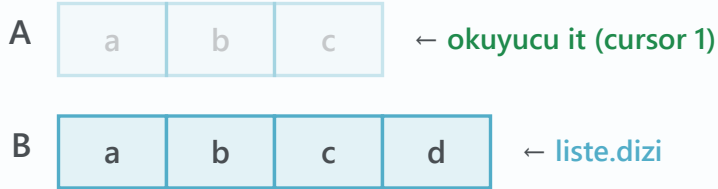
liste.dizi = B

ÇAĞRI YIĞINI

add("d")

main()

Kod İzleme: CopyOnWriteArrayList add(e)



true döner, kilit bırakılır.

Adım 8 / 10

```
1 public boolean add(E e) {  
2     synchronized (kilit) {  
3         Object[] eski = dizi;  
4         int n = eski.length;  
5         Object[] yeni = Arrays.copyOf(eski, n + 1);  
6         yeni[n] = e;  
7         dizi = yeni;        // tek referans ataması  
8         return true;  
9     }  
10 }
```

DEĞİŞKENLER

e = d

eski = A

n = 3

yeni = B

liste.dizi = B

ÇAĞRI YİĞİNİ

add("d")

main()

Kod İzleme: CopyOnWriteArrayList add(e)

A

a	b	c
---	---	---

 ← okuyucu it (cursor 3)

B

a	b	c	d
---	---	---	---

 ← liste.dizi

Okuyucu `it.next()` → "b", sonra "c": **anlık görüntüsünü** (A) gezer, "d"yi görmez, `ConcurrentModificationException` da olmaz.

Adım 9 / 10

```
1 public boolean add(E e) {  
2     synchronized (kilit) {  
3         Object[] eski = dizi;  
4         int n = eski.length;  
5         Object[] yeni = Arrays.copyOf(eski, n + 1);  
6         yeni[n] = e;  
7         dizi = yeni;          // tek referans ataması  
8         return true;  
9     }  
10 }
```

DEĞİŞKENLER

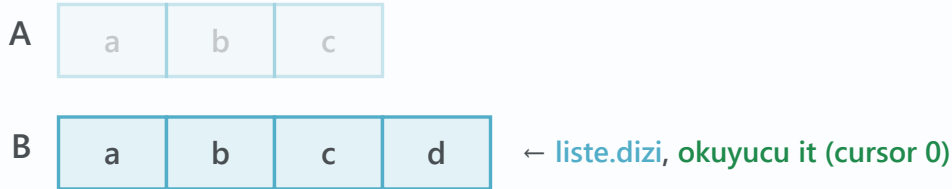
liste.dizi = B

okuyucu = A

ÇAĞRI YIĞINI

main()

Kod İzleme: CopyOnWriteArrayList add(e)



Yeni bir `iterator()` B'yi görür: [a, b, c, d]. Eski okuyucu bitince A'ya referans kalmaz; çöp toplayıcı temizler.

Adım 10 / 10

```
1 public boolean add(E e) {  
2     synchronized (kilit) {  
3         Object[] eski = dizi;  
4         int n = eski.length;  
5         Object[] yeni = Arrays.copyOf(eski, n + 1);  
6         yeni[n] = e;  
7         dizi = yeni;           // tek referans ataması  
8         return true;  
9     }  
10 }
```

DEĞİŞKENLER

`liste.dizi = B`

`okuyucu = A`

ÇAĞRI YIĞINI

`main()`

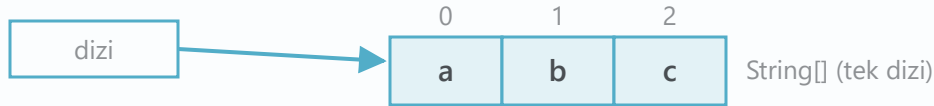
Arrays.asList(): Diziden Liste Görünümü

- Verilen diziye `List` arayüzüyle kullanılabilir kılar; veri **kopyalanmaz**, dizi ile liste aynı belleği paylaşır.
- Liste **sabit boyutludur** ama **değiştirilemez değildir**: `set` çalışır (ve diziye yansır), `add` / `remove` çalışmaz.

```
String[] dizi = {"a", "b", "c"};
List<String> liste = Arrays.asList(dizi);
liste.set(0, "x");      // çalışır: dizi[0] de artık "x"
dizi[2] = "z";          // liste de değişir → [x, b, z]
liste.add("d");          // UnsupportedOperationException (boyut sabit)
List<String> esnek = new ArrayList<>(Arrays.asList(dizi)); // bağımsız kopya
```

	set	add / remove	Diziyle bağlantı
<code>Arrays.asList(dizi)</code>	✓	X	paylaşır
<code>List.of(...)</code>	X	X	yok (değiştirilemez)
<code>new ArrayList<>(...)</code>	✓	✓	yok (kopya)

Kod İzleme: `Arrays.asList` Görünümü



3 ögeli dizi.

Adım 1 / 8

```
► String[] dizi = {"a", "b", "c"};
2 List<String> liste = Arrays.asList(dizi);
3 liste.set(0, "x");
4 dizi[2] = "z";
5 // liste.add("d"); → hata (sabit boyut)
6 List<String> esnek = new ArrayList<>(liste);
7 esnek.add("d");
8 System.out.println(liste + " " + esnek);
```

DEĞİŞKENLER

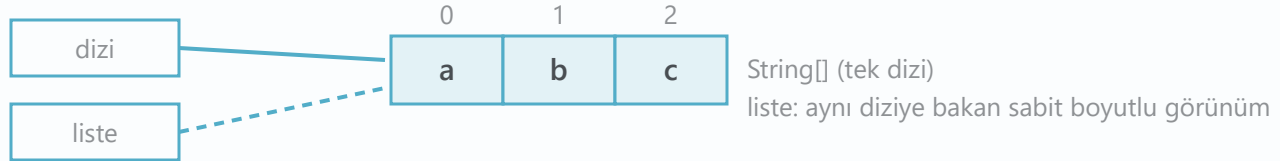
`dizi = [a, b, c]`

`liste = -`

`esnek = -`

ÇIKTI

Kod İzleme: `Arrays.asList` Görünümü



`Arrays.asList` kopyalamaz: dönen liste **aynı diziye** saran bir görünümdür → $O(1)$.

Adım 2 / 8

```
1 String[] dizi = {"a", "b", "c"};  
2 ► List<String> liste = Arrays.asList(dizi);  
3 liste.set(0, "x");  
4 dizi[2] = "z";  
5 // liste.add("d"); → hata (sabit boyut)  
6 List<String> esnek = new ArrayList<>(liste);  
7 esnek.add("d");  
8 System.out.println(liste + " " + esnek);
```

DEĞİŞKENLER

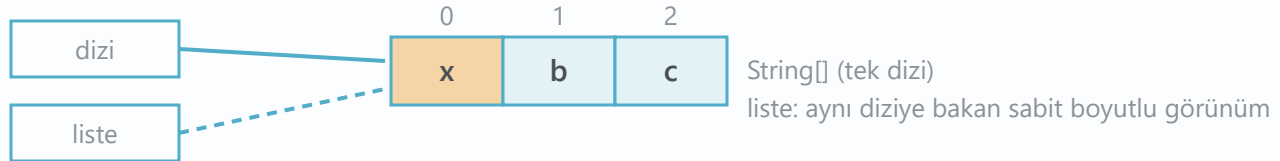
dizi = [a, b, c]

liste = [a, b, c]

esnek = -

ÇIKTI

Kod İzleme: `Arrays.asList` Görünümü



`liste.set(0, "x")` diziye yazar: `dizi[0]` da x.

Adım 3 / 8

```
1 String[] dizi = {"a", "b", "c"};
2 List<String> liste = Arrays.asList(dizi);
3 ► liste.set(0, "x");
4 dizi[2] = "z";
5 // liste.add("d"); → hata (sabit boyut)
6 List<String> esnek = new ArrayList<>(liste);
7 esnek.add("d");
8 System.out.println(liste + " " + esnek);
```

DEĞİŞKENLER

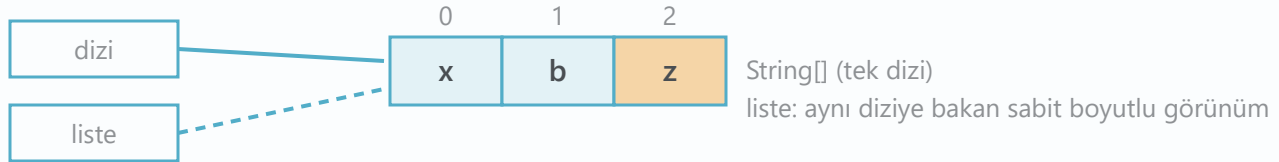
dizi = [x, b, c]

liste = [x, b, c]

esnek = -

ÇIKTI

Kod İzleme: `Arrays.asList` Görünümü



Ters yön de geçerli: diziye yazmak listeyi değiştirir.

Adım 4 / 8

```
1 String[] dizi = {"a", "b", "c"};
2 List<String> liste = Arrays.asList(dizi);
3 liste.set(0, "x");
4 ► dizi[2] = "z";
5 // liste.add("d"); → hata (sabit boyut)
6 List<String> esnek = new ArrayList<>(liste);
7 esnek.add("d");
8 System.out.println(liste + " " + esnek);
```

DEĞİŞKENLER

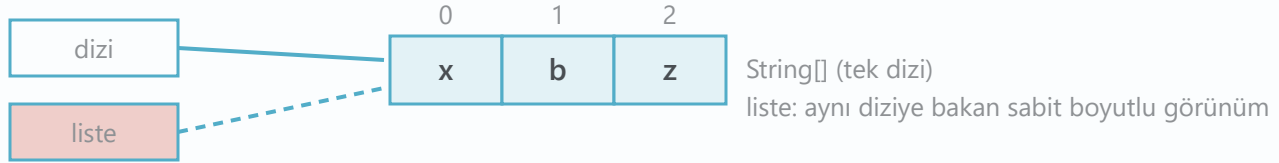
dizi = [x, b, z]

liste = [x, b, z]

esnek = -

ÇIKTI

Kod İzleme: `Arrays.asList` Görünümü



Yorum açılsaydı: dizinin boyu değişemeyeceğinden `add` / `remove` → `UnsupportedOperationException`.

Adım 5 / 8

```
1 String[] dizi = {"a", "b", "c"};
2 List<String> liste = Arrays.asList(dizi);
3 liste.set(0, "x");
4 dizi[2] = "z";
5 // liste.add("d"); → hata (sabit boyut)
6 List<String> esnek = new ArrayList<>(liste);
7 esnek.add("d");
8 System.out.println(liste + " " + esnek);
```

DEĞİŞKENLER

dizi = [x, b, z]

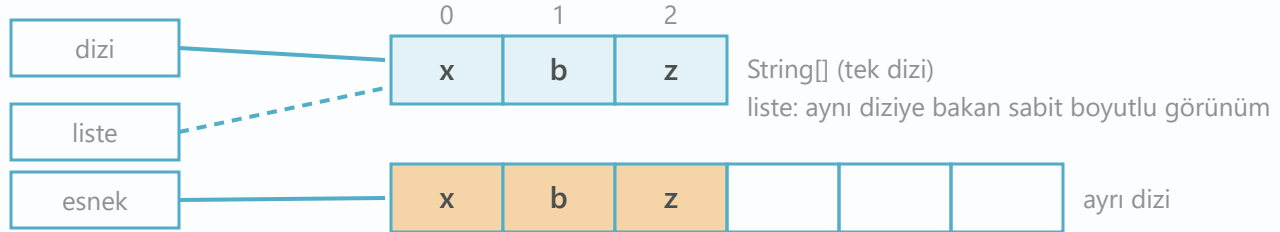
liste = [x, b, z]

esnek = -

ÇIKTI

[x, b, z] [x, b, z, d]

Kod İzleme: `Arrays.asList` Görünümü



Bağımsız ve büyüyeabilen liste için kopyala: `new ArrayList<>(liste)` yeni dizi ayırır.

Adım 6 / 8

```
1 String[] dizi = {"a", "b", "c"};
2 List<String> liste = Arrays.asList(dizi);
3 liste.set(0, "x");
4 dizi[2] = "z";
5 // liste.add("d"); → hata (sabit boyut)
▶ List<String> esnek = new ArrayList<>(liste);
7 esnek.add("d");
8 System.out.println(liste + " " + esnek);
```

DEĞİŞKENLER

`dizi = [x, b, z]`

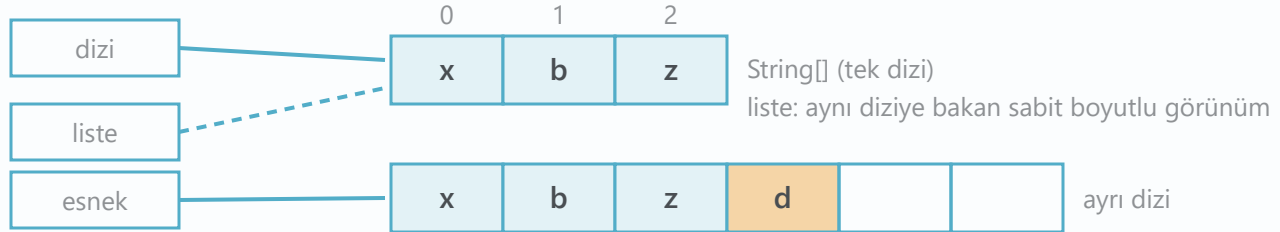
`liste = [x, b, z]`

`esnek = [x, b, z]`

ÇIKTI

[x, b, z] [x, b, z, d]

Kod İzleme: `Arrays.asList` Görünümü



Kopyaya ekleme serbest; `dizi` ve `liste` etkilenmez.

Adım 7 / 8

```
1 String[] dizi = {"a", "b", "c"};
2 List<String> liste = Arrays.asList(dizi);
3 liste.set(0, "x");
4 dizi[2] = "z";
5 // liste.add("d"); → hata (sabit boyut)
6 List<String> esnek = new ArrayList<>(liste);
7 ► esnek.add("d");
8 System.out.println(liste + " " + esnek);
```

DEĞİŞKENLER

`dizi = [x, b, z]`

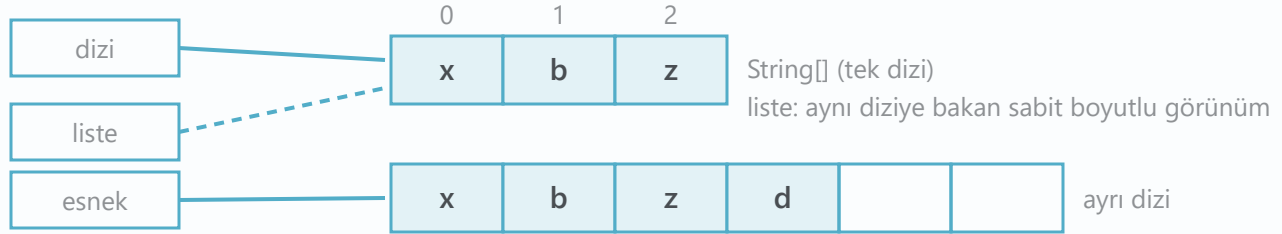
`liste = [x, b, z]`

`esnek = [x, b, z, d]`

ÇIKTI

[x, b, z] [x, b, z, d]

Kod İzleme: Arrays.asList Görünümü



Sonuç iki ayrı liste.

Adım 8 / 8

```
1 String[] dizi = {"a", "b", "c"};
2 List<String> liste = Arrays.asList(dizi);
3 liste.set(0, "x");
4 dizi[2] = "z";
5 // liste.add("d"); → hata (sabit boyut)
6 List<String> esnek = new ArrayList<>(liste);
7 esnek.add("d");
▶ System.out.println(liste + " " + esnek);
```

DEĞİŞKENLER

`dizi = [x, b, z]`

`liste = [x, b, z]`

`esnek = [x, b, z, d]`

ÇIKTI

`[x, b, z] [x, b, z, d]`

Özet: Liste Uygulamaları

Sınıf	get(i)	sona ekleme	başa / araya	İş parçacığı
ArrayList	$O(1)$	amortize $O(1)$	$O(n)$	güvenli değil
LinkedList	$O(n)$	$O(1)$	$O(1) / O(n)$	güvenli değil
Vector	$O(1)$	amortize $O(1)$	$O(n)$	senkronize
Stack	$O(1)$	push $O(1)$	—	senkronize
CopyOnWriteArrayList	$O(1)$	$O(n)$	$O(n)$	güvenli (kopyalayarak)
Arrays.asList	$O(1)$	X	X	güvenli değil

Gezinme

for-each / Iterator her listede $O(n)$. Gezerken silmek için `it.remove()` veya `removeIf`.

Etkileşimli simülatör

Liste ADT işlemlerini adım adım deneyin: [Liste ADT Simülatörü](#).

Örnek Problemler

1. `List<Integer> l = new ArrayList<>(List.of(5, 6, 7)); l.remove(1);` sonrası liste nedir?
2. Bir `LinkedList` 'i `for (int i...) get(i)` ile gezmenin karmaşıklığı nedir? Daha iyisi?
3. Kapasitesi 10 ve 10 ögesi olan `ArrayList`'e bir öge eklenince ne olur?
4. `Arrays.asList(dizi).add(x)` neden hata verir? `set` neden vermez?
5. Sık okunan, nadiren değişen ve birçok iş parçacığının paylaştığı bir dinleyici listesi için hangi sınıf?
6. [A, B, C] üzerinde `it = listIterator(); it.next(); it.add("X");` sonrası liste ve `it.next()` ?

► Cevapları göster

Alıştırmalar

1. `l = [A, B, C, D]` ; `l.subList(1, 3).clear();` sonrası `l` ? Sonra `l.add("E")` yapıp eski görünüm okunursa?
2. Kapasitesi her dolduğunda **2 artırılan** dinamik diziye n ekleme toplam kaç kopya yapar?
3. `Aralik.next()` , `k ≥ son` denetimini yapmasaydı hangi sözleşme bozulurdu?
4. Konumsal listede `remove(p)` neden $O(1)$, `LinkedList.remove(int)` neden $O(n)$?
5. `l = new ArrayList<>(List.of(3, 1, 2)); l.remove(Integer.valueOf(1)); l.remove(1);`
→ `l` ?

► Cevaplar / ipuçları

Kaynaklar

- Oracle, Java SE 17 `Collection` , `List` , `Iterator` , `ListIterator` ve `Spliterator` API belgeleri.
- Bloch, *Effective Java*, 3. baskı, 2018.
- Goodrich, Tamassia ve Goldwasser, *Data Structures and Algorithms in Java*, 6. baskı, 2014.

Son

Bölüm 6: Liste

Sonraki sunum: [Bölüm 7: Ağaçlar](#) · [Çalışma soruları](#) · Simülatör: [List ADT Simulator](#) · [Terim sözlüğü](#)